



ADDIS ABABA UNIVERSITY
COLLEGE OF NATURAL AND COMPUTATIONAL
SCIENCES
DEPARTMENT OF COMPUTER SCIENCE
CHARITY MANAGEMENT SYSTEM
FINAL YEAR PROJECT I

NAME OF THE STUDENTS	IDNO
Abel Getachew	UGR/6211/15
Dagmawi Heywot	UGR/4392/15
Geleta Tamiru	UGR/2035/15
Nathnael Lule	UGR/1003/15

PROJECT ADVISOR: Mr. Samuel.G

Date: January 2026

DECLARATION

This is to declare that this project work which is done under the supervision of Mr Samuel Getachew and having the title Charity Management System is the sole contribution of:

Abel Getachew

Dagmawi Heywot

Geleta Tamiru

Nathnael Lule

No part of the project work has been reproduced illegally (copy and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date: _____

Group Members:

Full Name

Signature

Abel Getachew

Dagmawi Heywot

Geleta Tamiru

Nathnael Lule

CERTIFICATE

I certify that this BSc final project report entitled Charity Management System by:

Abel Getachew

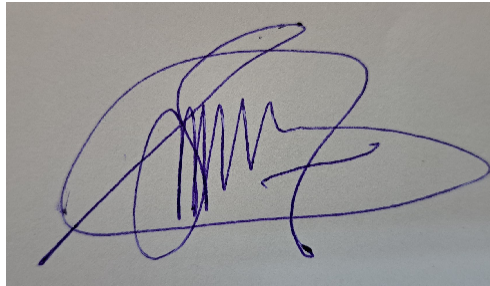
Dagmawi Heywot

Geleta Tamriru

Nathnael Lule

is approved by me for submission. I certify further that, to the best of my knowledge, the report represents work carried out by the students.

Mr. Samuel Getachew

A handwritten signature in blue ink, consisting of several loops and a central vertical stroke, is placed over the signature line.

January 28 , 2026

Date

Name and Signature of Supervisor

ACKNOWLEDGEMENT

First and above all, we thank **Almighty God** for His guidance, strength, wisdom, and grace throughout the completion of this project. Without His help, this work would not have been possible.

We would like to express our sincere gratitude to our project advisor, **Mr. Samuel Getachew**, for his continuous guidance, constructive feedback, and valuable support during the development of this project. His academic insight and encouragement greatly contributed to the success of this work.

We also extend our appreciation to the **Department of Computer Science** for providing the necessary academic support and environment to carry out this project.

Finally, we are grateful to our families, friends, and classmates for their encouragement, cooperation, and moral support throughout the project period.

TABLE OF CONTENTS

[DECLARATION](#)

[CERTIFICATE](#)

[ACKNOWLEDGEMENT](#)

[TABLE OF CONTENTS](#)

[LIST OF TABLES](#)

[LIST OF FIGURES](#)

[ACRONYMS](#)

[CHAPTER ONE](#)

[1.1 Introduction](#)

[1.2 Statement of the Problem and Justification](#)

[1.3 Project Objective](#)

[1.3.1 General Objective](#)

[1.3.2 Specific Objectives](#)

[To achieve the general objective, the project will focus on the following specific objectives:](#)

[1.4 Scope and Limitation of the Project](#)

[1.4.1 Scope of the project](#)

[1.4.2 Limitations of the Project](#)

[1.5 System Development Methodology](#)

[1.5.1. Investigation \(Fact-Finding\) Methods](#)

[To gather accurate and relevant information for the development of the charity management system, multiple fact-finding techniques were used:](#)

[1.5.2. System Development Tools](#)

[1.5.2.1 Front-End Development Tools](#)

[1.5.2.2 Back-End Development Tools](#)

[1.5.2.3 Database Development Tools](#)

[1.5.2.4 Mobile Application Development Tools](#)

[1.5.2.5 Other Development Tools](#)

[1.6 Significance of the Project](#)

[1.7 Beneficiaries](#)

[1.8 Time Schedule](#)

[CHAPTER TWO](#)

[2.1 Introduction](#)

[2.2 Current System](#)

[2.2.1 Major Functions of the Current System / Current System Description](#)

[2.2.2 Problems of the Existing System](#)

[2.3 Requirement Gathering](#)

[2.3.1 Requirement Gathering Methodologies](#)

[2.3.2 Results of Requirement Gathering](#)

[2.4 Proposed System](#)

[2.4.1 Overview](#)

2.4.2	Functional Requirements
2.4.3	Non-Functional Requirements
2.4.4	Constraints / Pseudo Requirements
2.5	System Model (Overview)
2.5.1	Scenario
2.5.2	Use Case Model
2.5.2.1	Use Case Diagram
2.5.2.2	Description of Use-Case Model
2.5.3	Object Model
2.5.3.1	Data Dictionary
2.5.3.2	Class Modeling
2.5.3.3	Dynamic Modeling
2.5.3.4	User Interfaces
CHAPTER 3	
SYSTEM DESIGN	
3.1	Introduction
3.2	Current Software Architecture
3.3	Proposed Software Architecture
3.3.1	Overview
3.3.2	Subsystem Decomposition
3.3.2.1	User Management Subsystem
3.3.2.2	Campaign Management Subsystem
3.3.2.3	Donation Management Subsystem
3.3.2.4	Notification Subsystem
3.3.2.5	Administration Subsystem
3.3.2.6	Reporting Subsystem
3.3.3	Hardware/Software Mapping
3.3.4	Persistent Data Management
3.3.5	Access Control and Security
3.4	Detailed Class Diagram
3.5	Packages
3.5.1	Dependency of Packages
4. IMPLEMENTATION	
4.1	ScreenShots
4.2	Code Snippets
5. TESTING	
5.1	Introduction
5.1.1	Purpose
5.2	Features to be Tested / Not to be Tested
5.2.1	Features to be Tested
	Authentication and User Management
	Campaign Management

<u>Campaign Discovery</u>	
<u>Donation Management</u>	
<u>Charity Verification</u>	
<u>Administrative Functions</u>	
<u>Validation and Security</u>	
<u>5.2.2 Features Not to be Tested</u>	
<u>Third-Party Services</u>	
<u>Performance and Load Testing</u>	
<u>Infrastructure Testing</u>	
<u>Security Penetration Testing</u>	
<u>Browser and Device Compatibility Testing</u>	
<u>5.3. Pass/Fail Criteria</u>	
<u>5.3.1 Pass Criteria</u>	
<u>5.3.2 Fail Criteria</u>	
<u>5.3.3 Test Execution Result</u>	
<u>5.4. Approach/Strategy</u>	
<u>5.4.1 Testing Approach</u>	
<u>Functional Testing</u>	
<u>Integration Testing</u>	
<u>Validation Testing</u>	
<u>Security and Authorization Testing</u>	
<u>5.5 Testing Tools</u>	
<u>5.6 Metrics Collected</u>	
<u>5.7 Configuration Management</u>	
<u>5.8 Test Environment</u>	
<u>Hardware Configuration</u>	
<u>Software Configuration</u>	
<u>5.9 Regression Testing</u>	
<u>5.10. Test cases with specifications</u>	
<u>REFERENCES</u>	

LIST OF TABLES

Table 1.1 Front-End Development Tools	4
Table 1.2 Back-End Development Tools	5
Table 1.3 Database Development Tools	6
Table 1.4 Database Development Tools	6
Table 1.5 Other Development Tools	6
Table 2.1 Summary of Requirement Gathering Survey Results.....	15
Table 2.2 User Login Scenario.....	23
Table 2.3 Search Campaigns Scenario	23
Table 2.4 View Campaign Details Scenario.....	23
Table 2.5 Make Donation Scenario.....	24
Table 2.6 Create Campaign Scenario.....	25
Table 2.7 Verify Charity Registration Scenario	26
Table 2.8 Login Use Case Description	28
Table 2.9 Manage User Profile Use Case Description	29
Table 2.10 View Active Campaigns Use Case Description	29
Table 2.11 Search Campaigns Use Case Description	30
Table 2.12 Filter Campaigns Use Case Description	30
Table 2.13 View Campaign Details Use Case Description	31
Table 2.14 Receive Notifications Use Case Description	31
Table 2.15 Make Donation Use Case Description	32
Table 2.16 View Donation History Use Case Description	33
Table 2.17 Download Donation Receipt Use Case Description	34
Table 2.18 Upload Charity Registration Document Use Case Description	35
Table 2.19 Upload Campaign Document Use Case Description	36

Table 2.20 Create Campaign Use Case Description	37
Table 2.21 Edit Campaigns Use Case Description	38
Table 2.22 Close Campaign Use Case Description	39
Table 2.23 View Donation Summary Use Case Description	40
Table 2.24 Verify Charity Registration Use Case Description	41
Table 2.25 Review Donation Logs Use Case Description	42
Table 2.26 Review Financial Transactions Use Case Description	43
Table 2.27 Follow Campaign Use Case Description.....	44
Table 2.28: User	45
Table 2.29: CharityOrganization	46
Table 2.30 Campaign.....	46
Table 2.31 Donation.....	47
Table 2.32: DonationReceipt.....	47
Table 2.33 CampaignDocument	48
Table 2.34: CampaignFollow.....	48
Table 2.35 Notification.....	48
Table 2.36 ReviewLog.....	49
Table 2.37 CharityVerification.....	49
Table 2.38 PasswordReset.....	50
Table 3.1 Core Data Objects.....	77
Table 3.2 Access Control.....	79
Table 5.1: Testing Tools	115
Table 5.2: Test Case Specification for Login.....	116
Table 5.3: Test Case Specification for Reset Password	118
Table 5.4: Test Case Specification for Register	119
Table 5.5: Test Case Specification for Create Campaign	120
Table 5.6: Test Case Specification for Edit Campaign	121

Table 5.7: Test Case Specification for Close Campaign	122
Table 5.8: Test Case Specification for View Campaign Details	123
Table 5.9: Test Case Specification for Search Campaign	124
Table 5.10: Test Case Specification for Filter Campaign	125
Table 5.11: Test Case Specification for Follow Campaign.....	126
Table 5.12: Test Case Specification for Make Donation	127
Table 5.13: Test Case Specification for View Donation Summary	128
Table 5.14: Test Case Specification for Verify Charity Organization	129
Table 5.15: Test Case Specification for Review Donation Logs	130
Table 5.16: Test Case Specification for Donation History	131

LIST OF FIGURES

Figure 1.1 Gantt Chart.....	9
Figure 2.1 Donor preferences regarding anonymity.....	16
Figure 2.2 Survey Results: Major Issues in Existing Charity Workflow.....	17
Figure 2.3 Use case diagram.....	27
Figure 2.4 Class diagram.....	51
Figure 2.5 Login Sequence diagram with reset password.....	52
Figure 2.6 Search and filter campaigns sequence diagram.....	53
Figure 2.7 Create campaign sequence diagram.....	54
Figure 2.8 View campaign details sequence diagram.....	55
Figure 2.9 Make donation sequence diagram.....	56
Figure 2.10 Close campaign sequence diagram.....	57
Figure 2.11 View donation history and download receipt sequence diagram.....	58

Figure 2.12 View donation summary sequence diagram.....	59
Figure 2.13 Verify Charity registrations sequence diagram.....	60
Figure 2.14 Login activity diagram.....	61
Figure 2.15 View Active Campaigns Activity Diagram.....	62
Figure 2.16 Search Campaigns Activity Diagram	62
Figure 2.17 View Campaign Details Activity Diagram.....	63
Figure 2.18 Make Donation Activity Diagram.....	64
Figure 2.19 Create Campaign Activity Diagram.....	65
Figure 2.20 Close Campaign Activity Diagram.....	66.
Figure 2.21 Verify Charity Registration Activity Diagram.....	67
Figure 2.22 Users landing page UI.....	68
Figure 2.23 Users login page UI.....	69
Figure 3.1 Layered Software Architecture.....	72
Figure 3.2 Subsystem Decomposition	75
Figure 3.3 Deployment Diagram.....	76
Figure 3.4 Donor Side Class Diagram.....	80
Figure 3.5 Charity Organization Side Class Diagram.....	81
Figure 3.6 Administrator Side Class Diagram.....	82
Figure 3.7 Package Diagram.....	83
Figure 3.8 Package Dependency Diagram.....	83
Figure 4.1 Landing page	87
Figure 4.2 Login page	88
Figure 4.3 Charity Registration	88
Figure 4.4 Donor Dashboard	89
Figure 4.5 Campaign details page	89

Figure 4.6 Chapa Payment	90
Figure 4.7 Charity Dashboard.....	90
Figure 4.8 Contributions page for charity organizations	91
Figure 4.9 Admin Dashboard	91
Figure 4.10 Reports Page(Admin)	92
Figure 4.11 Admin side charity verification page	92
Figure 4.12 Admin side user management page.....	93

ACRONYMS

FR : Functional Requirement

NFR : Non-Functional Requirement

PR : Pseudo Requirement

UI : User Interface

API : Application Programming Interface

CSS : Cascading Style Sheets

ODM : Object Data Modeling

ID : Identifier

UC : Use Case

SC : Scenario

HTTPS : Hypertext Transfer Protocol Secure

SSL : Secure Sockets Layer

WCAG : Web Content Accessibility Guidelines

DB : Database

NoSQL : Not Only Structured Query Language

JSON : JavaScript Object Notation

IDE : Integrated Development Environment

URL : Uniform Resource Locator

API : Application Programming Interface

RBAC : Role-Based Access Control

UI/UX : User Interface / User Experience

IT : Information Technology

CHAPTER ONE

PROJECT PROPOSAL

1.1 Introduction

Charity organizations play an important role in supporting vulnerable communities, yet many of them in Ethiopia still depend on outdated and mostly manual methods to manage donations, record financial transactions, and communicate with donors or beneficiaries. These practices often involve paper forms, scattered spreadsheets, and unstructured updates through phone calls or social media. As a result, important information may be lost, misrecorded, or delayed, and there is no consistent way to monitor how much a campaign has collected or how the collected resources were distributed.

A charity management system introduces a structured and organized way to handle these operations. By providing a web-based platform, charities can present their campaigns more clearly, donors can give with confidence, and administrators can oversee the entire process with proper records.

The system supports features such as campaign creation, donation tracking, donor anonymity, and beneficiary recording. It serves as a digital environment where all participants, donors, charity staff, and administrators interact through consistent and verifiable information. This project focuses on designing and developing such a system to minimize inefficiencies, improve transparency, and strengthen the overall trust surrounding charitable activities.

In general, this project aims to simplify and organize how charitable organizations manage their activities. By providing a single platform for donation tracking and campaign management it reduces errors, saves time, and ensures information is clear and reliable. The system helps charities operate more efficiently and transparently, allowing them to focus on supporting those in need.

1.2 Statement of the Problem and Justification

In many local charity settings, the absence of a unified management system creates significant operational challenges. Donation records are often maintained on paper or stored informally, making them prone to loss, damage, or inconsistencies. Donors rarely receive updates on how their contributions are utilized, and campaign organizers spend

considerable time manually calculating collected amounts, tracking remaining needs, and preparing reports for supervisors or supporters. The scattered and unverifiable nature of this information can lead to donor hesitation, as concerns about misuse or unclear fund distribution persist.

Traditional manual processes in these charity organizations are therefore inefficient, time-consuming, and prone to errors. This not only affects the trust and engagement of donors but also limits the ability of charities to manage campaigns effectively and transparently.

Introducing a centralized online charity management system addresses these challenges by providing a stable, auditable, and accessible digital platform. Such a system can record every donation, specify its purpose, and link it to a particular campaign, reducing errors and misunderstandings. Donors gain confidence through transparent progress updates, while charities benefit from decreased administrative burden and improved efficiency. Features such as anonymous donations further enhance inclusivity and participation.

In summary, the problem lies in the fragmented and unreliable methods currently used to manage charitable activities, underscoring the need for a web-based system that ensures transparency, accountability, and efficient management, ultimately fostering greater donor trust and sustainable charitable engagement.

1.3 Project Objective

1.3.1 General Objective

The general objective of this project is to design and develop a web-based and mobile app charity management system.

1.3.2 Specific Objectives

To achieve the general objective, the project will focus on the following specific objectives:

- Conduct data collection to understand the requirements of the system.
- Analyze the collected data to identify key features and functionalities.
- Design the proposed system architecture.

- Identify appropriate tools and technologies for the design and implementation of the system.
- Implement the system according to the design specifications.
- Test and evaluate the system to ensure it functions correctly and meets project requirements.

1.4 Scope and Limitation of the Project

1.4.1 Scope of the project

- The system provides an online platform that facilitates interaction between donors, charity organizations, and administrators through a web interface.
- It enables charity organizations to register, manage campaigns, record donations, update progress, and track beneficiaries.
- Donors are able to browse campaigns, make contributions, choose anonymity, and receive digital confirmations.
- Administrators are given tools to verify charity organizations, monitor activities, and generate basic reports.
- All information is stored in a centralized database designed to maintain consistency and improve traceability.
- The platform focuses on offering a clear and structured flow of activities that replaces traditional paper-based or informal record-keeping methods.

1.4.2 Limitations of the Project

- It does not implement automated verification methods such as ID checking or fraud detection, relying instead on manual oversight..
- It does not attempt to cover legal or tax-related procedures associated with charity operations.
- Advanced features such as multi-currency handling, or predictive analytics fall outside the intended scope of this version.

1.5 System Development Methodology

The Agile Software Development Methodology is used for this project because it supports iterative development, flexibility, and continuous feedback. It allows requirements to be refined gradually and supports documentation and implementation.

1.5.1. Investigation (Fact-Finding) Methods

To gather accurate and relevant information for the development of the charity management system, multiple fact-finding techniques were used:

- **Interview** : Discussions were held with people familiar with charity activities, including charity organizers, donation coordinators, and potential donors. These interviews helped identify challenges such as lack of transparency, difficulty tracking funds, and poor communication between donors and beneficiaries.
- **Document Review** : Existing donation forms, financial logs, and social media postings were reviewed. This helped identify gaps in manual record-keeping.
- **Survey**: A survey was conducted with potential donors and charity workers to gather insights on donation preferences, transparency, anonymity, and campaign tracking. The results helped identify challenges and informed the design of the charity management system.

1.5.2. System Development Tools

1.5.2.1 Front-End Development Tools

Tool	Version	Purpose
React	19.2.4	Used to build the application's user interface, manage state, and enable dynamic user interactions using a component-based architecture.

Tailwind CSS	4.1.18	Used for styling the user interface with utility-first CSS, enabling responsive design and consistent styling without writing custom CSS.
axios	1.13.6	Used for making HTTP requests to backend APIs (GET, POST, PUT, DELETE).
recharts	2.4.0	Used to build charts like bar charts, pie charts, and dashboards for data visualization.
jspdf	4.2.1	Generates PDF files directly from the frontend (reports, receipts, certificates).

Table 1.1 Front-End Development Tools

1.5.2.2 Back-End Development Tools

Tool Name	Version	Purpose
Node.js	25.5.0	Serves as the JavaScript runtime environment for executing server-side code and handling scalable, event-driven operations.
Express.js	5.2.1	Provides a lightweight web framework for handling routing, middleware, server logic, and API endpoint management.
jsonwebtoken	9.0.2	Generates and verifies JWT authentication tokens

bcrypt	6.0.0	Password hashing for secure storage
--------	-------	-------------------------------------

Table 1.2 Back-End Development Tools

1.5.2.3 Database Development Tools

Tool Name	Version	Purpose
mysql2	3.20.0	MySQL database driver for connection handling
Prisma	6.16.3	ORM for database modeling, queries, and migrations

Table 1.3 Database Development Tools

1.5.2.4 Mobile Application Development Tools

Tool Name	Version	Purpose
Flutter	SDK	Cross-platform mobile application development for Android and iOS
Dart	SDK 3.12.0	Primary language used for Flutter application development

Table 1.4 Database Development Tools

1.5.2.5 Other Development Tools

Category	Tool Name	Version	Purpose
----------	-----------	---------	---------

Design & Modeling	Figma	Latest(Web-based)	Design UI, wireframes, mockups, and user flow prototypes
Development Environment	Visual Studio Code	Latest	Source code editing and project management
Testing Tool	Postman	Web-based	Testing backend API endpoints
Documentation Tool	Google Docs	Web-based	Writing, editing, and collaborative documentation
Version Control	GitHub	Web-based	Source code management and version control

Table 1.5 Other Development Tools

1.6 Significance of the Project

Societal Benefits:

- Promotes transparency by allowing donors to track how their contributions are utilized, building trust between donors and charity organizations.
- Improves accountability, enabling charities to maintain accurate records and report progress efficiently.
- Ensures beneficiaries receive organized and verified support, helping aid reach those who need it most.
- Encourages donations by providing easy online access to donation platforms.
- Enhances privacy through anonymous donation options, allowing donors to contribute comfortably.

Technological Benefits:

- Facilitates digital transformation of charity operations, replacing manual record-keeping with an automated online system.
- Improves data management by reducing errors and duplication through centralized storage.
- Enhances security by safeguarding sensitive information via authentication and secure transaction mechanisms.
- Provides scalability to accommodate a growing number of users, campaigns, and functionalities.
- Offers real-time tracking for donors and administrators, ensuring timely monitoring of campaign progress.
- Makes charitable activities more efficient, transparent, and impactful through technology.

1.7 Beneficiaries

Charity Organizations

- Efficiently manage donation campaigns.
- Track contributions and generate detailed reports.
- Communicate effectively with donors and beneficiaries.

Donors

- View available campaigns and make secure donations.
- Choose to donate anonymously.
- Monitor past contributions and track the impact of their support.

Government

- Supervision of charity organization by the government body will be easier since it can find multiple charity organization data in one place.
- Can help in reviewing campaign activities in order to prove the legitimacy by seeing the revenue that campaign generated in a transparent way.

1.8 Time Schedule

Gantt Chart



Figure 1.1 Gantt Chart

CHAPTER TWO

REQUIREMENT ANALYSIS

2.1 Introduction

The requirement analysis section forms the conceptual foundation of the proposed charity management system by establishing a clear and structured understanding of what the system must accomplish, how it is expected to function, and the constraints under which it will operate.

This chapter plays a crucial role in the system development process as it ensures that the planned solution does not emerge solely from technical assumptions but directly responds to real-world challenges faced by charity organizations, donors, and administrative bodies. By systematically examining the shortcomings of the current charity management landscape, this analysis bridges the gap between user expectations and practical system design, thereby offering a coherent roadmap for implementation.

In many communities, charitable activities play an essential role in supporting vulnerable populations, funding social projects, and sustaining community welfare initiatives. However, the operational environment within which these activities take place is often fragmented, heavily manual, and lacking the transparency necessary to inspire donor confidence.

Campaign announcements are dispersed across informal channels, donation tracking lacks consistency, and reporting mechanisms rely on handwritten notes or improvised documentation. These inefficiencies not only slow down the workflow but also create opportunities for miscommunication, mistrust, and data loss.

The proposed charity management system aims to address these challenges by digitizing the core functions involved in organizing and overseeing charitable campaigns. Designed as a web-based platform, the system will provide a centralized environment where donors can easily access verified campaigns, make secure contributions, and optionally remain anonymous. Charity organizations will be equipped with structured tools for creating campaigns, logging donations, presenting progress updates, and preparing financial reports. Administrators will oversee platform activities through an integrated control

panel that enhances accountability and ensures the legitimacy of participating organizations.

This requirement analysis identifies both the functional components of what the system must do and the non-functional elements how well the system must perform. It ensures that each proposed feature aligns with the broader objectives of transparency, efficiency, and trust-building within the charitable ecosystem.

2.2 Current System

Despite the increasing societal need for transparent, efficient, and accountable donation management, the operational environment remains largely informal and fragmented. Most charitable initiatives operate using manual, paper-based processes or ad hoc digital tools that lack centralization, accuracy, and long-term reporting capabilities.

The analysis below outlines the key operational workflows in the current environment, identifies the main stakeholders engaged in charity work, and highlights the tools and communication channels commonly relied upon. By examining the existing system in depth, this section clarifies the inefficiencies and risks that necessitate a structured digital platform.

2.2.1 Major Functions of the Current System / Current System Description

1. Campaign Announcement and Promotion

Most charities communicate new campaigns using informal channels such as printed posters, social media posts, word of mouth, or messaging apps. There is no centralized digital dashboard where donors can browse ongoing activities, verify legitimacy, or track previous events. As a result, campaign visibility depends heavily on human networks and inconsistent outreach.

2. Donation Collection and Recording

Donations are typically collected in person through cash handover, bank transfers, or mobile money payments shared directly with organizers. These transactions are documented manually through: handwritten notes, simple receipt books, personal notebooks of volunteers, scattered Excel sheets saved individually.

Because there is no consolidated database, donor records are often incomplete, duplicated, or misplaced.

3. Report Preparation and Accountability

Financial reports and summary documents are prepared manually by compiling receipts, tallying contributions, and summarizing activities. This process is slow, error-prone, and highly dependent on individual volunteers. Stakeholders, including donors, rarely receive consistent or verifiable updates.

Typical Workflow in the Current System

1. A campaign is announced informally through posters, messages, or social media.
2. Donors contact organizers and send cash or transfers manually.
3. Organizers record donations on paper or spreadsheets.
- 4 Funds or goods are delivered in person.
- 5 Organizers compile handwritten notes to produce a basic report.

This workflow lacks automation, security, and long-term storage.

2.2.2 Problems of the Existing System

1. Lack of Transparency and Trust

Manual record-keeping creates opportunities for errors, loss of information, and even suspicion of mismanagement. Donors have no way to verify whether funds were used as intended.

2. Fragmented and Inconsistent Data

Donation records are scattered across paper notebooks, screenshots, and spreadsheets. This fragmentation prevents efficient tracking of donor history or campaign performance.

3. Time-Consuming Reporting

Creating financial reports requires manually checking receipts and summarizing totals. This leads to delays and increases the risk of mistakes.

4. High Risk of Data Loss

Paper receipts can be damaged or lost; personal phones containing screenshots can break or be formatted. There is no reliable backup system.

5. No Centralized Communication

Important updates are shared across multiple channels, making coordination difficult and sometimes leading to misunderstandings.

6. Limited Donor Engagement

Donors contribute once but rarely receive structured updates, progress trackers, or impact reports that would encourage repeat donations.

7. Inefficient Workflow for Organizers

Volunteers spend significant time: rewriting donation lists, sorting receipts, manually contacting donors, updating spreadsheets.

This reduces efficiency and increases stress during high-volume campaigns.

2.3 Requirement Gathering

The development of the charity management system requires a structured and thorough understanding of the operational needs, existing challenges, and expectations of the individuals and groups involved in managing donation-related activities. The requirement-gathering process examined current charity workflows, observed how information is recorded and communicated, and analyzed practices commonly used by local charity initiatives. Since most existing processes rely on manual, non-standardized tools, this phase focused on identifying functional gaps and determining essential features needed to build an organized, transparent, and user-friendly digital platform.

2.3.1 Requirement Gathering Methodologies

Requirement-gathering methodologies for the charity management system utilized a combination of research and stakeholder oriented techniques to ensure a clear and comprehensive understanding of user needs.

Online Research Across Social Media and the Web

Research was conducted on social media platforms, charity websites, and online fundraising pages to understand how local organizations present campaigns, manage

donations, and interact with donors in a digital environment. This helped reveal common practices, user expectations, and challenges such as campaign visibility, donor feedback, and confirmation mechanisms. Insights gathered highlighted essential features such as donation tracking, campaign transparency, and customizable donor anonymity.

Review of Existing Documents and Records

Existing physical and digital materials including receipt books, handwritten donor lists, Excel sheets, and social media announcements were analyzed to understand what information is currently collected and how it is maintained. This review exposed inconsistencies in data formats, missing donor details, irregular reporting, and the absence of standardized templates for tracking campaign goals, collected amounts, and remaining balances.

Analysis of Common System Patterns

A brief analysis of common digital charity and fundraising platforms was performed to identify standard system patterns that could inform the design of the proposed solution. Features such as structured campaign listings, donor dashboards, secure donation logging modules, and administrative monitoring tools were particularly relevant for shaping core components of the system.

Stakeholder Expectation Analysis (Informal Discussions)

Informal discussions with people involved in local charitable activities provided valuable context about user expectations. Stakeholders emphasized the importance of clearer donation confirmations, improved campaign visibility, options for donor anonymity, and the secure storage of financial information. These insights offered a user-centered perspective that complemented findings from online observations and document analysis.

Requirements Categories:

- **Functional:** Specific actions the system must perform, such as campaign creation, donation logging, and donor management.
- **Non-functional:** System qualities including security, usability, performance, reliability, and data consistency.

2.3.2 Results of Requirement Gathering

The requirement-gathering activities revealed several major gaps in the current charity workflow, which directly shaped the functional and non-functional requirements of the proposed system.

Issue Identified	Description	Percentage of Respondents (%)
Lack of centralized record-keeping	Financial data is maintained using handwritten notes or scattered digital formats, making verification and reporting difficult	82%
Inefficient administrative oversight	Difficulty in supervising charity operations and generating consolidated financial summaries	79%
Absence of structured campaign visibility	Donors lack clear and consistent information on active campaigns and their progress	76%
Need for donor anonymity	Donors prefer privacy when making contributions and avoiding public disclosure	48%

Table 2.1 Summary of Requirement Gathering Survey Results

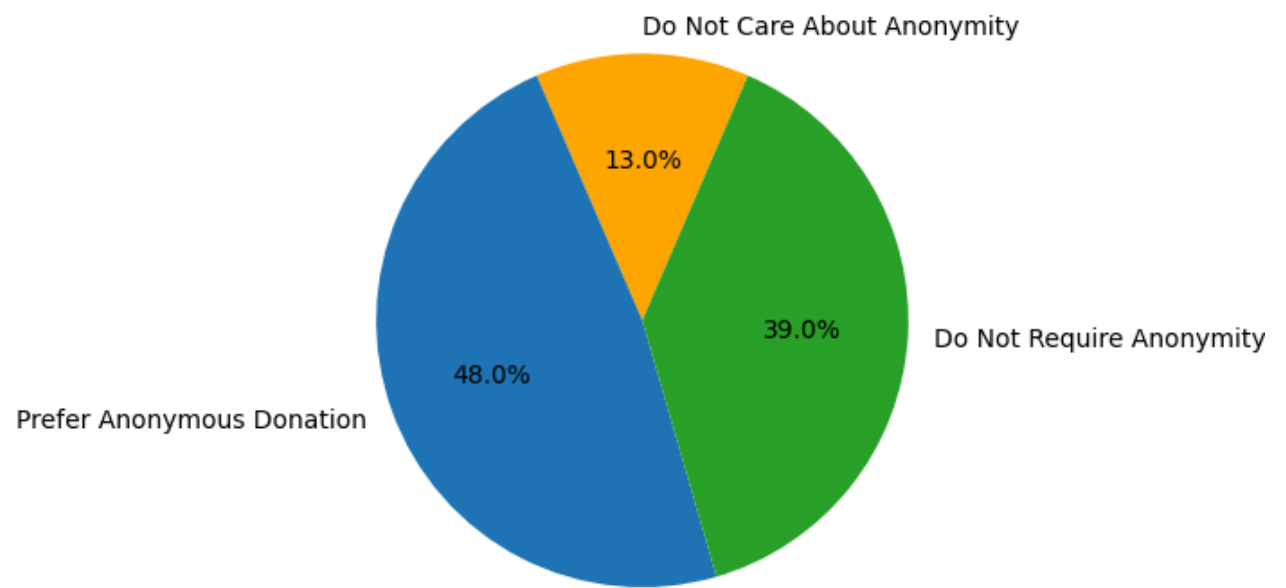


Figure 2.1 Donor preferences regarding anonymity

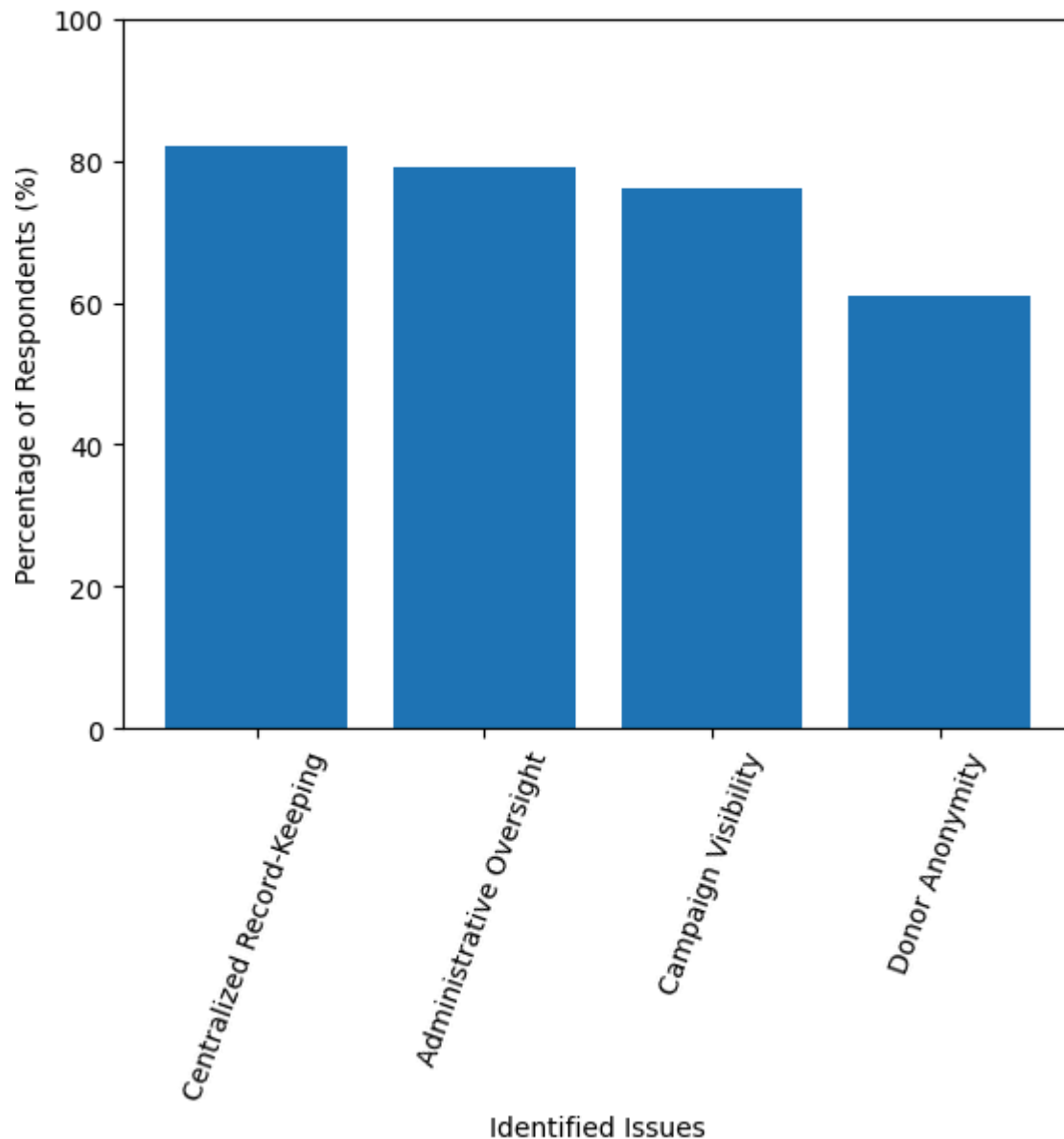


Figure 2.2 Survey Results: Major Issues in Existing Charity Workflow

2.4 Proposed System

2.4.1 Overview

The proposed charity management system is an integrated, web based platform designed to simplify, centralize, and digitize all charity-related operations. It connects donors, charity organizations, and administrators through a secure, transparent, and user-friendly interface. The system supports creating and managing campaigns, processing donations, tracking beneficiary allocations, and ensuring accountability through clear and accessible

records. By replacing the traditional manual process with automated workflows, real time updates, role-based access control, and data-driven reporting, the platform functions as a centralized hub where verified charities can present campaigns, donors can contribute securely and optionally anonymously, and administrators can supervise all activities.

2.4.2 Functional Requirements

A. Common Functional Requirements

FR-1: User Registration

The system must allow users of roles donor and charity to securely register, log in, log out, and reset passwords.

FR-2: Search Campaigns

The system shall allow users to search for charity campaigns using keywords such as campaign title, charity name, or campaign description, enabling users to quickly locate specific campaigns that match their interests or intended causes.

FR-3: Filter Campaigns

The system shall allow users to filter available charity campaigns based on predefined criteria such as category, location, and campaign status (e.g., active or closed), helping users narrow down search results and easily identify campaigns that meet their preferences

FR-4: Notifications

The system must send email or in-app notifications for key events such as donation confirmations, campaign approvals, and system updates. This keeps users informed and ensures they receive timely updates relevant to their activities.

B. Donor Functional Requirements

FR-5: View Active Campaigns

The donor must be able to browse all approved and active campaigns. Campaign details including description, goal amount, progress, and charity information are visible to help donors make informed decisions.

FR-6: Make Donations

Donors must be able to donate securely through integrated payment gateways. The system must support optional anonymity, ensuring that donor identity is hidden from the charity when chosen.

FR-7: View Donation History

Donors must have access to a complete history of their donations, including timestamps, amounts, and associated campaigns. They must also be able to download official donation receipts for recordkeeping or tax purposes.

FR-8: Follow up Campaigns

Donors can choose to follow specific campaigns to receive updates such as progress notifications, closure announcements, or milestone achievements. This encourages ongoing engagement with causes they care about.

C. Charity Organization Functional Requirements**FR-9: Create Campaign**

Charity organizations must be able to submit new campaigns for admin approval. Campaign details include title, description, target amount, documents, and optional media. Submission triggers a workflow requiring administrative review.

FR-10: Edit Campaign

Charities can update certain campaign details, such as goal amount or campaign title. Every modification must be logged to maintain transparency and track changes over time.

FR-11: Close Campaign

Charities can close or complete campaigns once goals are met or when they choose to discontinue them. Closing stops new donations and updates the campaign's visibility status.

FR-12: View Donation Summary

Charities must be able to view summarized donation data related to their campaigns. This includes the total amount raised, number of donors, and basic information needed for reporting and internal planning.

FR-13: Receive Notifications

Charity organizations receive alerts for key events such as campaign approvals, new donations, or messages from the administrator. This ensures they remain updated and can manage their campaigns effectively.

D. Administrator Functional Requirements**FR-14: User Management**

Administrators must review and approve charity registrations, manage all user accounts, and handle issues such as disputes or policy violations. They hold the highest level of system privileges to maintain platform integrity.

FR-15: Donation Log Review

Admins must be able to view and review donation logs for compliance and transparency. This includes the ability to inspect transaction histories, and confirm donation legitimacy.

2.4.3 Non-Functional Requirements**NFR-1. Security**

All user data must be fully protected from unauthorized access and cyber threats. Sensitive information including passwords, donor identities, and payment details must be encrypted both in transit using HTTPS/SSL and at rest. Role-based access control should ensure that only authorized users can access restricted areas, and secure authentication methods such as strong password policies and token-based sessions must be enforced. Additionally, the system must be safeguarded against common cybersecurity attacks to maintain overall protection.

NFR-2. Data Integrity

All stored data must remain accurate, consistent, and trustworthy. Unauthorized modifications to donation logs, campaign details, or beneficiary information must be prevented through strict controls. Validation rules must ensure that all entered data is correct and complete, while audit logs should record all important changes along with the responsible user and timestamp. Transaction management must prevent partial or corrupted entries, particularly during payment processes, to preserve data accuracy.

NFR-3. Reliability

The system must operate smoothly with minimal downtime or disruptions. It should remain functional even during sudden spikes in traffic or partial system failures. Regular backups must be performed to prevent data loss, and automatic recovery mechanisms such as failover systems should restore service quickly when issues occur. Critical operations like donations must never fail silently; errors should be logged, monitored, and retried whenever possible to ensure reliability.

NFR-4. Usability

The platform must be intuitive and easy to use for all users, regardless of their technical background. Interfaces should remain clean, simple, and consistent across all pages, while navigation must be straightforward enough to allow users to donate or create campaigns with minimal steps. Clear instructions, tooltips, and feedback messages should guide

users throughout the process, and the UI must support accessibility needs through basic WCAG compliance, including readable fonts, proper contrast, and keyboard navigation.

NFR-5. Performance

The system must provide high responsiveness and fast loading times across all pages and user actions. Pages should load quickly even during peak donation periods with heavy traffic, and optimized database queries must be used to avoid delays when retrieving campaign or donation data. Caching frequently accessed information should reduce repeated processing, and APIs must respond within acceptable time limits to ensure a smooth and seamless user experience.

NFR-6. Scalability

The architecture must support growing numbers of users, campaigns, and donation volumes. The system should scale without performance degradation as more charities and donors join the platform. Both horizontal scaling (adding servers) and vertical scaling (upgrading server capacity) should be possible, and the database must be able to handle increasing records efficiently as the platform expands.

NFR-7. Maintainability

The system should be easy to update, debug, and extend over time. Code should follow modular, clean architecture principles so that new features can be added without breaking existing functionality. Developers must be able to understand and modify each module with minimal effort, supported by proper documentation such as API references, comments, and coding standards. Automated testing should also be in place to detect issues early and ensure stable deployments.

NFR-8. Compatibility

The platform must function correctly across different devices and technologies. It should work smoothly on all major browsers including Chrome, Firefox, Safari, and Edge, and the UI must adapt effectively to various screen sizes such as laptops, tablets, and mobile phones.

2.4.4 Constraints / Pseudo Requirements

PR-1: Verified Charities

The system shall restrict participation to only registered and verified charity organizations. This constraint requires the platform to enforce a manual verification and approval process, where charities must submit legal and identification documents that are reviewed by system administrators before activation. As a result, charity onboarding cannot be fully automated and depends on external validation procedures, which may delay availability and limit rapid scaling.

PR-2: Legal and Regulatory Compliance

The system must comply with applicable data protection, financial, and charitable regulations within the operating region. This constraint limits how user and donation data can be collected, stored, processed, and retained. For example, personal data may require explicit user consent, sensitive information must be encrypted, and data retention periods may be legally fixed. The system cannot implement features that conflict with these regulatory requirements, even if they are technically feasible.

2.5 System Model (Overview)

The system model of the charity management system describes how different users interact with the system to achieve their goals. It provides a structured representation of the system's functional behavior from the user's perspective. This model focuses on identifying key actors, their interactions, and the services offered by the system.

The system model is represented using **scenarios and use case models**, which help illustrate real-world usage of the system in a clear and understandable manner. Scenarios explain specific sequences of actions performed by users, while use case models present the overall functionality and relationships between actors and the system. Together, these models help ensure that system requirements are clearly understood and provide a foundation for system design and implementation.

2.5.1 Scenario

A scenario represents a specific instance of a use case, describing a concrete sequence of actions that fulfills a particular goal within the system. Scenarios offer detailed insights into how users interact with the system, highlighting key processes and their realizations.

Scenario ID	SC-01
-------------	-------

Scenario Name	User Login
Main Flow	1. The user navigates to the login page. 2. The user enters their email and password. 3. The system verifies the credentials. 4. Upon successful authentication, the system grants access and redirects the user to the appropriate dashboard.

Table 2.2 User Login Scenario

Scenario ID	SC-02
Scenario Name	Search Campaigns
Main Flow	1. The user navigates to the campaign listing or search page. 2. The user enters keywords or selects search criteria. 3. The system searches the campaign database based on the given input. 4. The system displays matching campaign results to the user.

Table 2.3 Search Campaigns Scenario

Scenario ID	SC-03
Scenario Name	View Campaign Details

Main Flow	1. The user selects a campaign from the campaign list or search results. 2. The system retrieves the campaign details. 3. The system displays the full campaign information to the user.
-----------	--

Table 2.4 View Campaign Details Scenario

Scenario ID	SC-04
Scenario Name	Make Donation
Main Flow	1. The donor accesses the system. 2. The donor browses or searches for active campaigns. 3. The donor selects a campaign to support. 4. The system displays detailed campaign information. 5. The donor enters the donation amount. 6. The donor chooses whether to donate anonymously or provide basic information. 7. The donor selects a payment method. 8. The system securely processes the payment. 9. The system records the donation and updates the campaign progress. 10. The system displays a donation confirmation message and sends a notification if contact details were provided

Table 2.5 Make Donation Scenario

Scenario ID	SC-05
Scenario Name	Create Campaign
Main Flow	1. The charity organization logs into the system. 2. The charity selects the option to create a new campaign. 3. The system displays the campaign creation form. 4. The charity enters campaign details such as title, description, target amount, category, and duration. 5. The charity uploads required supporting documents and optional media files. 6. The charity submits the campaign for approval. 7. The system validates the submitted information. 8. The system stores the campaign with a pending status. 9. The administrator is notified about the new campaign submission.

Table 2.6 Create Campaign Scenario

Scenario ID	SC-06
Scenario Name	Verify Charity Registration

Main Flow	1. Administrator navigates to the pending charity registrations list. 2. Administrator selects a charity registration to review. 3. Administrator verifies submitted details, documents, and compliance requirements. 4. The administrator approves or rejects the registration. 5. The system updates the charity's account status and notifies the charity organization.
-----------	--

Table 2.7 Verify Charity Registration Scenario

2.5.2 Use Case Model

A use-case model is a model of how different types of users interact with the system. Use cases describe the typical interactions between the users of a system and the system itself, providing a brief narrative of how a system is used.

2.5.2.1 Use Case Diagram

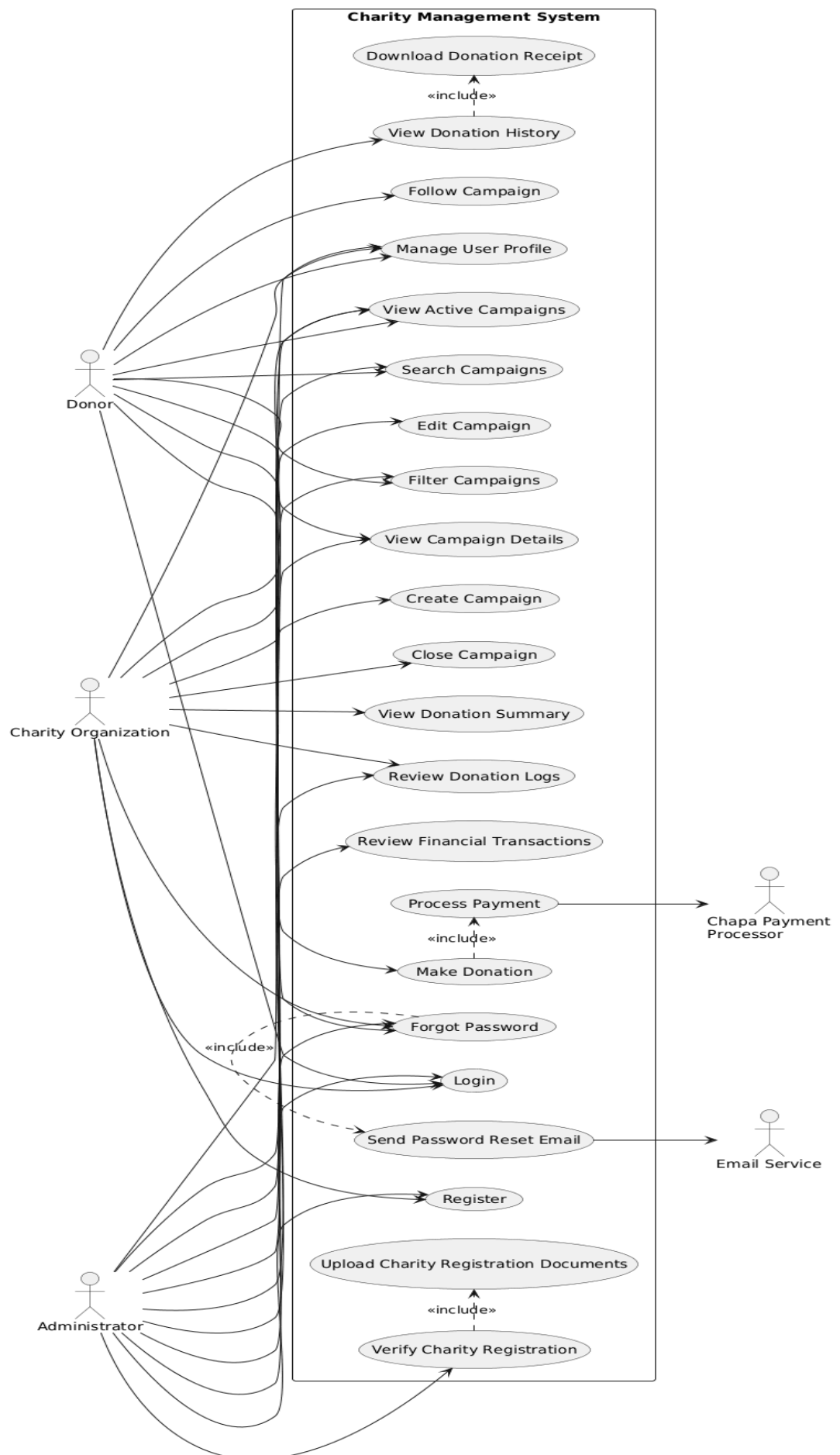


Figure 2.3 Use case diagram

2.5.2.2 Description of Use-Case Model

Use Case ID	UC-01
Use Case Name	Login
Participating Actors	Donor, Charity Organization, Administrator
Use Case Description	Allows registered users to authenticate into the system using valid credentials. If the user forgets their password, the system provides a password reset option as part of the login process.
Precondition	The user is registered in the system and has a valid email and password.
Flow of Events	1. The user navigates to the login page. 2. The user enters email and password. 3. The system validates the credentials. 4. If valid, the system grants access based on the user's role. 5. Users are redirected to their respective dashboard.
Alternate Flow (Reset Password)	1. The user selects the "Forgot Password" option. 2. The system prompts the user to enter their registered email. 3. The system sends a password reset link or code. 4. The user sets a new password. 5. The user returns to the login page and logs in successfully.

Postconditions	The user is successfully authenticated and gains access to authorized system features.
----------------	--

Table 2.8 Login Use Case Description

Use Case ID	UC-02
Use Case Name	Manage User Profile
Actor	Donor, Charity Organization, Administrator
Description	Allows a registered user to view and update their personal profile information.
Preconditions	User is logged in
Flow of Events	1. User accesses their profile page 2. System displays current profile information 3. User edits profile details (e.g., name, contact information, password) 4. User submits the changes 5. System validates and saves the updated information 6. System confirms successful update
Alternative Flows	Invalid or incomplete information entered - system displays error message User cancels profile update - no changes are saved
Postconditions	User profile information is updated successfully

Table 2.9 Manage User Profile Use Case Description

Use Case ID	UC-03
-------------	-------

Use Case Name	View Active Campaigns
Participating Actors	Donor, Charity Organization, Adminstrator
Use Case Description	Allows users to see all campaigns that are currently active and open for donations. This helps donors choose campaigns to contribute to and charities monitor ongoing campaigns.
Precondition	The system has active campaigns stored in the database. User may or may not be logged in.
Flow of Events	1. The user navigates to the campaign list. 2. The system retrieves all active campaigns. 3. The system displays active campaigns with details (title, description, goal, progress, and charity info). 4. The user selects a campaign for further actions (e.g., view details, donate).
Alternate Flow	3a. If there are no active campaigns, the system displays a “No active campaigns found” message.
Post Condition	Users are presented with a current list of campaigns open for donations, ready for further interaction.

Table 2.10 View Active Campaigns Use Case Description

Use Case ID	UC-04
Use Case Name	Search Campaigns
Participating Actors	Donor, Charity Organization, Administrator

Use Case Description	Allows users to search for charity campaigns based on keywords or basic criteria in order to find relevant campaigns.
Precondition	The user has access to the system (login is optional).
Flow of Events	1. The user navigates to the campaign listing or search page. 2. The user enters search keywords or criteria. 3. The system processes the search request. 4. The system displays a list of matching campaigns.
Alternate Flow	If no matching campaigns are found, the system displays a “No campaigns found” message.
Postconditions	The user receives a list of campaigns matching the search criteria. No changes are made to the system data.

Table 2.11 Search Campaigns Use Case Description

Use Case ID	UC-05
Use Case Name	Filter Campaigns
Participating Actors	Donor, Charity Organization, Administrator
Use Case Description	Allows users to narrow down the list of campaigns based on selected criteria such as category, location, or campaign status to find relevant campaigns efficiently.

Precondition	The system is available and a list of campaigns exists. Users may or may not be logged in.
Flow of Events	1. The user opens the campaign list. 2. The user selects one or more filter options (e.g., category, location, status). 3. The system applies the selected filters. 4. The system displays the filtered list of campaigns.
Alternate Flow	If no campaigns match the selected criteria, the system displays a “No campaigns found” message..
Post Condition	The user views a filtered list of campaigns based on the selected criteria.

Table 2.12 Filter Campaigns Use Case Description

Use Case ID	UC-06
Use Case Name	View Campaign Details
Participating Actors	Donor, Charity Organization, Administrator
Use Case Description	Allows users to view detailed information about a selected charity campaign, including description, goal amount, progress, and status.
Precondition	The campaign exists in the system.
Flow of Events	1. The user selects a campaign from the campaign list or search results. 2. The system retrieves the campaign details. 3. The system displays the full campaign information to the user.

Postconditions	The user has viewed the campaign details. No changes are made to the campaign data
----------------	---

Table 2.13 View Campaign Details Use Case Description

Use Case ID	UC-07
Use Case Name	Receive Notifications
Participating Actors	Donor, Charity Organization, Administrator
Use Case Description	Allows users to receive real-time alerts and messages from the system regarding important events such as donations, campaign updates, and system announcements.
Precondition	The user is registered in the system (optional: logged in for in-app notifications). Notifications are enabled in the system.
Flow of Events	1. An event occurs in the system (e.g., a donation is made, campaign status changes, or system update). 2. The system generates a notification. 3. The system delivers the notification to the relevant user(s) via email or in-app message. 4. The user views the notification.
Alternate Flow	3a. If the notification cannot be delivered (e.g., email failure), the system logs the event for later delivery or retries.
Post Condition	Users are informed about relevant events, enabling timely actions and awareness of system updates.

Table 2.14 Receive Notifications Use Case Description

Use Case ID	UC-08
Use Case Name	Make Donation
Actor	Donor (Registered User or Guest)
Description	Allows a donor to contribute funds to a selected charity campaign.
Preconditions	Campaign exists and is active
Flow of Events	1. Donor selects a campaign 2. Donor enters donation amount 3. Donor proceed to payment 4. Donor confirms the donation 5. System processes the payment 6. System records the donation 7. System displays donation confirmation
Alternative Flows	Donor selects anonymous donation Payment fails - system displays error and donation is not recorded Donor cancels donation before confirmation
Postconditions	Donation is stored Campaign total amount is updated

Table 2.15 Make Donation Use Case Description

Use Case ID	UC-9
Use Case Name	View Donation History
Actor	Donor (Registered User)

Description	Allows a donor to view their past donation records.
Preconditions	User is logged in
Flow of Events	1. User accesses donation history 2. System displays list of past donations
Alternative Flows	User downloads donation receipt No donation records - system displays message
Postconditions	Donation history is displayed Receipt may be downloaded

Table 2.16 View Donation History Use Case Description

Use Case ID	UC-10
Use Case Name	Download Donation Receipt
Participating Actors	Donor(Registered User)
Use Case Description	Allows donors to download official receipts for their donations. Receipts can be used for personal records or tax purposes.
Precondition	Donor has completed at least one donation and is logged in to the system. The system has recorded the transaction.
Flow of Events	1. Donor navigates to their donation history. 2. Donor selects a completed donation. 3. The system generates the official receipt in a downloadable format (e.g., PDF). 4. Donor downloads or saves the receipt.

Alternate Flow	3a. If the receipt cannot be generated due to a system error, the system displays an error message and logs the issue.
Post Condition	Donor successfully obtains the official receipt for recordkeeping or tax purposes.

Table 2.17 Download Donation Receipt Use Case Description

Use Case ID	UC-11
Use Case Name	Upload Charity Registration Document
Participating Actors	Charity Organization
Use Case Description	Allows a charity organization to upload required legal and registration documents during the registration process. These documents are reviewed by the administrator to verify the legitimacy of the charity before account approval.
Preconditions	- Charity organization is registering or updating registration details- User has access to the registration form
Flow of Events	1. Charity organization initiates registration or verification process 2. System displays document upload section 3. Charity uploads required registration documents (e.g., license, legal certificate) 4. System validates file type and size 5. System stores documents securely and marks registration as pending review
Alternate Flow	4a. Invalid or unsupported document format – system displays error message 3a. Charity skips upload – system prevents submission and displays warning

Postconditions	- Registration documents are stored in the system- Charity registration status is set to “Pending Approval”- Administrator is notified for verification
----------------	---

Table 2.18 Upload Charity Registration Document Use Case Description

Use Case ID	UC-12
Use Case Name	Upload Campaign Document
Participating Actors	Charity Organization
Use Case Description	Allows a charity organization to upload supporting documents required for a campaign before or during campaign creation. These documents help validate the legitimacy and purpose of the campaign and are reviewed by the administrator as part of the campaign approval process.
Preconditions	- Charity organization is registered and logged into the system - Charity has initiated the campaign creation process
Flow of Events	1. Charity organization navigates to the “Create Campaign” page 2. System displays campaign creation form including document upload section 3. Charity uploads required campaign documents (e.g., approval letters, project plans, licenses) 4. System validates document type and size 5. System securely stores uploaded documents and links them to the campaign 6. Charity completes and submits the campaign for approval

Alternate Flow	3a. Charity skips required document upload – system prevents submission and displays warning 4a. Unsupported or invalid document format – system displays an error message and requests re-upload
Postconditions	-Campaign documents are successfully stored in the system- Documents are associated with the submitted campaign- Campaign status remains Pending Approval until reviewed by administrator

Table 2.20 Create Campaign Use Case Description

Use Case ID	UC-13
Use Case Name	Edit Campaigns
Participating Actors	Charity Organization
Use Case Description	Allows a charity organization to modify existing campaign information such as title, goal amount.
Precondition	The charity organization is logged into the system and owns the campaign being edited.
Flow of Events	1. The charity organization navigates to its list of campaigns. 2. The charity organization selects a campaign to edit. 3. The system displays the campaign's current details. 4. The charity organization updates the desired fields. 5. The system validates and saves the changes. 6. The system confirms the successful update.

Alternate Flow	If the charity organization tries to edit a campaign that is already closed or locked by the system, the edit action is not allowed. The system informs the user that the campaign cannot be modified, and no changes are saved.
Postconditions	The campaign's information is updated in the system. Changes are reflected in all related views (campaign listings, search results, donation logs). A system log entry is created to record the update (optional but recommended).

Table 2.21 Edit Campaigns Use Case Description

Use Case ID	UC-14
Use Case Name	Close Campaign
Actor	Charity Organization
Description	Allows a charity organization to close an active campaign when fundraising ends.
Preconditions	- Campaign exists - Campaign is active - User is authorized owner
Flow of Events	1. Charity selects an active campaign 2. Charity chooses "Close Campaign" 3. System marks campaign as closed 4. System disables further donations 5. System sends notifications to relevant users
Alternative Flows	Campaign already closed - message displayed Unauthorized access - action denied

Postconditions	Campaign status updated Relevant users notified
----------------	--

Table 2.22 Close Campaign Use Case Description

Use Case ID	UC-15
Use Case Name	View Donation Summary
Actor	Charity Organization
Description	Allows a charity organization to view summarized donation information for their campaigns.
Preconditions	- User is logged in - User owns at least one campaign
Flow of Events	1. Charity selects a campaign 2. System displays total donations and summary statistics
Alternative Flows	No donations received - empty summary shown Campaign not found - error message
Postconditions	Donation summary is displayed

Table 2.23 View Donation Summary Use Case Description

Use Case ID	UC-16
Use Case Name	Verify Charity Registration
Participating Actors	Administrator

Use Case Description	Allows the administrator to review and verify newly registered charity organizations to ensure they meet eligibility criteria and comply with platform policies. This ensures only legitimate charities can create campaigns and receive donations.
Precondition	The charity organization has submitted a registration request and the administrator is logged in to the system.
Flow of Events	1. Administrator navigates to the pending charity registrations list. 2. Administrator selects a charity registration to review. 3. Administrator verifies submitted details, documents, and compliance requirements. 4. The administrator approves or rejects the registration. 5. The system updates the charity's account status and notifies the charity organization.
Alternate Flow	4a. If the registration is incomplete or invalid, the administrator rejects the registration and may send feedback for correction.
Post Condition	Charity registration is either approved (account activated) or rejected (account remains inactive), ensuring only verified charities participate on the platform.

Table 2.24 Verify Charity Registration Use Case Description

Use Case ID	UC-17
Use Case Name	Review Donation Logs
Participating Actors	Charity Organization, Administrator
Use Case Description	Allows a charity organization to review donation records related to its campaigns, and allows the administrator to monitor all donation activities for reviewing and system oversight purposes.

Precondition	The user is logged into the system and has appropriate access privileges.
Flow of Events	1. The user navigates to the donation logs section. 2. The system verifies the user's role and access privileges. 3. If the user is a Charity Organization: • The system retrieves donation records related only to the organization's campaigns. 4. If the user is an Administrator: • The system retrieves all donation records across the platform for monitoring purposes. 5. The system displays the donation logs in a structured format (e.g., donor name, amount, date, campaign). 6. The user reviews the displayed donation information.
Alternate Flow	If no donation records are found, the system displays an appropriate message.
Postconditions	The user has reviewed donation records appropriate to their role. No donation records are modified during review. System logs the access for reviewing purposes (especially for administrators).

Table 2.25 Review Donation Logs Use Case Description

Use Case ID	UC-18
Use Case Name	Review Financial Transactions
Participating Actors	Administrator

Use Case Description	Allows the administrator to review donation logs and monitor all financial transactions in the system to ensure transparency and maintain system oversight. This use case is limited to viewing transaction records only and does not permit modification.
Precondition	The administrator is logged into the system. The administrator has permission to view transaction records.
Flow of Events	1. The administrator navigates to the transaction review logs section. 2. The system retrieves transaction records from the database. 3. The system displays transaction details such as donor (or anonymous), campaign, donated amount, date, and transaction status. 4. The administrator reviews the transaction records. 5. The administrator may mark transactions for further review if irregularities are noticed. 6. The system records that the transactions were viewed by the administrator.
Alternate Flow	If the system cannot retrieve the financial transactions due to a database error or connectivity issue, it displays an error message and prompts the administrator to try again later.
Postconditions	The administrator has reviewed the available financial transactions. Any transactions marked for further review are recorded by the system. No transaction data is modified. Transaction records remain unchanged and intact.

Table 2.26 Review Financial Transactions Use Case Description

Use Case ID	UC-19
-------------	-------

Use Case Name	Follow Campaign
Participating Actors	Donor
Use Case Description	This use case allows a donor to follow a specific charity campaign in order to receive updates such as donation milestones, campaign progress, and closure notifications.permit modification.
Precondition	The donor must be logged in and have access to the campaign listing or campaign details page.
Flow of Events	1. Donor selects a campaign to follow. 2. Donor clicks the “Follow” button. 3. The system records the donor as a follower of the campaign. 4. The system confirms that the donor is now following the campaign.
Alternate Flow	If the donor is already following the campaign, the system notifies them that they are already a follower and prevents duplicate entries.
Postconditions	The donor is added to the campaign’s follower list and will receive notifications for updates related to that campaign.

Table 2.27 Follow Campaign Use Case Description

2.5.3 Object Model

2.5.3.1 Data Dictionary

Field	Type	Length	Description	Example
-------	------	--------	-------------	---------

user_id	INT	6	Primary key, auto-incremented user identifier	000001
full_name	VARCHAR	100	User's full name	Abel Getachew
email	VARCHAR	100	User email address	abel@email.com
password_hash	VARCHAR	255	Encrypted password	\$2y\$10\$...
role	ENUM	–	User role (Donor, Charity, Admin)	Donor
status	ENUM	–	Account status (Active, Suspended)	Active
registration_date	DATETIME	–	Date of account registration	2025-03-20
last_login	DATETIME	–	Last login timestamp	2025-03-22
donations	ENUM	–	Foreign key to Donations	

Table 2.28: User

Field	Type	Length	Description	Example
charity_id	INT	6	Primary key	101
user_id	INT	6	Foreign key to User	000002
organization_name	VARCHAR	150	Registered charity name	Help Ethiopia
registration_number	VARCHAR	50	Legal registration number	CH-88921
description	VARCHAR	–	Description of Charity Organization	Cancer charity organization

campaignRequests	ENUM	–	Foreign key to CampaignRequest	
verification_status	ENUM	–	Pending, Verified, Rejected	Verified
verification_date	DATETIME	–	Date of verification	2025-03-21
campaigns	ENUM	–	Foreign key to campaign	

Table 2.29: CharityOrganization

Field	Type	Length	Description	Example
campaign_id	INT	6	Primary key	201
charity_id	INT	6	Foreign key to CharityOrganization	101
title	VARCHAR	150	Campaign title	Feed Children
description	TEXT	–	Campaign description	Food aid program
category	ENUM	–	Campaign Categories	Education
location	ENUM	–	Targeted Location	Addis Ababa
donorCount	INT	–	Number of donors	10
target_amount	DECIMAL	–	Fundraising goal	50000
current_amount	DECIMAL	–	Amount raised	32000
status	ENUM	–	Active, Closed	Active
start_date	DATE	–	Campaign start date	2025-03-01
end_date	DATE	–	Campaign end date	2025-06-01

Table 2.30 Campaign

Field	Type	Length	Description	Example
donation_id	INT	6	Primary key	301
campaign_id	INT	6	Foreign key to Campaign	201
donor_id	INT	6	Foreign key to User	000003
amount	DECIMAL	–	Donation amount	500
donation_date	DATETIME	–	Date of donation	2025-03-22
is_anonymous	BOOLEAN	–	Anonymous donation flag	true
message	VARCHAR	–	Optional message donor inserts	Keep Up the good work
guestEmail	EMAIL	–	Email of not signed in donors	abelget@gmail.com
guestName	VARCHAR	–	Name of the unsigned in donor	Abel Getachew

Table 2.31 Donation

Field	Type	Length	Description	Example
receipt_id	INT	6	Primary key	401
donation_id	INT	6	Foreign key to Donation	301
issued_date	DATETIME	–	Receipt issue date	2025-03-22
receipt_reference	VARCHAR	50	Receipt reference code	REC-90321

Table 2.32: DonationReceipt

Field	Type	Length	Description	Example
document_id	INT	6	Primary key	501
campaign_id	INT	6	Foreign key to Campaign	201
document_type	VARCHAR	50	Type of document	License
file_path	VARCHAR	255	Document file path	/docs/license.pdf
upload_date	DATETIME	–	Date uploaded	2025-03-18

Table 2.33 CampaignDocument

Field	Type	Length	Description	Example
follow_id	INT	6	Primary key	601
campaign_id	INT	6	Foreign key to Campaign	201
user_id	INT	6	Foreign key to User	000003
follow_date	DATETIME	–	Date followed	2025-03-19

Table 2.34: CampaignFollow

Field	Type	Length	Description	Example
notification_id	INT	6	Primary key	701
user_id	INT	6	Recipient user	000001
title	VARCHAR	100	Notification title	Campaign Closed
message	TEXT	–	Notification message	Fundraising ended
is_read	BOOLEAN	–	Read status	false
sent_date	DATETIME	–	Date sent	2025-03-23

Table 2.35 Notification

Field	Type	Length	Description	Example
log_id	INT	6	Primary key	801
admin_id	INT	6	Foreign key to User (Admin)	000010
action_type	VARCHAR	100	Type of action	Verify Charity
action_date	DATETIME	–	Date of action	2025-03-24
description	TEXT	–	Action description	Charity approved

Table 2.36 ReviewLog

Field	Type	Length	Description	Example
verification_id	INT	6	Primary key y	901
charity_id	INT	6	Foreign key to CharityOrganization	101
admin_id	INT	6	Admin who verified	000010
status	ENUM	–	Approved, Rejected	Approved
remarks	TEXT	–	Verification remarks	Valid documents
verification_date	DATETIME	–	Date verified	2025-03-24

Table 2.37 CharityVerification

Field	Type	Length	Description	Example
reset_id	INT	6	Primary key	1001

user_id	INT	6	Foreign key to User	000001
reset_code	CHAR	6	Password reset code	458392
expiration_date	DATETIME	–	Code expiration time	2025-03-25

Table 2.38 PasswordReset

2.5.3.2 Class Modeling

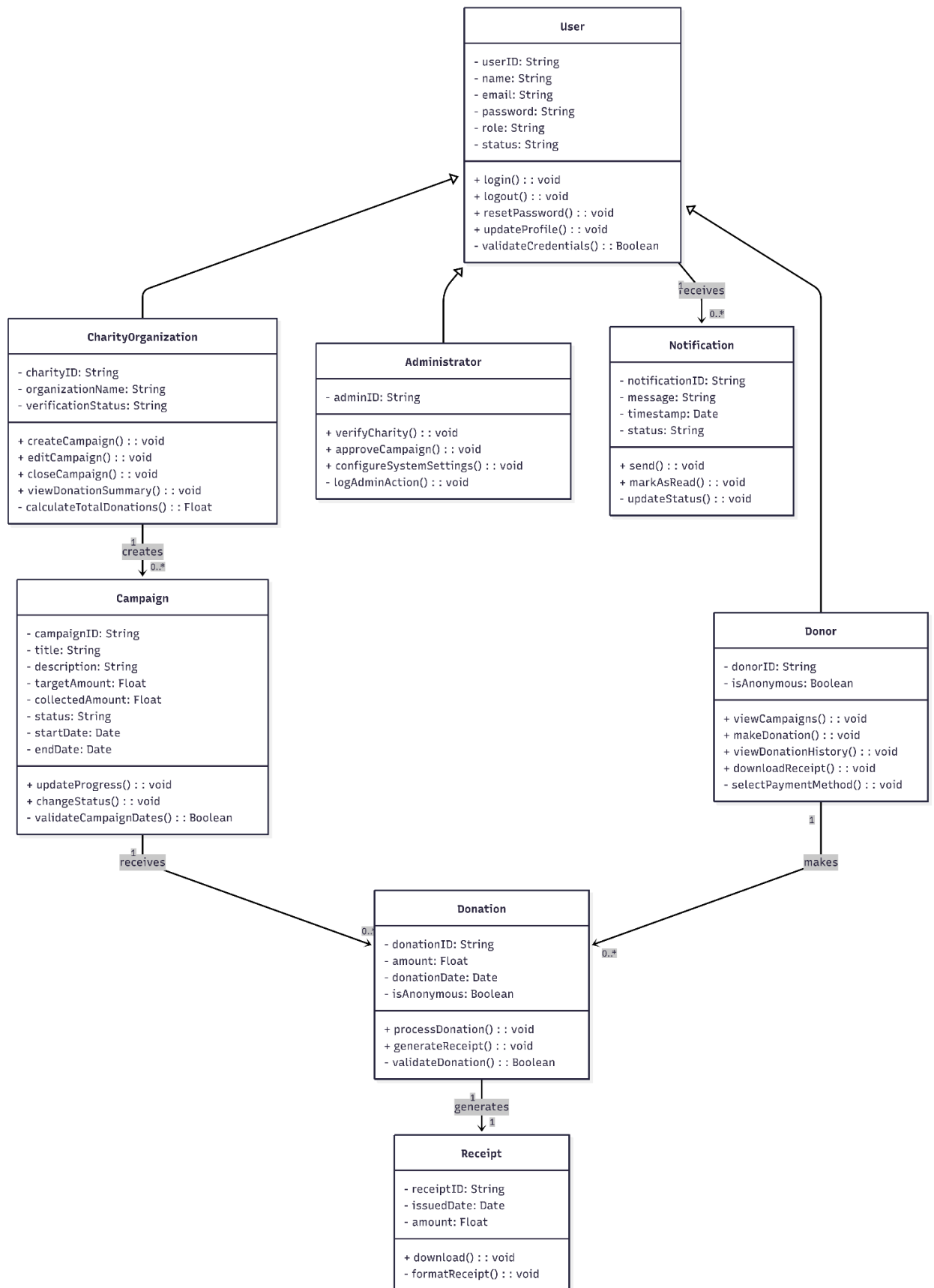


Figure 2.4 Class diagram

2.5.3.3 Dynamic Modeling

Sequence diagrams

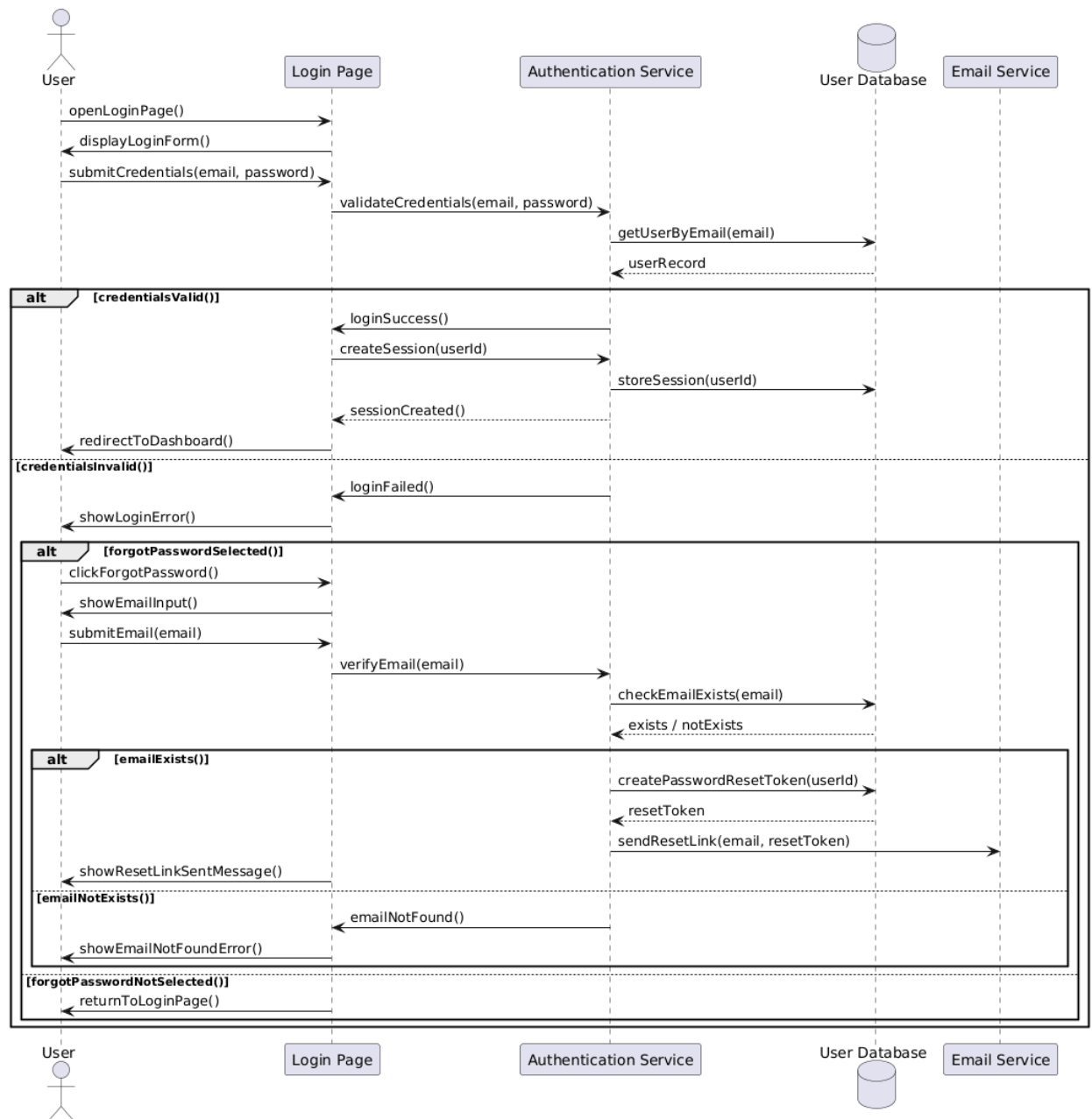


Figure 2.5 Login Sequence diagram with reset password

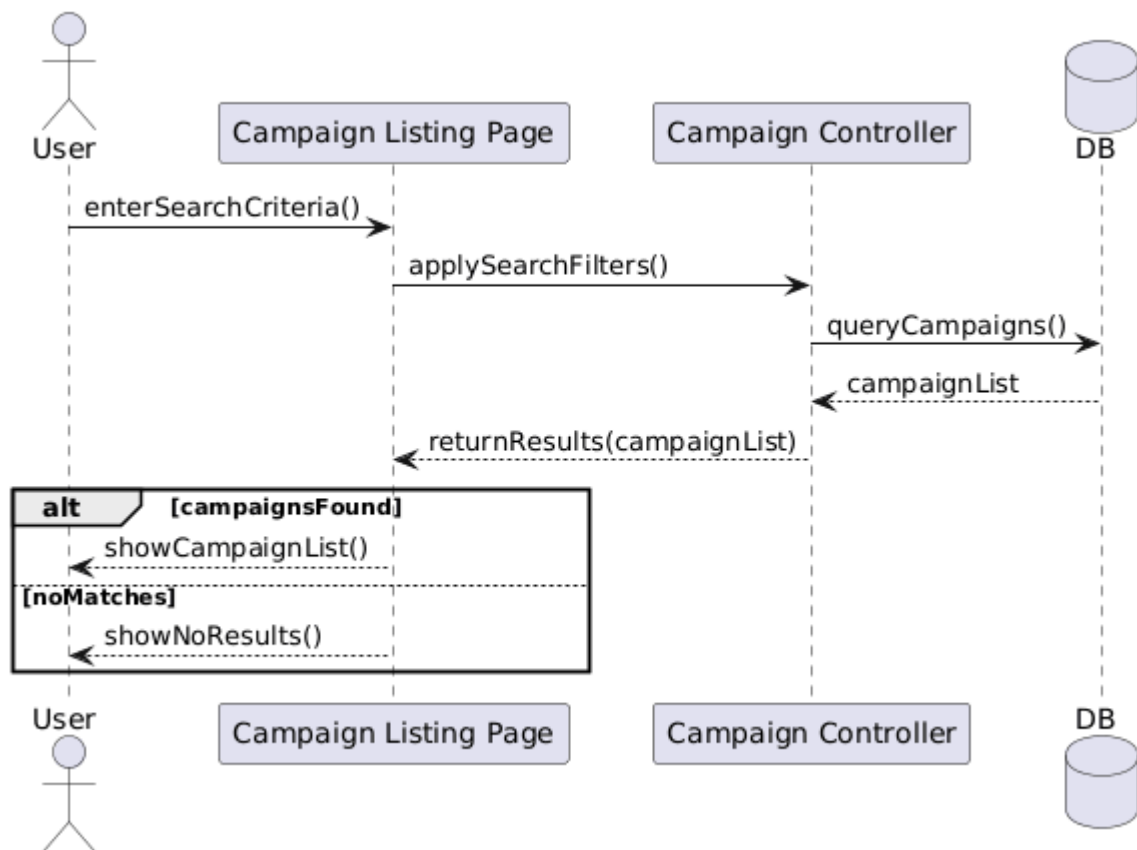


Figure 2.6 Search and filter campaigns sequence diagram

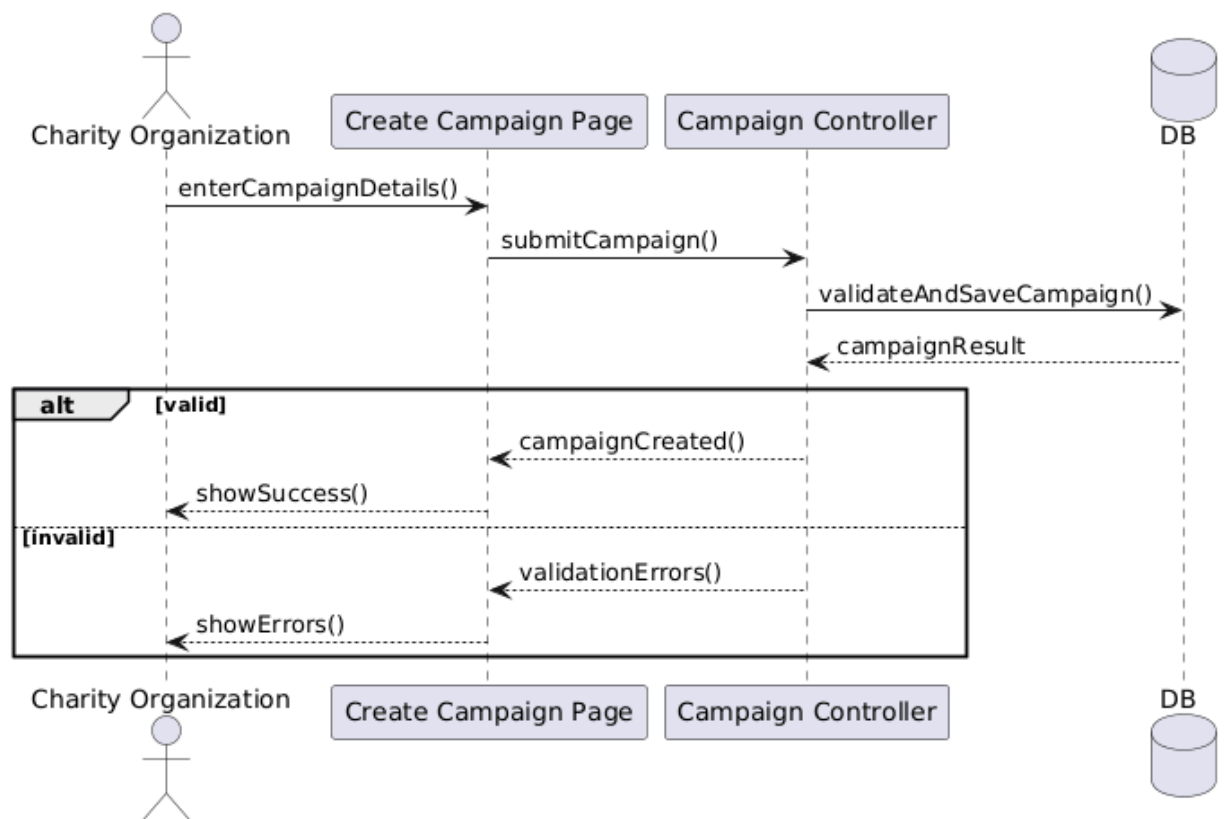


Figure 2.7 Create campaign sequence diagram

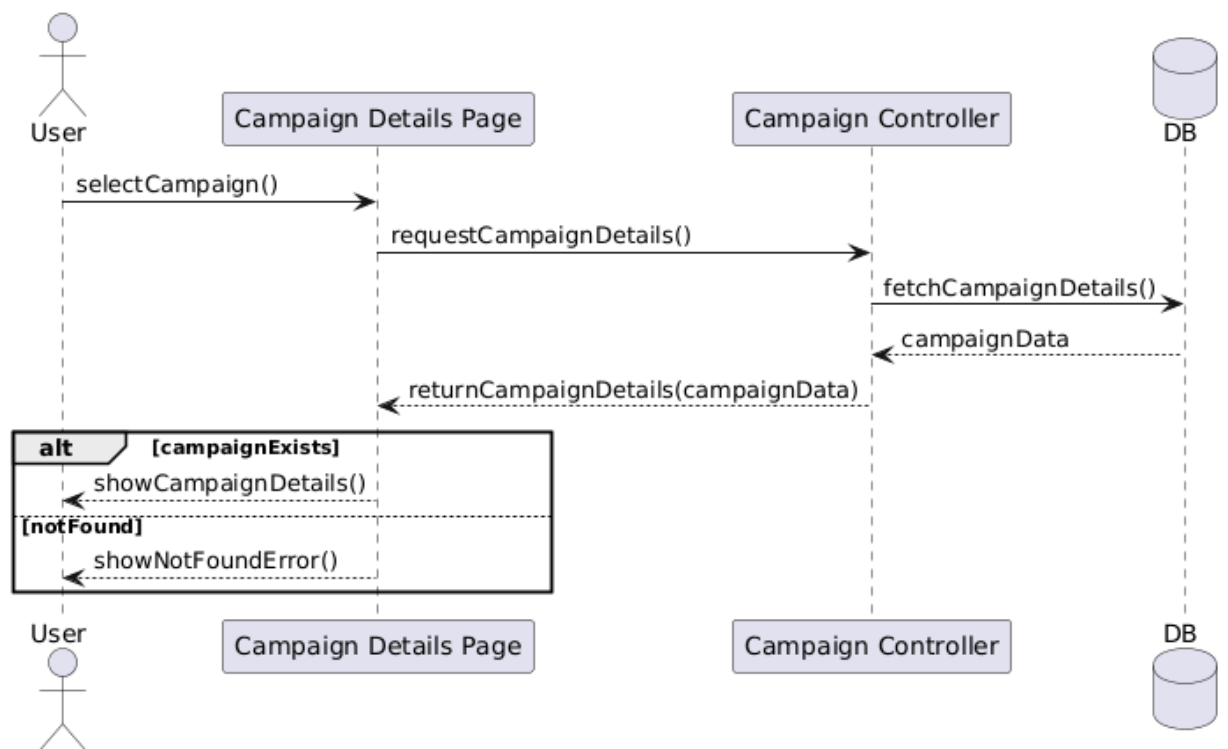


Figure 2.8 View campaign details sequence diagram

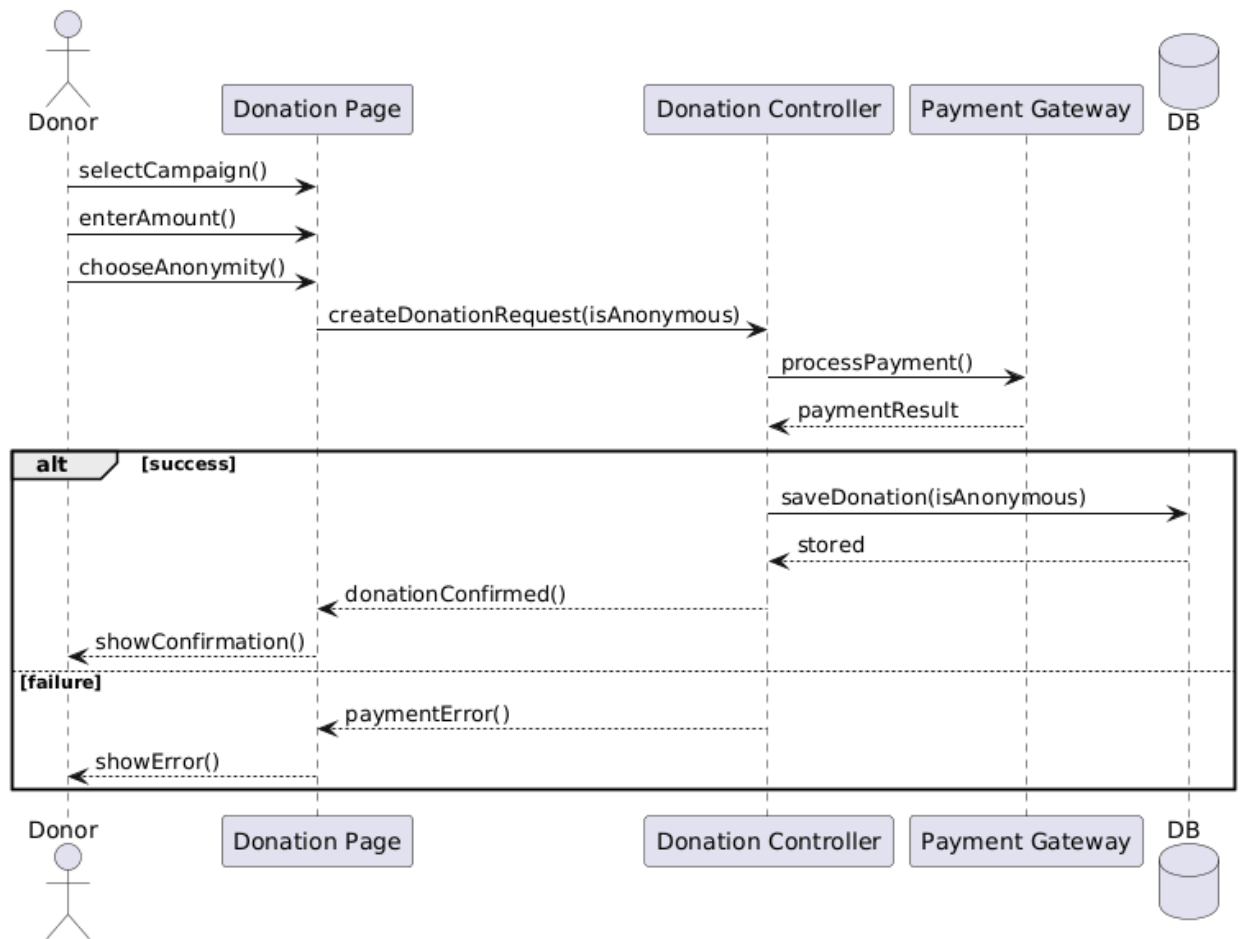


Figure 2.9 Make donation sequence diagram

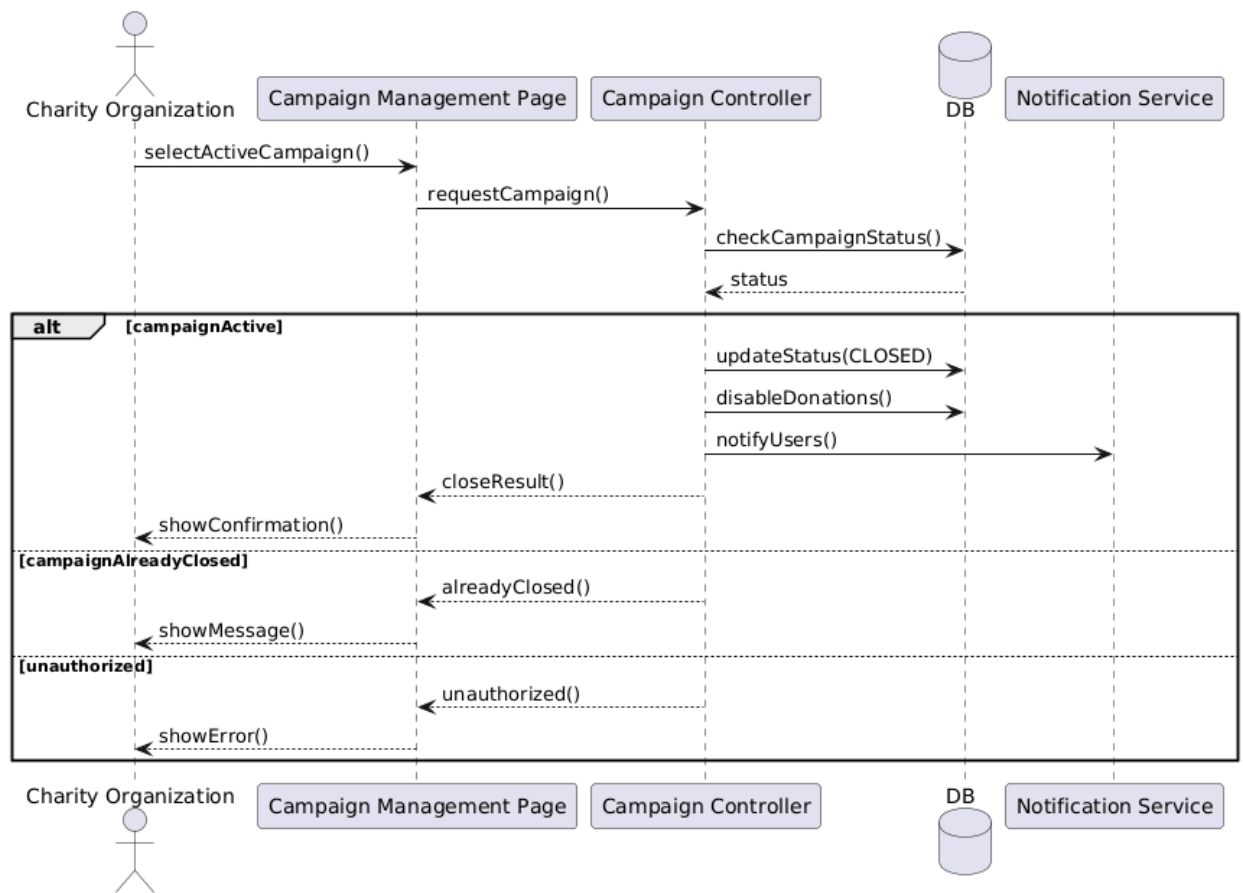


Figure 2.10 Close campaign sequence diagram

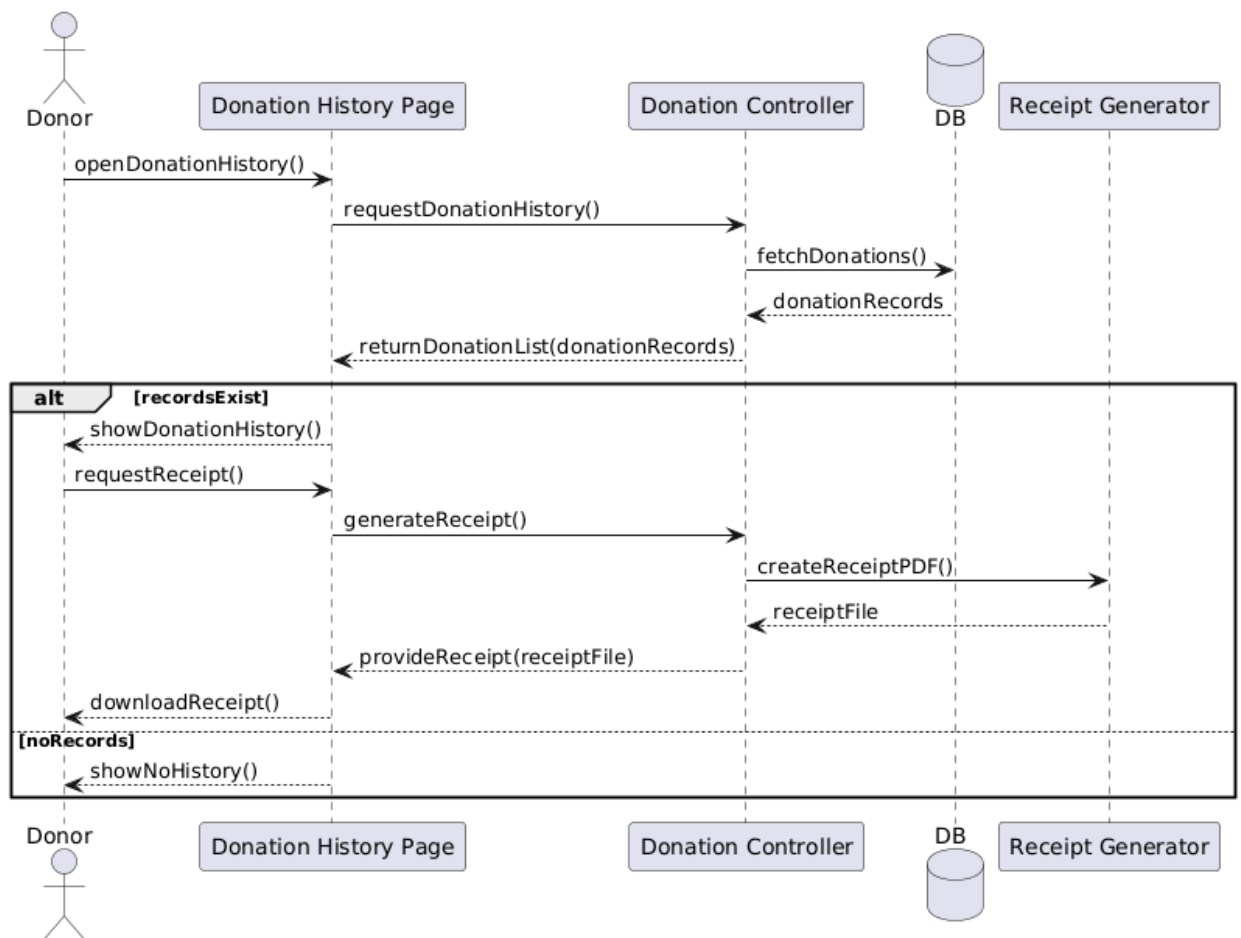


Figure 2.11 View donation history and download receipt sequence diagram

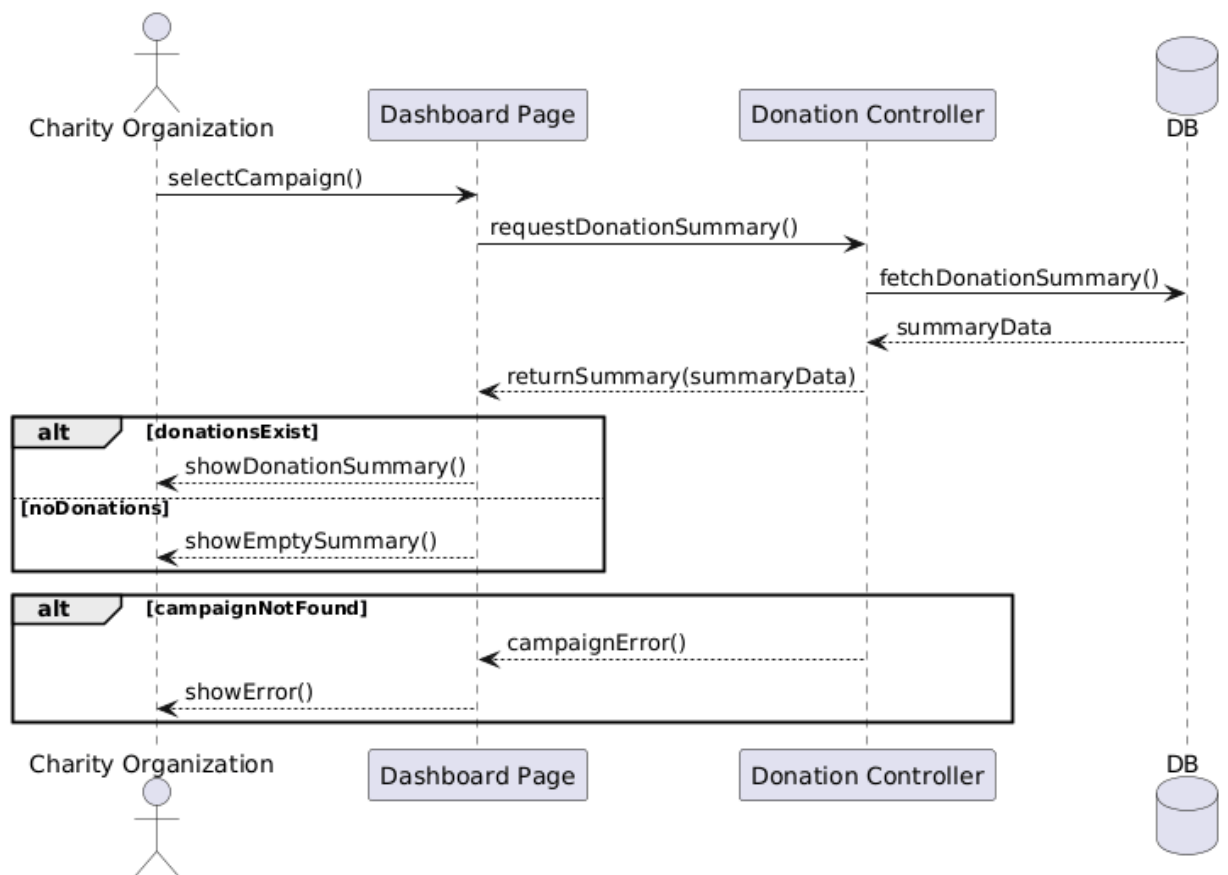


Figure 2.12 View donation summary sequence diagram

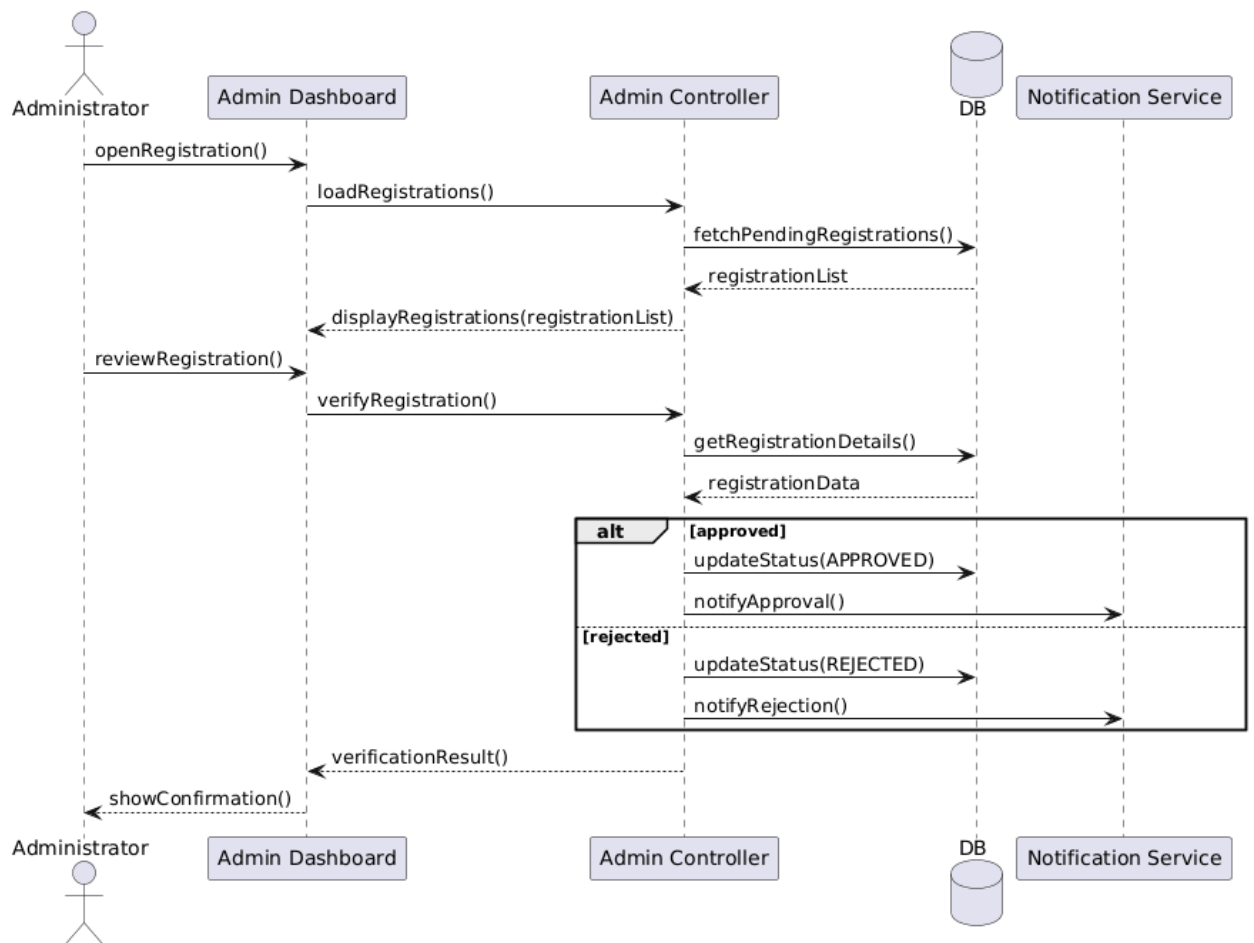


Figure 2.13 Verify Charity registrations sequence diagram

Activity Diagram

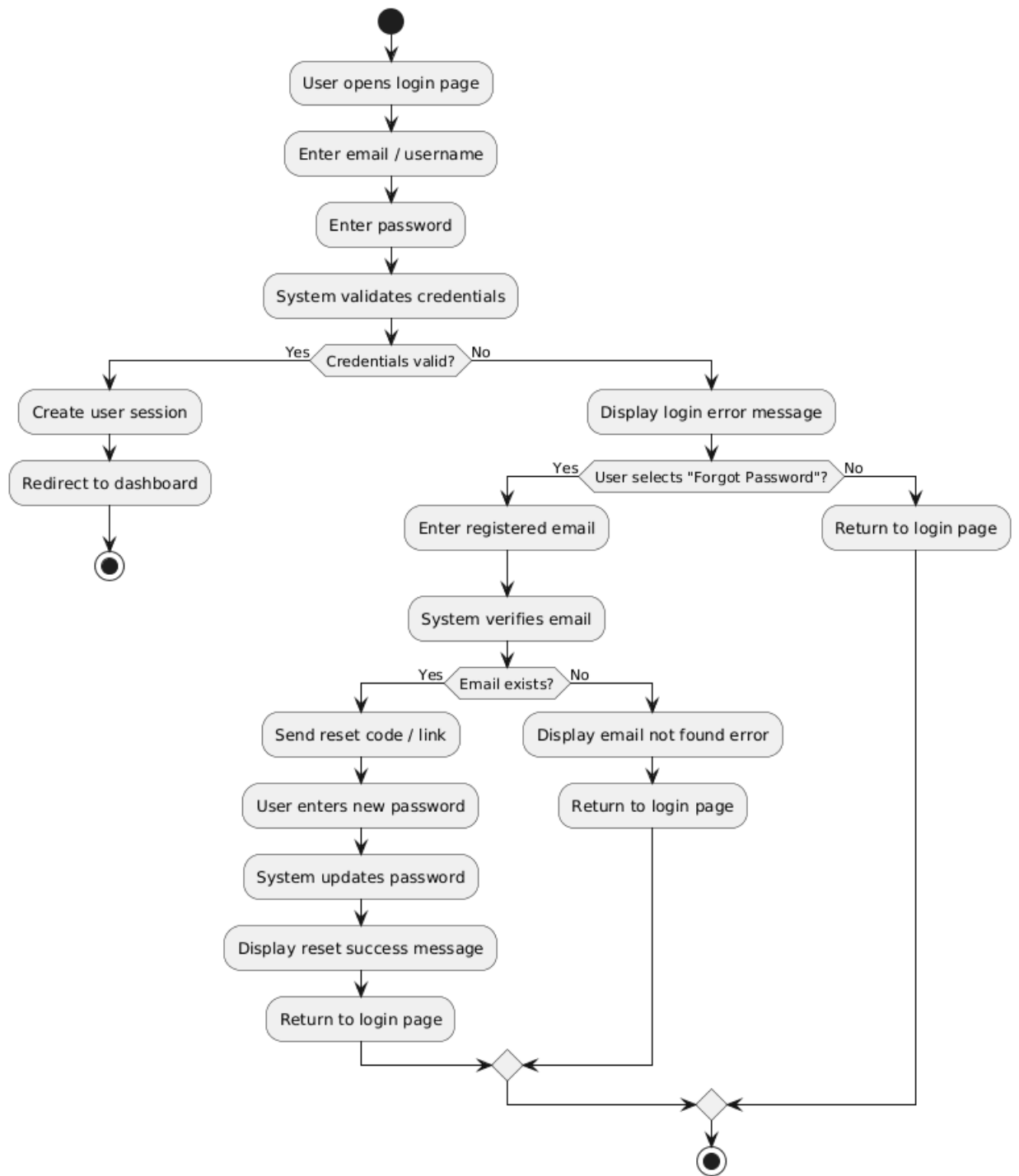


Figure 2.14 Login activity diagram

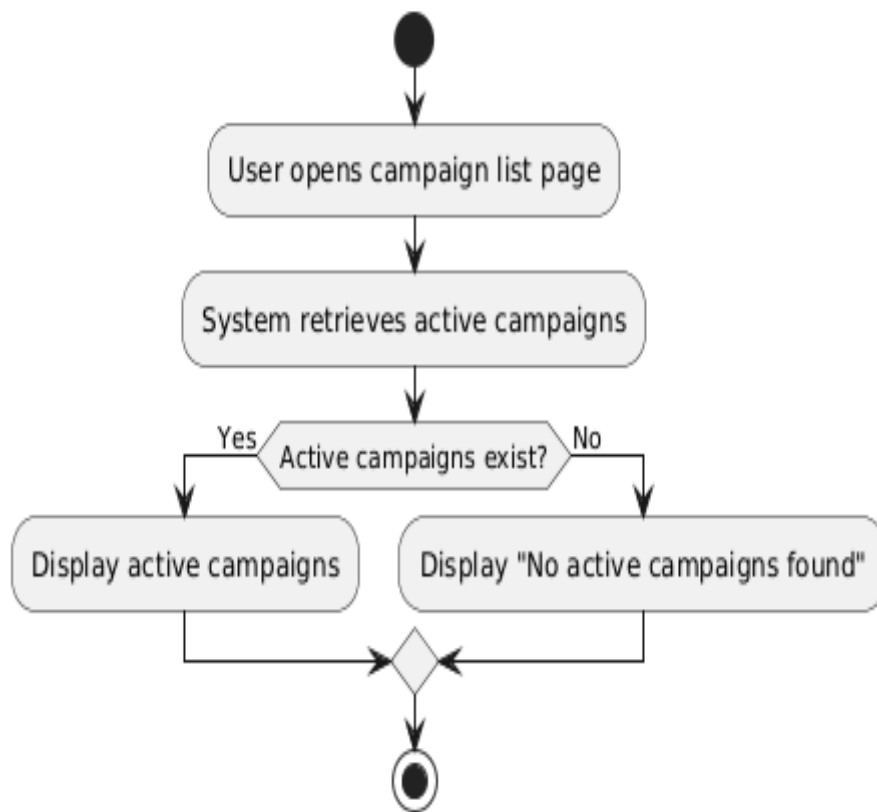


Figure 2.15 View Active Campaigns Activity Diagram



2.16 Search Campaigns Activity Diagram

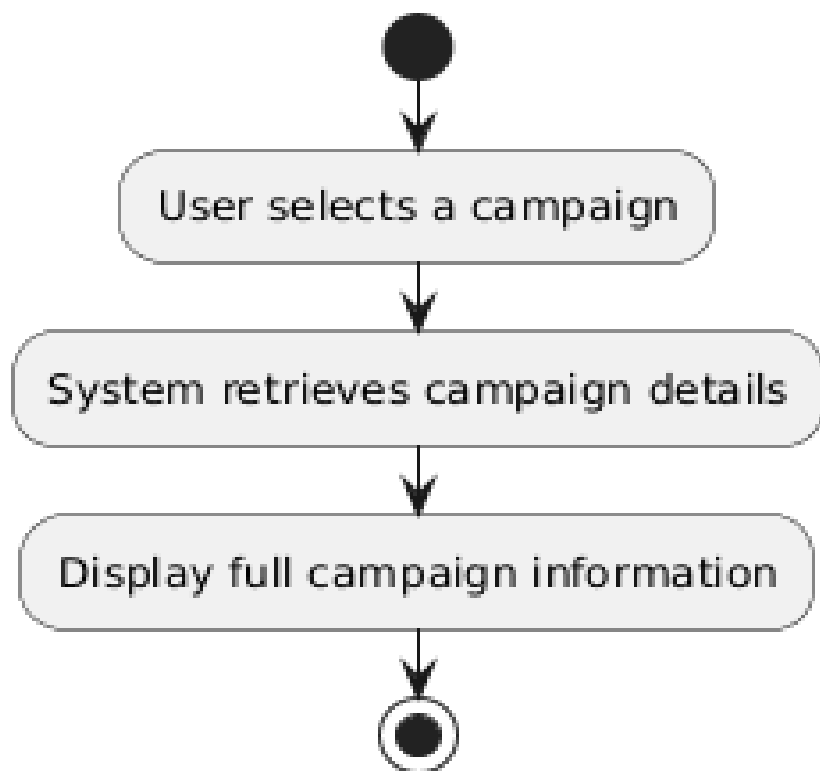


Figure 2.17 View Campaign Details Activity Diagram

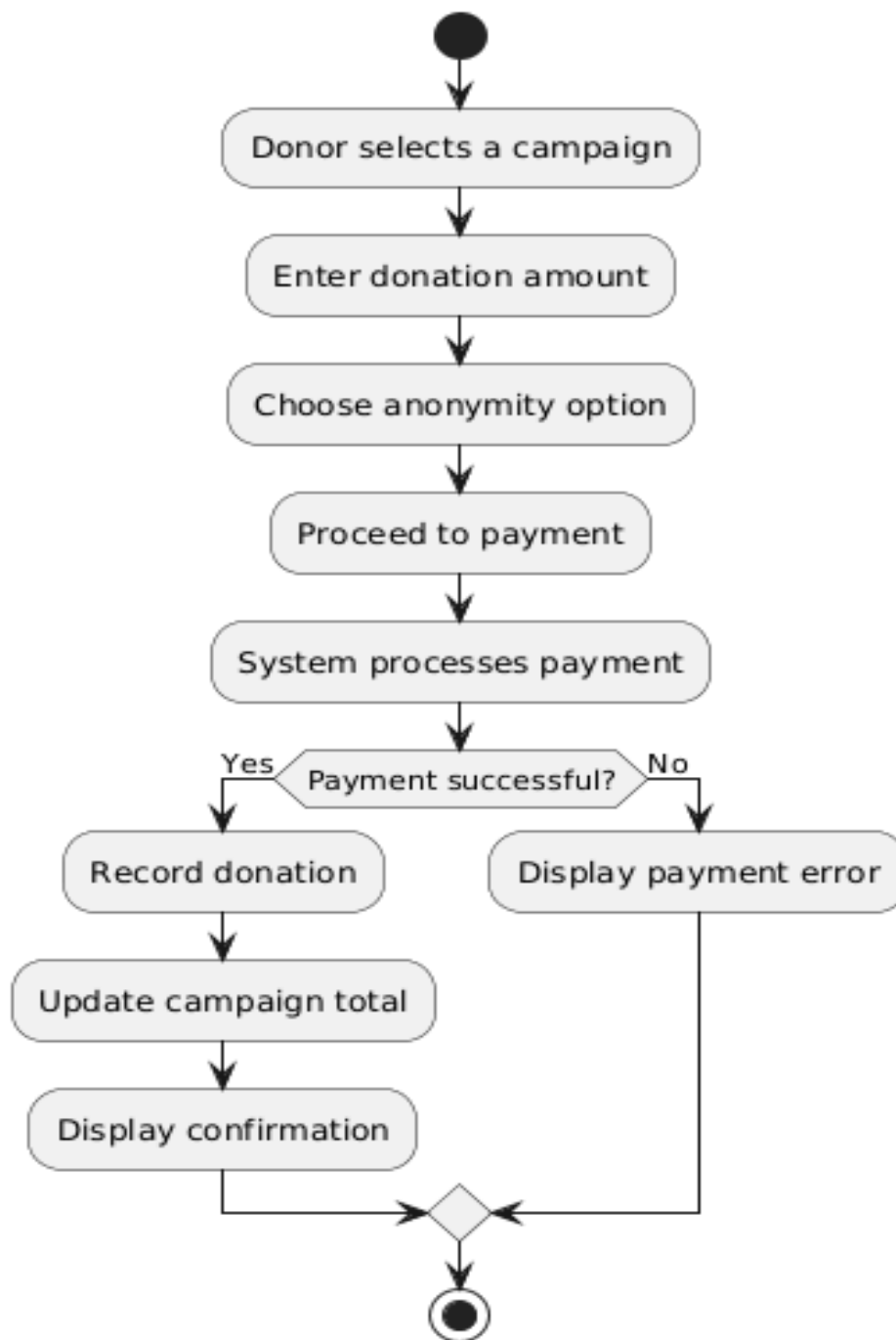


Figure 2.18 Make Donation Activity Diagram

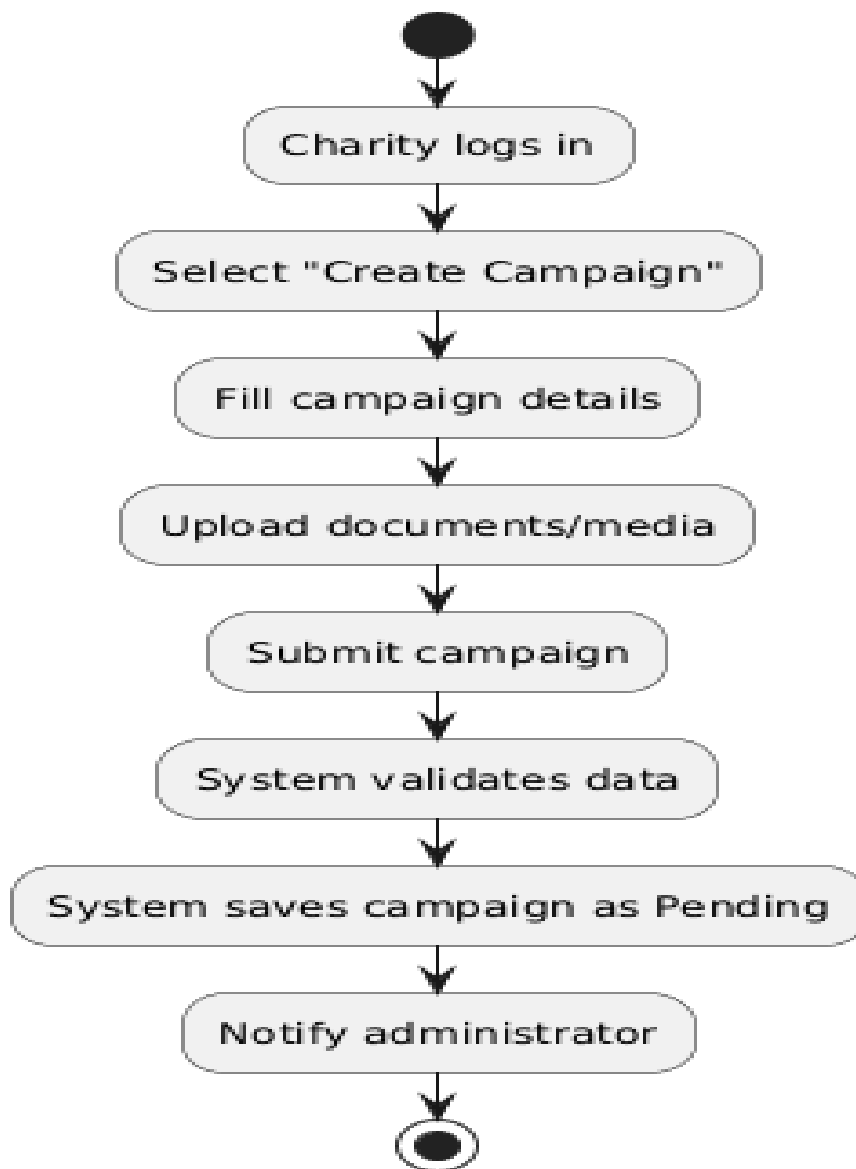


Figure 2.19 Create Campaign Activity Diagram

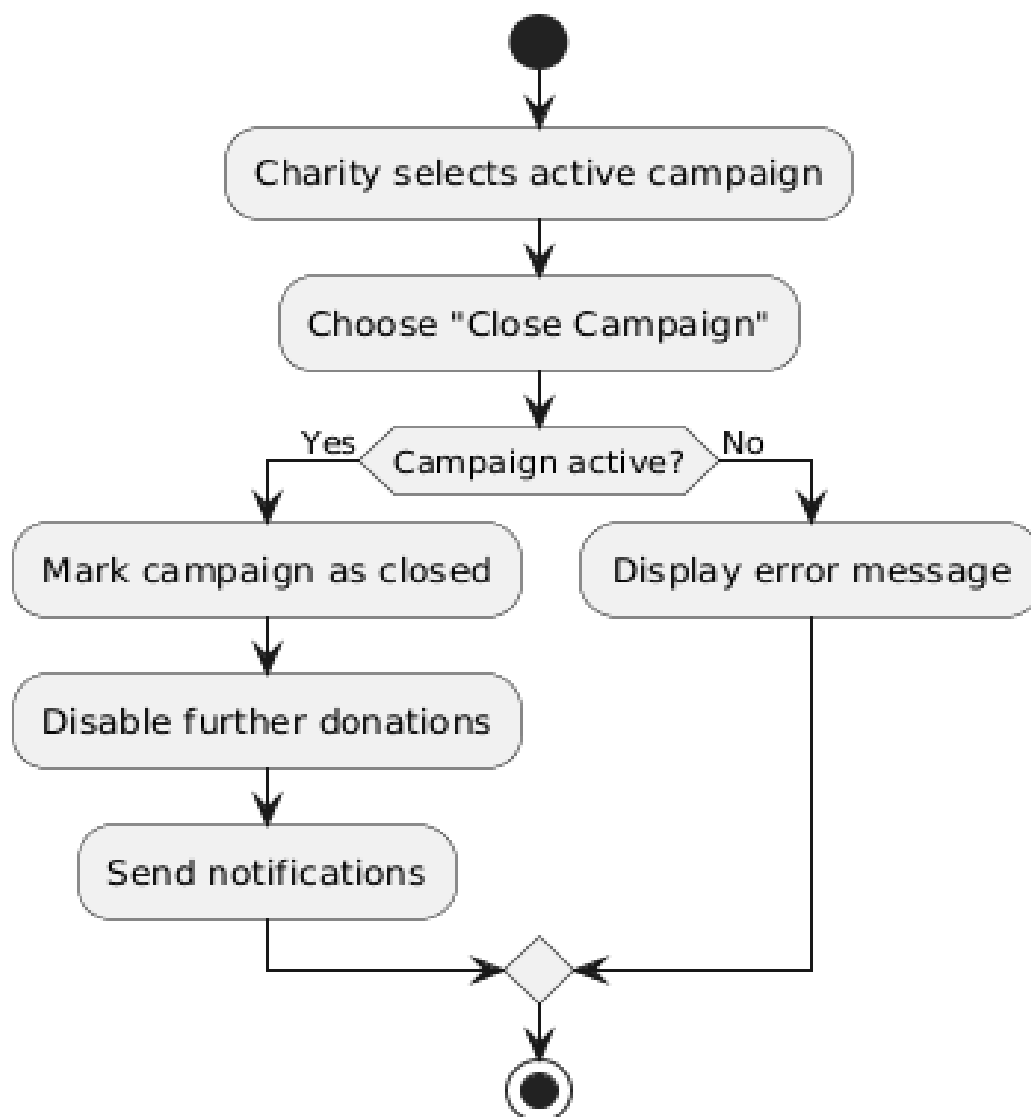


Figure 2.20 Close Campaign Activity Diagram

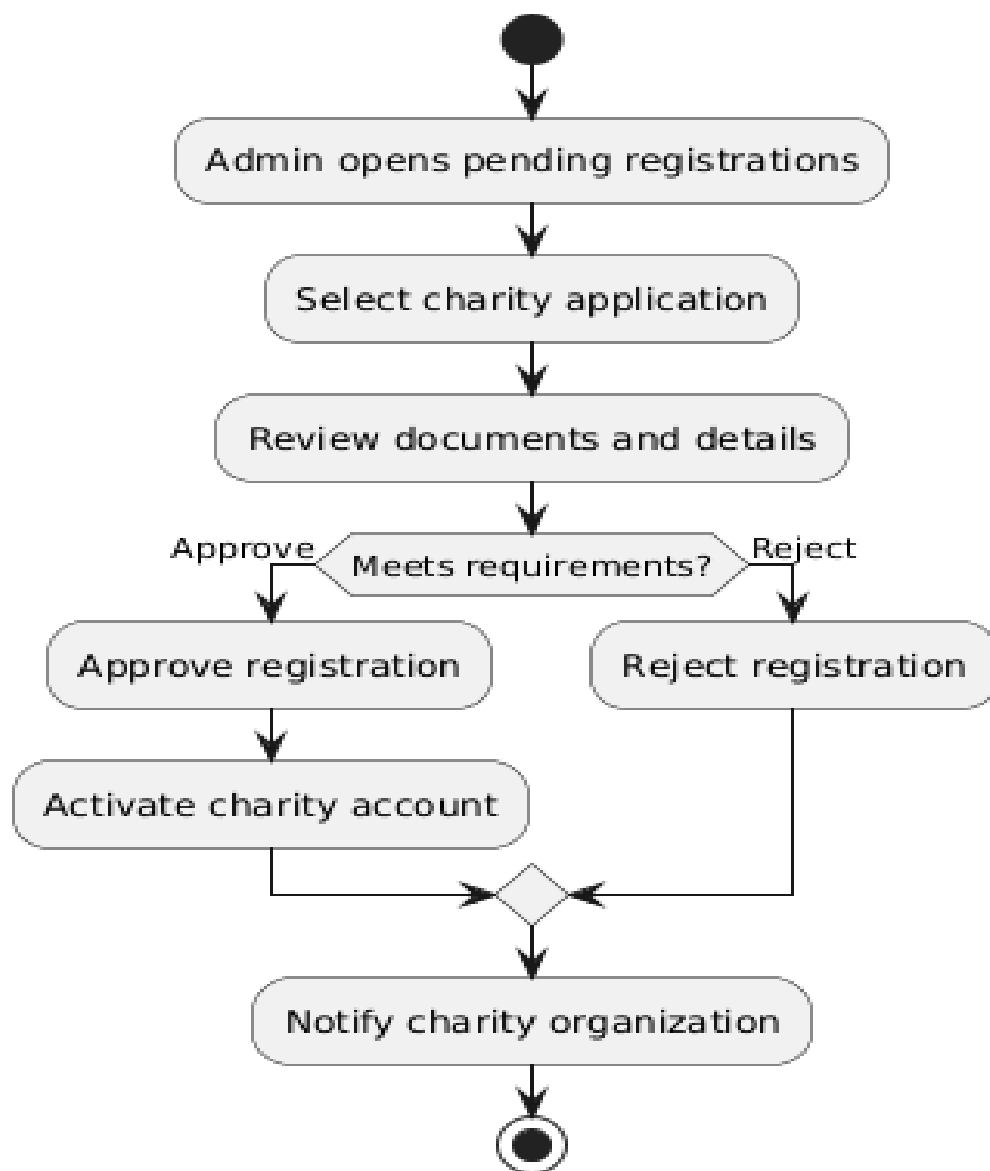


Figure 2.21 Verify Charity Registration Activity Diagram

2.5.3.4 User Interfaces



Figure 2.22 Users landing page UI

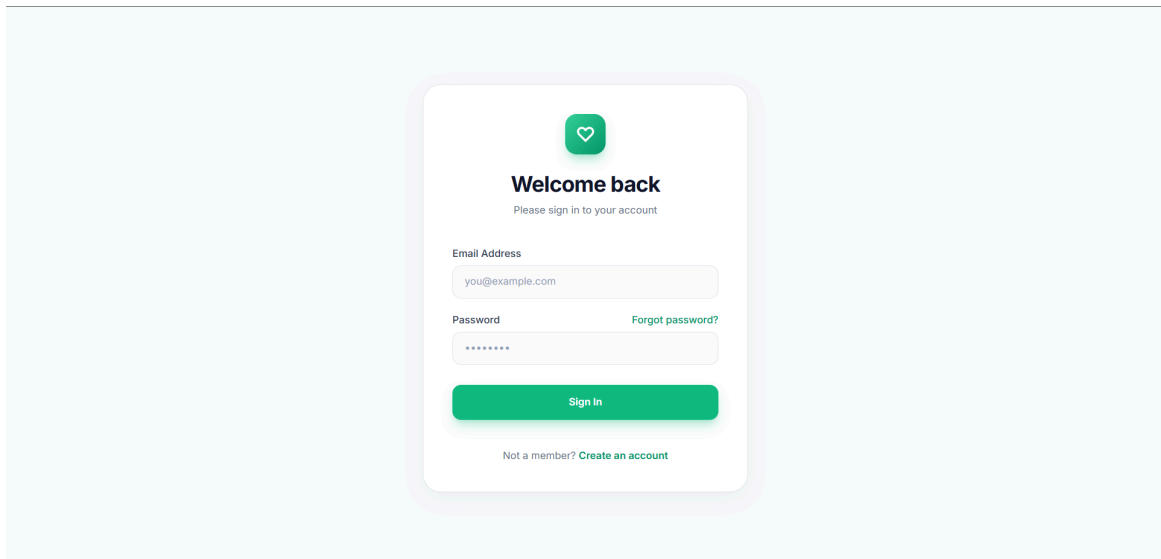


Figure 2.23 Users login page UI

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction

The system design phase provides an overall view of the system from a functional and architectural perspective. It is the process of defining a system's components, modules, interfaces, and data structures in order to satisfy the specified requirements identified during the analysis phase. System design serves as a bridge between requirement analysis and implementation, ensuring that functional and non-functional requirements are translated into a structured and implementable software solution [1].

Effective system design is essential to ensure that all system components operate cohesively to meet user needs and operational constraints. It involves analyzing scalability, performance, reliability, and security requirements while defining clear interfaces and responsibilities for each subsystem. Additionally, the design phase addresses access control mechanisms and data management strategies to protect sensitive information and maintain system integrity. By incorporating these considerations early in the development process, the system becomes more robust, maintainable, and adaptable to future enhancements [2].

3.2. Current Software Architecture

This project proposes a new charity management system and does not replace any existing software. Currently, charity operations are handled manually or through informal tools like spreadsheets and messaging apps, lacking centralization and proper tracking. Therefore, there is no current software architecture to analyze.

3.3 Proposed Software Architecture

3.3.1 Overview

The proposed charity management system adopts a layered software architecture to ensure modularity, maintainability, and clear separation of concerns. This architectural style organizes the system into well-defined layers, where each layer is responsible for a specific aspect of the system's functionality. The layered approach simplifies development and testing while allowing the system to evolve without significantly impacting other components. It is particularly suitable for web-based information systems

that require structured interaction between the user interface, business logic, and persistent data storage .

At a high level, the system is composed of three main layers:

- Presentation Layer
- Application (Business Logic) Layer
- Database Layer

The Presentation Layer provides user interfaces for donors, charity organizations, and administrators, enabling them to interact with the system through web-based views. The Application Layer contains the core business logic, including campaign management, donation processing, notification handling, and access control enforcement. Finally, the Database Layer is responsible for persistent storage of system data such as users, campaigns, donations, and transaction logs.

This architectural design directly supports the functional requirements identified in the requirement analysis phase, such as secure user authentication, campaign creation and management, donation tracking, and administrative oversight. By separating responsibilities across layers, the system enhances scalability, improves security, and reduces coupling between components. Additionally, the layered architecture provides a solid foundation for future enhancements, such as integrating external payment gateways or expanding notification services, without requiring a complete redesign of the system .

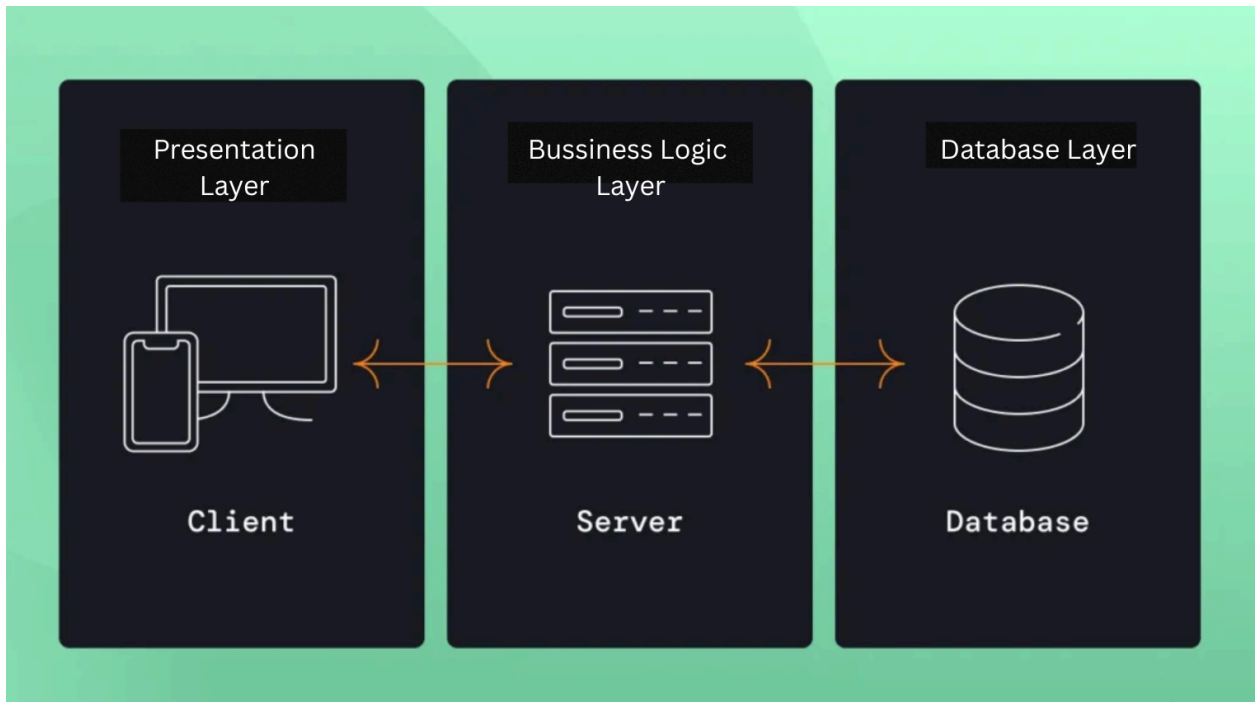


Figure 3.1 Three Tiered Software Architecture

3.3.2 Subsystem Decomposition

Subsystem decomposition is the process of dividing the overall system into smaller, manageable, and logically independent subsystems. Each subsystem is responsible for a specific set of functionalities and encapsulates related operations and data. This approach improves modularity, scalability, maintainability, and clarity of the system design. In the charity management system, the decomposition is based on functional requirements, non-functional constraints, and the roles of system actors.

3.3.2.1 User Management Subsystem

The User Management Subsystem is responsible for handling all user-related operations within the system. This includes user registration, authentication, logout, password reset, and profile management. It supports multiple user roles, namely donors, charity organizations, and administrators, and enforces role-based access control to ensure that users can only access authorized functionalities.

This subsystem plays a critical role in maintaining system security by managing credentials and authentication sessions. It serves as a foundational component upon which other subsystems depend for user identity verification and access control.

3.3.2.2 Campaign Management Subsystem

The Campaign Management Subsystem is responsible for managing the complete lifecycle of charity campaigns within the system. It allows authorized charity organizations to create, edit, and close campaigns while maintaining essential campaign information such as descriptions, funding goals, supporting documents, timelines, and campaign status.

The subsystem also supports administrative workflows by enabling administrators to review, approve, and monitor campaigns to ensure compliance with platform policies. All campaign-related changes are tracked to ensure consistency, transparency, and traceability throughout the campaign lifecycle.

In addition, the Campaign Management Subsystem provides campaign discovery functionality for donors and other users. It enables users to browse, search, and filter available campaigns based on criteria such as keywords, category, location, and campaign status. By integrating discovery features within the same subsystem, the system ensures efficient access to up-to-date campaign information while optimizing read operations for frequent user interactions.

3.3.2.3 Donation Management Subsystem

The Donation Management Subsystem is responsible for recording and managing donation transactions. It allows donors to make contributions, optionally remain anonymous, and view their donation history. The subsystem also supports the generation and retrieval of digital donation receipts.

This subsystem ensures data integrity and accountability by maintaining accurate donation records and transaction logs. It is a critical component for financial transparency and supports both donor reporting and administrative reviewing.

3.3.2.4 Notification Subsystem

The Notification Subsystem manages user subscriptions to campaigns and delivers notifications related to system events. Users can follow campaigns to receive updates on progress, milestones, approvals, and closures.

Notifications may be delivered through email or in-application messages. This subsystem enhances user engagement and ensures timely communication between the system and its users, while remaining loosely coupled with other subsystems through event-based interactions.

3.3.2.5 Administration Subsystem

The Charity Verification and Administration Subsystem supports administrative control and governance of the platform. It enables administrators to verify charity organization registrations, manage user accounts, review donation logs, and transactions.

This subsystem enforces platform rules and ensures compliance with organizational policies. By centralizing administrative functions, it contributes to system integrity, trust, and accountability.

3.3.2.6 Reporting Subsystem

The Reporting Subsystem provides functionality for generating structured reports and summaries related to system activities. It supports the creation of donation summaries, campaign performance reports, and basic statistical overviews based on the data stored in the system.

This subsystem processes data such as donation amounts, campaign progress, and user participation to produce meaningful reports for administrators and authorized stakeholders. Reports may include total donations per campaign, donation trends over a given period, and overall system usage summaries.

By offering accurate and up-to-date reporting capabilities, the Reporting Subsystem enhances transparency and supports informed decision-making without introducing additional system complexity.

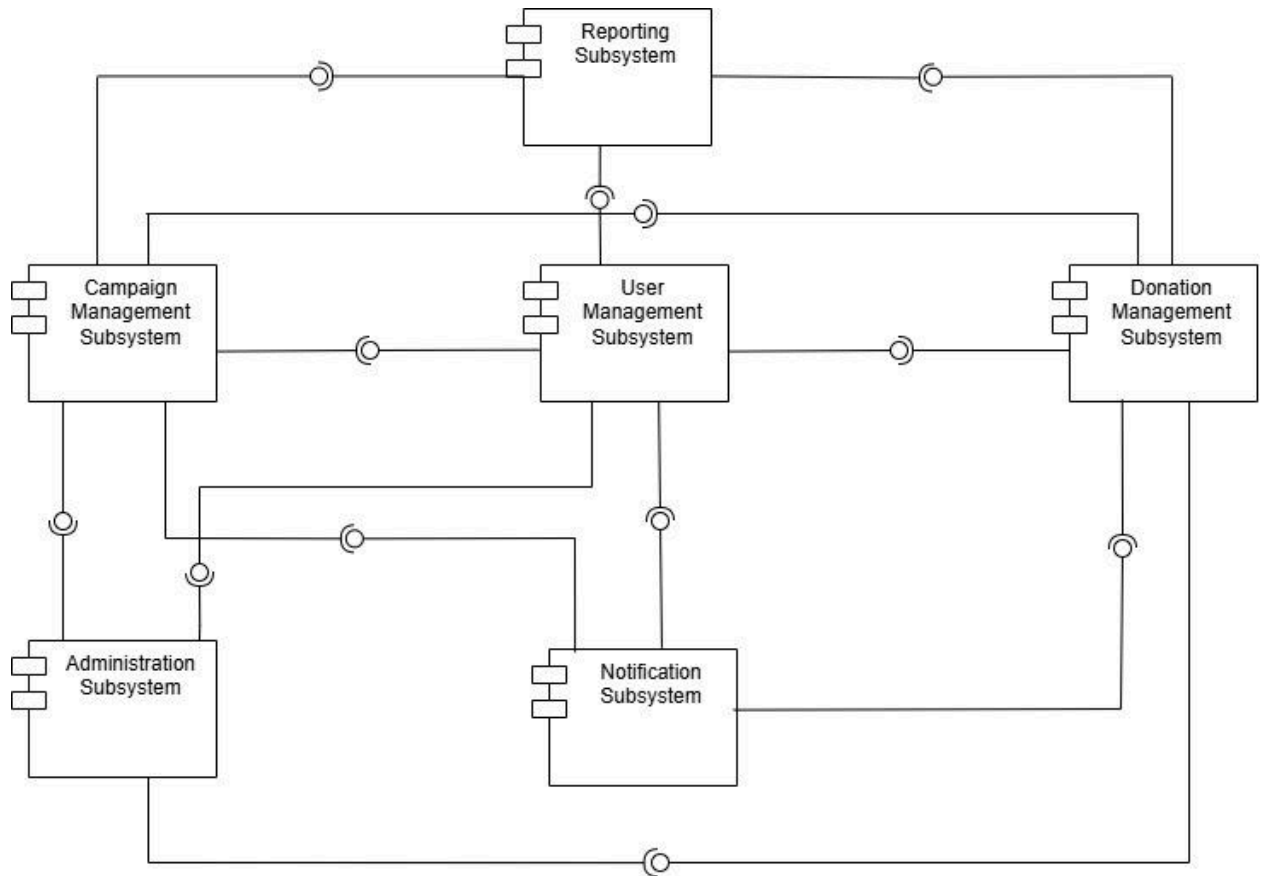


Figure 3.2 Subsystem Decomposition

3.3.3 Hardware/Software Mapping

It describes information such as the system's hardware configuration, the nodes responsible for a particular functionality, and so on. Unified Modeling Language (UML) deployment diagrams are used to depict the Hardware/software mapping. It is used to describe the hardware components where software components are deployed. The Hardware/software mapping is described below.

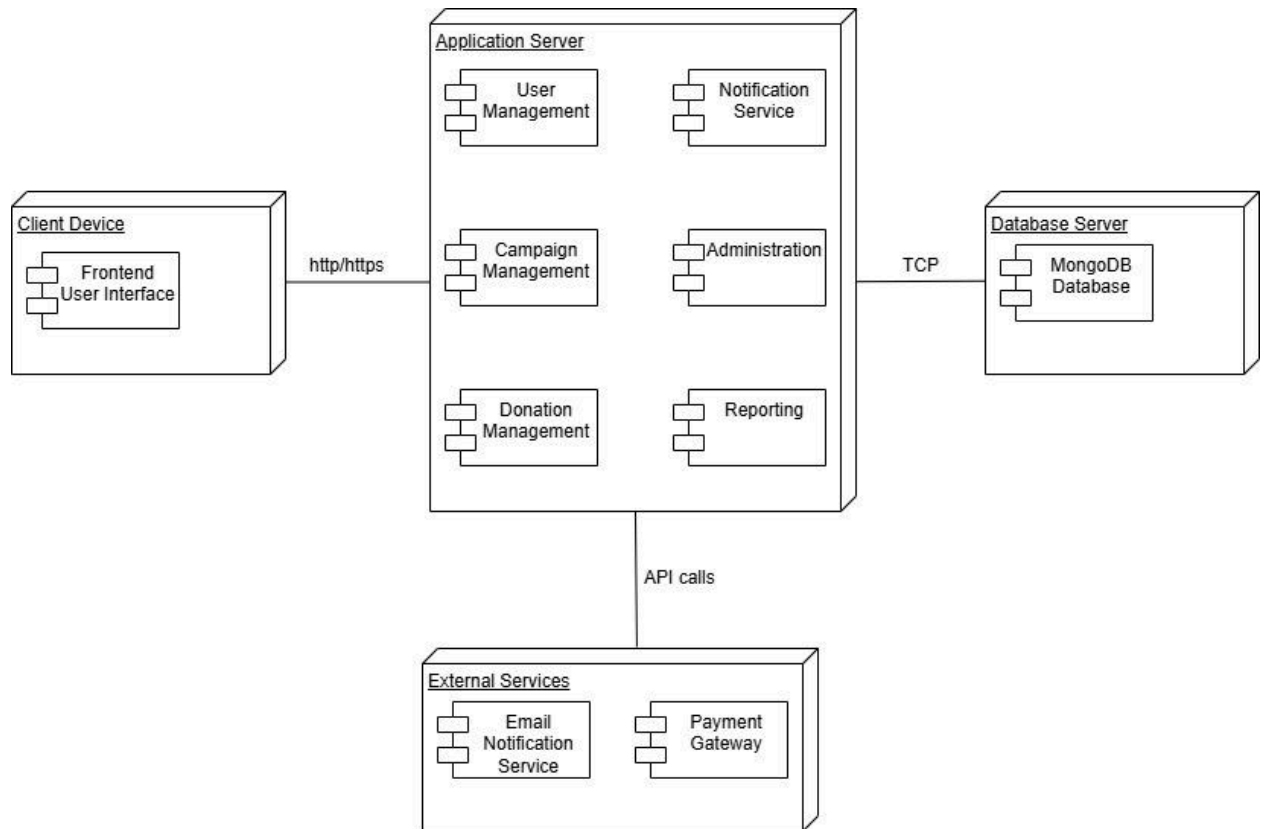


Figure 3.3 Deployment Diagram

3.3.4 Persistent Data Management

The charity management system uses MySQL, a relational SQL database, to manage persistent data. MySQL was selected because of its reliability, performance, and strong support for structured data and relationships. Its relational model enables efficient management of interconnected entities such as users, campaigns, donations, and transactions through tables, primary keys, and foreign key constraints. MySQL also provides data integrity, consistency, and scalability, making it well-suited for handling the growing number of users, donations, and organizational records within the system.

Tables	Description
User	Stores donor, charity organization, and administrator account details

Campaign	Stores campaign information created by charity organizations
Donation	Stores donation transactions made by donors
Campaign Document	Stores campaign-related uploaded documents
Notification	Stores system-generated notifications

Table 3.1 Core Data Objects

3.3.5 Access Control and Security

The Charity Management System uses Role-Based Access Control (RBAC) to ensure that system resources are accessed only by authorized users. There are three roles: Donor, Charity Organization, and Administrator, each with permissions tailored to their responsibilities. Donors can browse campaigns, make donations, and view donation history; charities can manage campaigns; administrators can verify charities, review transactions, and configure system settings.

Our system employs JSON Web Tokens (JWT) for user authentication. JWTs are chosen for their compactness, self-contained nature, ease of use, and the ability to be transmitted via HTTP headers. They provide a secure method of authenticating users by including a token with each request. The token's JSON payload can be digitally signed or encrypted, ensuring the secure transmission of information. Additionally, JWTs seamlessly integrate into existing systems and enable user authentication across multiple platforms.

Sensitive operations, including donations, campaign updates, and user management, are logged to ensure accountability. Input validation and access checks protect against unauthorized data manipulation, maintaining the confidentiality, integrity, and availability of system data.

Functionality	Donor	Charity Organization	Administrator
User Registration	✓	✓	✓
User Login / Logout	✓	✓	✓
Reset Password	✓	✓	✓
Manage User profile	✓	✓	✓
View Active Campaigns	✓	✓	✓
View Campaign Details	✓	✓	✓
Search Campaigns	✓	✓	✓
Filter Campaigns	✓	✓	✓
Make Donation	✓	✗	✗
View Donation History	✓	✗	✗
Follow Campaign	✓	✗	✗
Download Donation Receipt	✓	✗	✗
Review Donation Logs	✗	✓	✓
Create Campaign	✗	✓	✗
Edit / Close Campaign	✗	✓	✗
Upload Campaign Documents	✗	✓	✗

Upload Charity Registration Document	X	✓	X
View Donation Summary	X	✓	X
Verify Charity Registration	X	X	✓
Review Financial Transactions	X	X	✓
Configure System Settings	X	X	✓
Receive Notifications	✓	✓	✓

Table 3.2 Access Control

3.4 Detailed Class Diagram

This section offers an in-depth review of the system's classes, along with their dependencies, constraints, attributes, and functions.

1. Donor Side Class Diagram

This class diagram illustrates the core classes involved in the donation process from the donor's perspective. It shows how a donor interacts with charity campaigns by making donations, optionally choosing anonymity. Each donation is associated with a specific campaign and may generate a receipt as confirmation of a successful transaction. The diagram focuses on capturing donation details, campaign progress, and receipt generation while keeping the structure simple and aligned with donor-related system use cases.

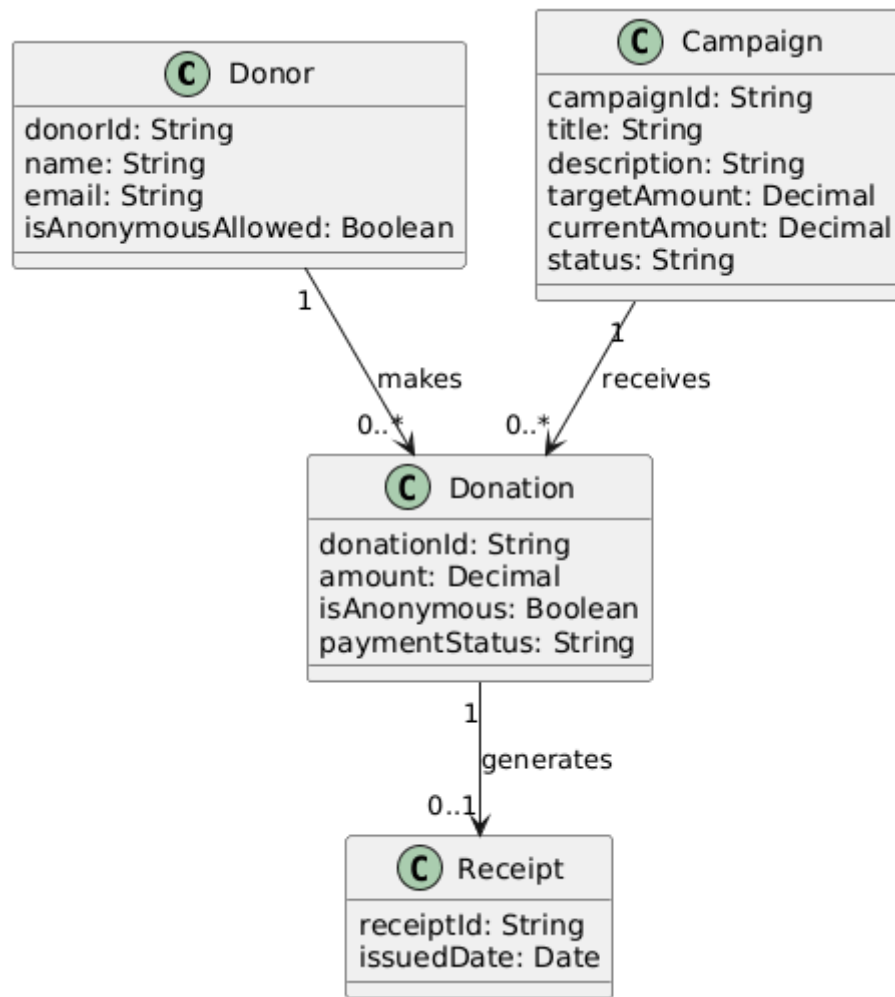


Figure 3.4 Donor Side Class Diagram

2. Charity Organization Side Class Diagram

This class diagram represents the classes used by charity organizations to manage fundraising activities within the system. It highlights how a charity organization creates and manages campaigns and how donations are linked to those campaigns for tracking and reporting purposes. The diagram emphasizes campaign lifecycle management, donation monitoring, and the organization's verification status, reflecting the operational responsibilities of charity organizations in the system.

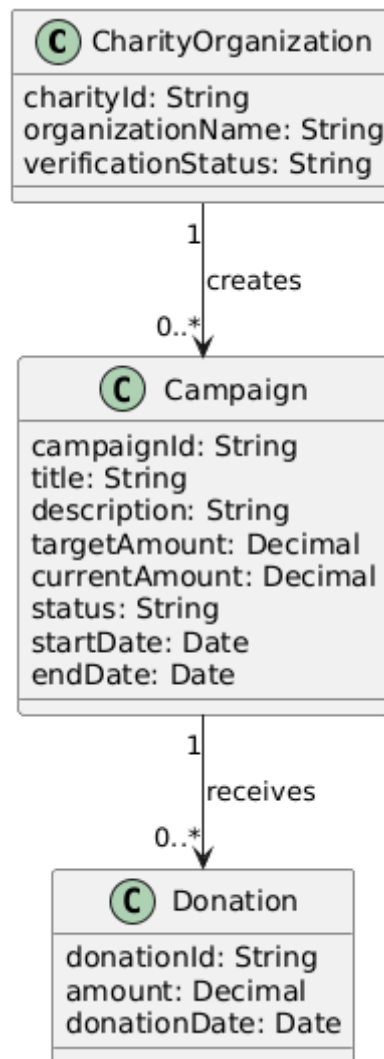


Figure 3.5 Charity Organization Side Class Diagram

3. Administrator Side Class Diagram

This class diagram represents the classes responsible for **administrative management and system oversight**. It illustrates how administrators verify charity organizations, manage user accounts, review system records, and oversee campaigns and donations. The diagram also includes notification-related classes used to communicate important system events to users. Overall, it focuses on **governance, access control, and supervision tasks** performed by administrators to ensure the system operates correctly and transparently.

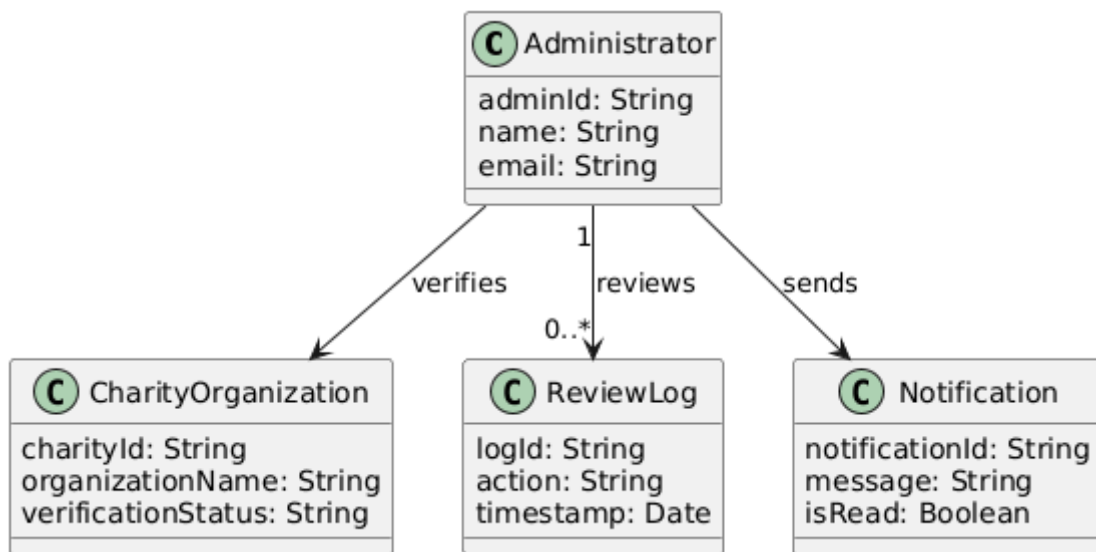


Figure 3.6 Administrator Side Class Diagram

3.5 Packages

1. User Management Package

The User Management package takes care of everything related to users in the system. It allows donors, charity organizations, and administrators to create accounts, log in securely, and manage their profiles. It also controls what each user is allowed to do by assigning roles, making sure everyone only has access to the features meant for them.

2. Campaign Management Package

The Campaign Management package handles all charity campaigns from start to finish. It lets charity organizations create and update campaigns, while administrators review and approve them before they go live. This package makes sure campaign details are accurate, up to date, and clearly visible to donors.

3. Donation Management Package

The Donation Management package manages all donation-related activities in the system. It records each donation, tracks payment status, supports anonymous giving, and generates receipts for donors. This ensures that donations are properly documented and that financial information remains transparent and reliable.

4. Administration Package

The Administration package is used by administrators to oversee and control the platform. It allows them to verify charity organizations, monitor system activities, and

enforce platform rules. This package helps keep the system trustworthy, secure, and well-organized.

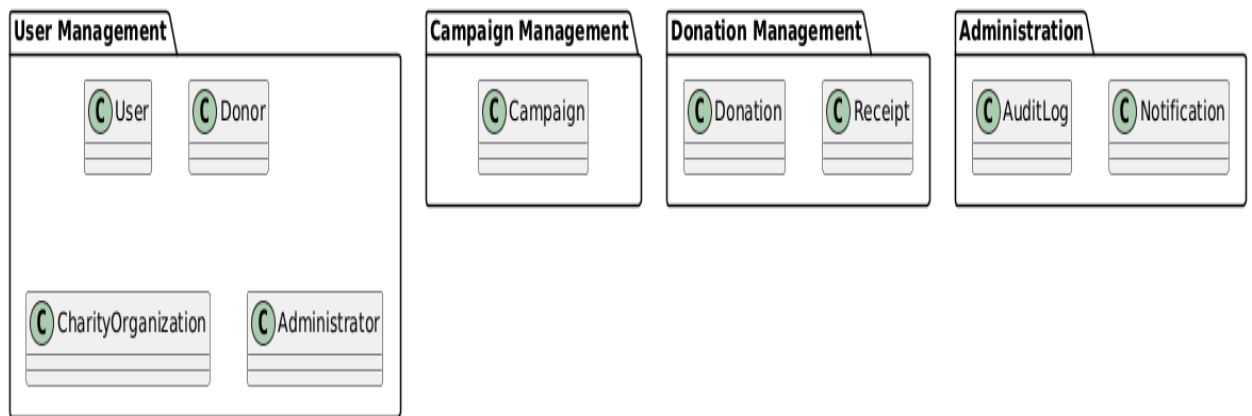


Figure 3.7 Package Diagram

3.5.1 Dependency of Packages

The package dependency structure shows how different parts of the charity management system work together. The User Management package supports both the Campaign Management and Donation Management packages by handling user authentication and role-based access. The Campaign Management package relies on Donation Management to record and track donations made to each campaign. The Administration package connects with all other packages so administrators can verify users, approve campaigns, and review donation activities. This structure keeps responsibilities well separated while allowing the system components to work smoothly together.

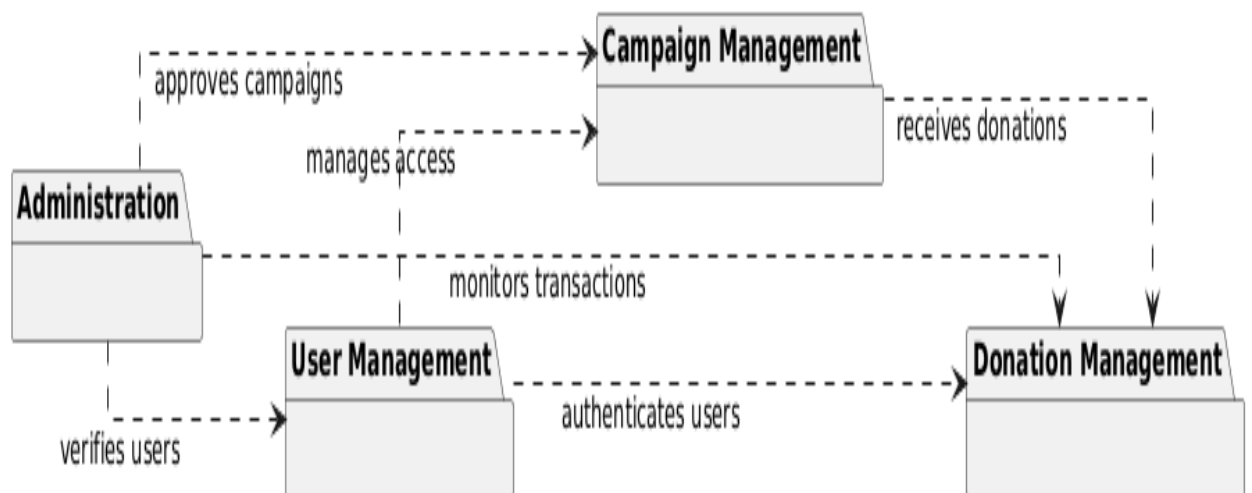


Figure 3.8 Package Dependency Diagram

4. IMPLEMENTATION

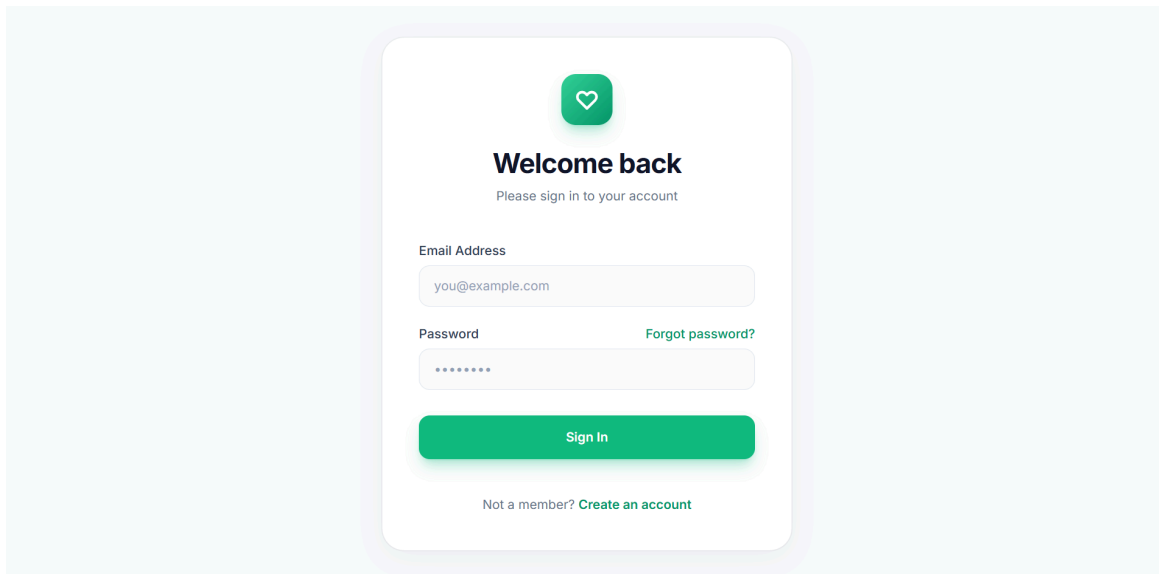
This section documents the visual layout and user experience (UX) profiles of the implemented Charity Management System. Detailed screenshots are provided below, organized sequentially according to the primary user workflows, including donor interactions, campaign management, donation processing, and administrative operations. Screen captures of the corresponding interfaces may be inserted into the designated spaces to illustrate the system's functionality and navigation flow.

4.1 ScreenShots

Figure 4.1 Landing page



Figure 4.2 Login page



The login page features a central white card on a light blue background. At the top of the card is a green heart icon. Below it, the text "Welcome back" is displayed in bold, followed by "Please sign in to your account". The form includes an "Email Address" field with the placeholder "you@example.com", a "Password" field with masked characters, and a "Forgot password?" link. A prominent green "Sign In" button is centered below the fields. At the bottom, a link for "Not a member? Create an account" is provided.

Welcome back
Please sign in to your account

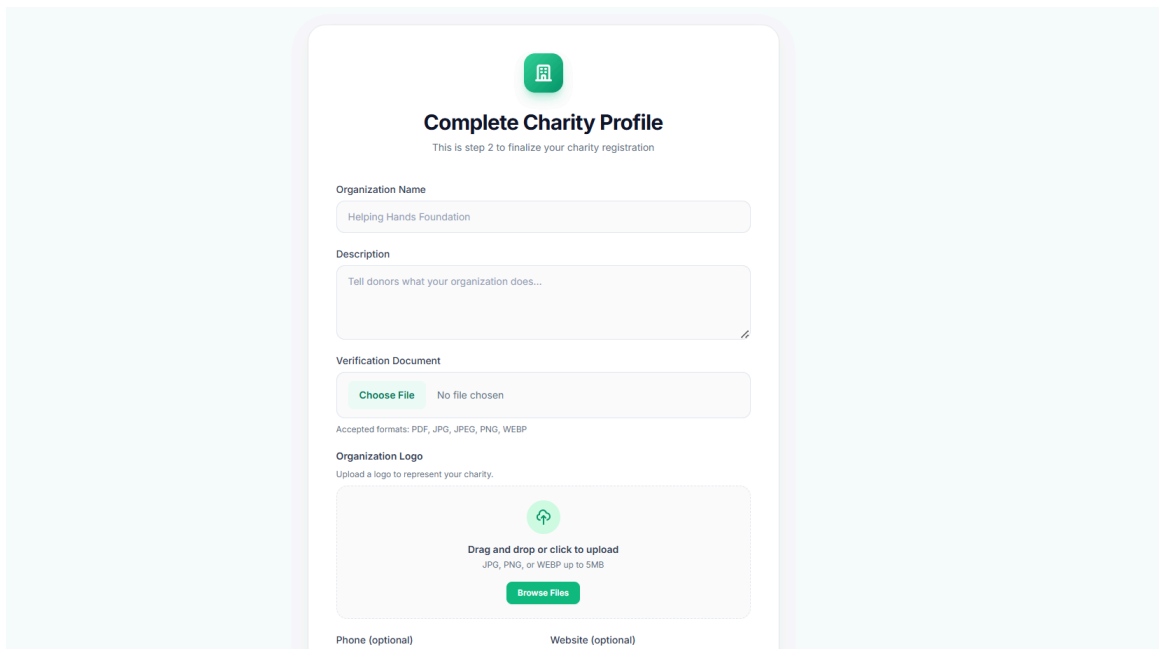
Email Address
you@example.com

Password

[Forgot password?](#)

Sign In

Not a member? [Create an account](#)



The charity registration page is a white card on a light blue background. It starts with a green icon of a building and the title "Complete Charity Profile", followed by the instruction "This is step 2 to finalize your charity registration". The form consists of several sections: "Organization Name" with the text "Helping Hands Foundation"; "Description" with a text area containing "Tell donors what your organization does..."; "Verification Document" with a "Choose File" button and the text "No file chosen"; and "Organization Logo" with a drag-and-drop area and a "Browse Files" button. Accepted formats (PDF, JPG, JPEG, PNG, WEBP) are listed. At the bottom, there are optional fields for "Phone" and "Website".

Complete Charity Profile
This is step 2 to finalize your charity registration

Organization Name
Helping Hands Foundation

Description
Tell donors what your organization does...

Verification Document
Choose File No file chosen

Accepted formats: PDF, JPG, JPEG, PNG, WEBP

Organization Logo
Upload a logo to represent your charity.

Drag and drop or click to upload
JPG, PNG, or WEBP up to 5MB

Browse Files

Phone (optional) Website (optional)

Figure 4.3 Charity Registration

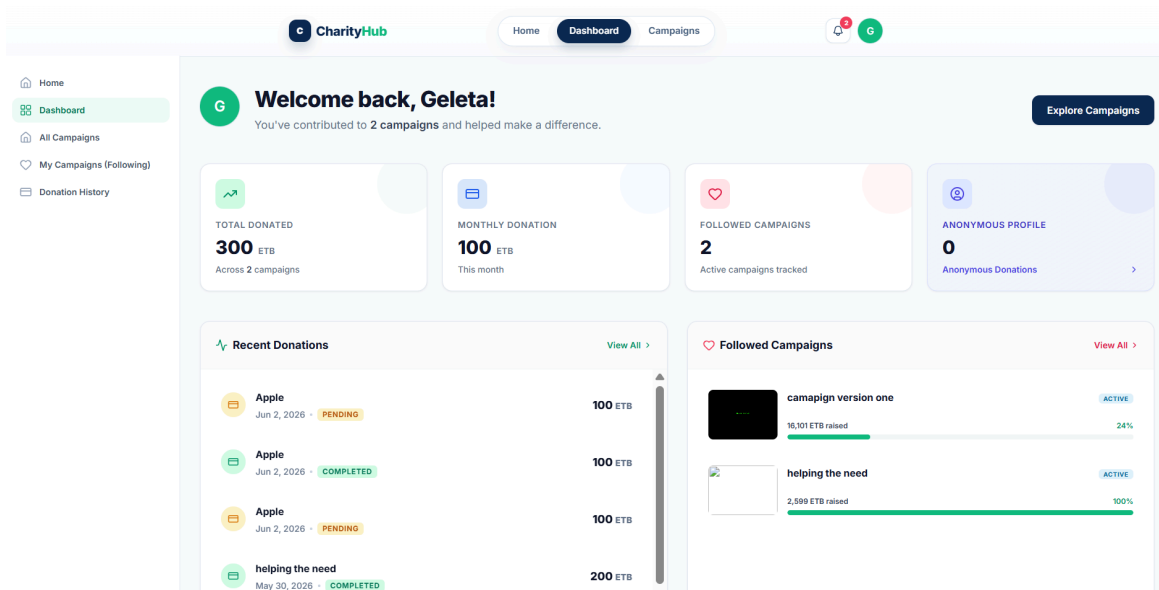


Figure 4.4 Donor Dashboard

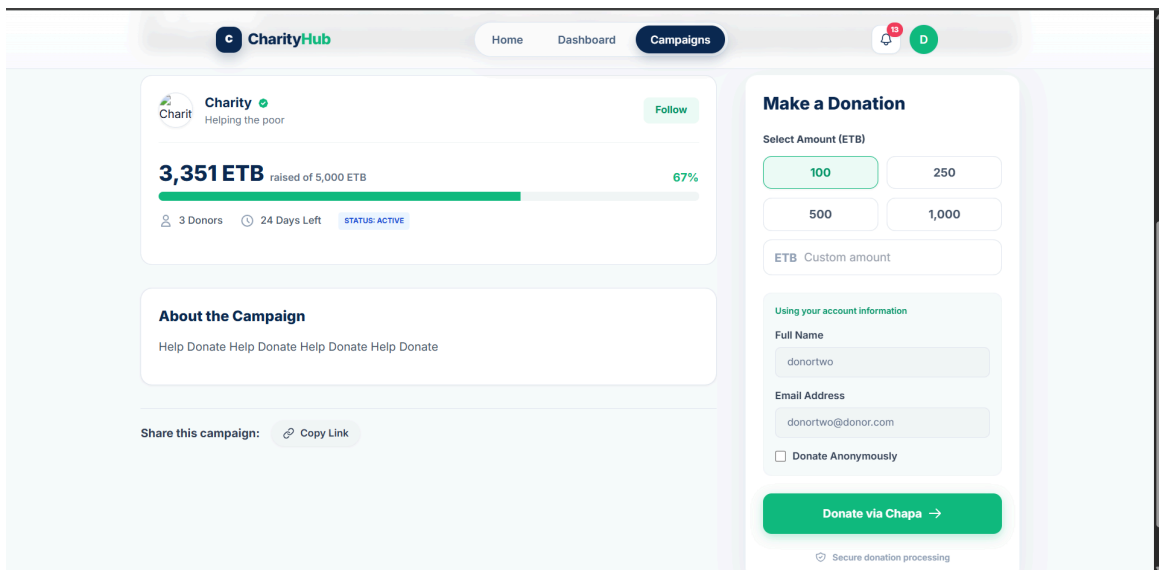


Figure 4.5 Campaign details page

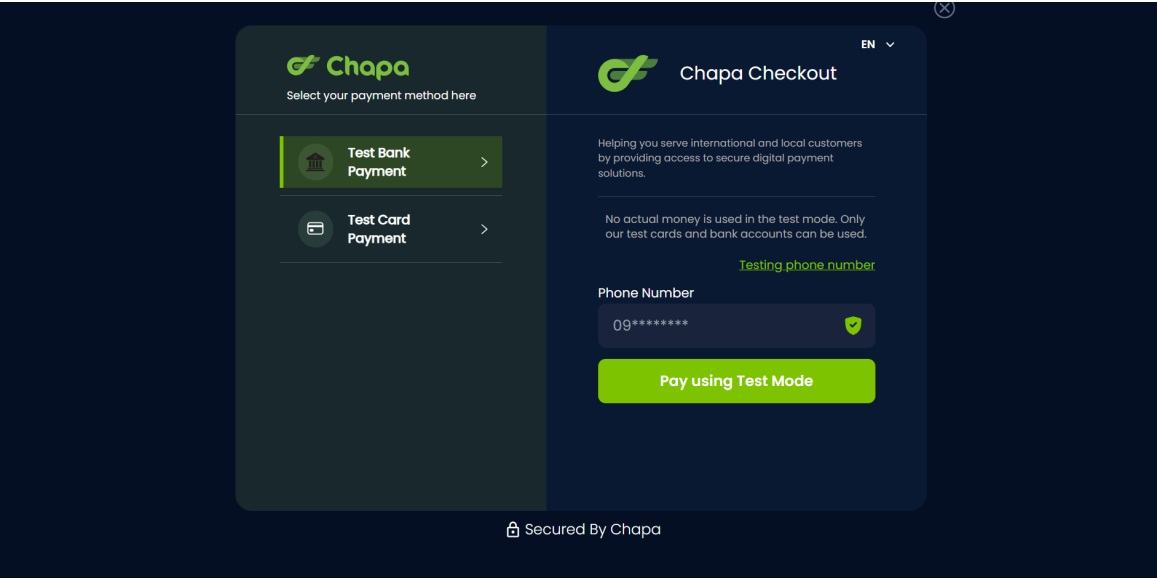


Figure 4.6 Chapa Payment

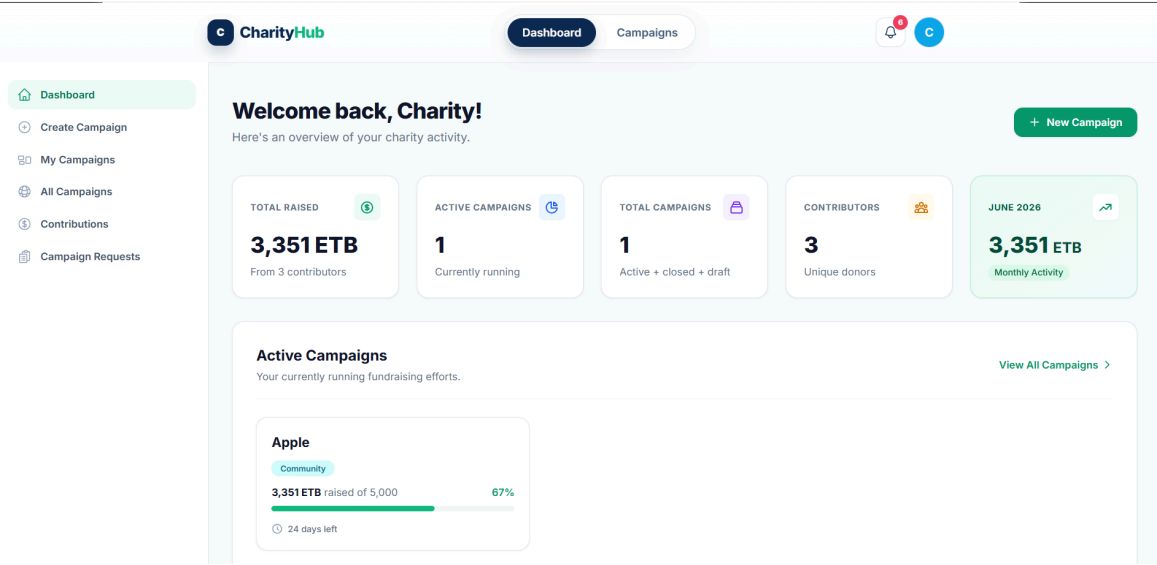


Figure 4.7 Charity Dashboard

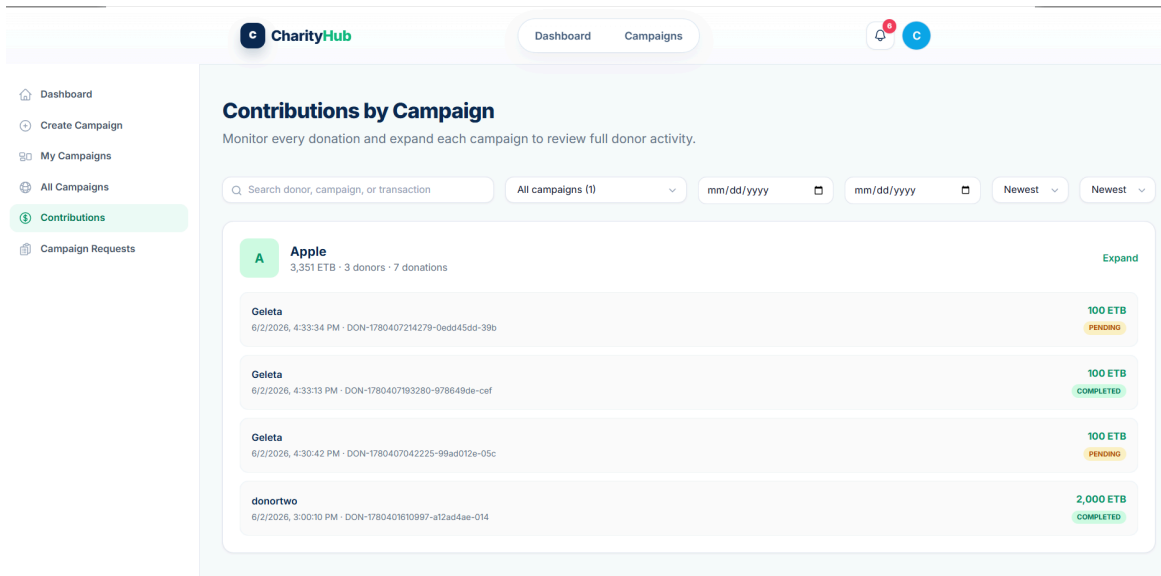


Figure 4.8 Contributions page for charity organizations

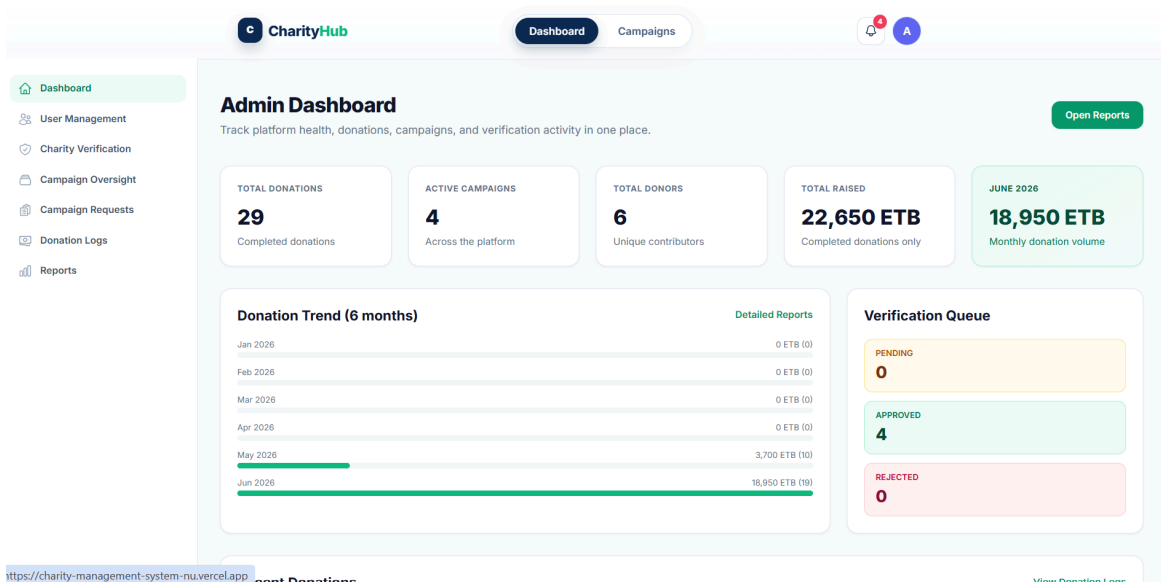


Figure 4.9 Admin Dashboard

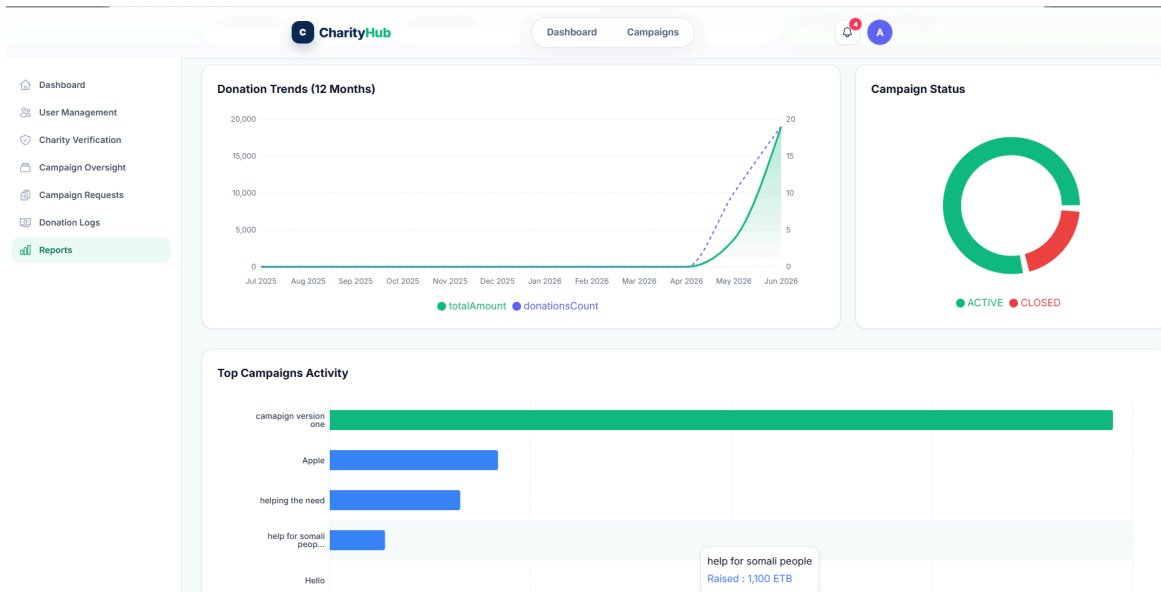


Figure 4.10 Reports Page(Admin)

The image displays the 'Charity Verification' page in the CharityHub admin interface. The page features a sidebar with navigation links: Dashboard, User Management, Charity Verification (highlighted), Campaign Oversight, Campaign Requests, Donation Logs, and Reports. The main content area includes:

- Charity Verification Header:** A section titled 'Charity Verification' with a subtitle 'Review pending registrations, and monitor approved or rejected charity profiles.'
- Filters:** A 'Pending' dropdown menu and an 'All Categories' dropdown menu. The status counts are: PENDING (1), APPROVED (4), and REJECTED (0).
- Search:** A search bar with the placeholder text 'Search organization, owner name, or email'.
- Charity Profile:** A card showing details for a charity named 'Charity' (charity@gmail.com). The profile was submitted on 6/2/2026 at 4:45:27 PM. The description is 'Charity is helping people in need.' The status is 'PENDING'. There are 'Approve' and 'Reject' buttons.
- Document:** A 'View Document' button and a document ID '0923423324'.
- Navigation:** 'Previous' and 'Next' buttons at the bottom.

Figure 4.11 Admin side charity verification page

CharityHub

Dashboard

Campaigns

4

A

Dashboard

User Management

Charity Verification

Campaign Oversight

Campaign Requests

Donation Logs

Reports

User Management

Browse all registered users, filter by role, and monitor account verification.

All Roles

All Categories

14 users

Q Search name, email, or organization

Newest

Descending

14 users

USER	ROLE	VERIFICATION	JOINED
Charity charity@gmail.com Charity	CHARITY	PENDING	6/2/2026
Abe Kebe kasela@gmail.com	DONOR	N/A	6/2/2026
Charity hands charityhands@charity.com Chairty helps	CHARITY	APPROVED	6/1/2026

Figure 4.12 Admin side user management page

93

4.2 Code Snippets

To demonstrate technical integration, code snippets representing core business functions from both the backend API server and frontend React dashboard are documented in this section.

1. Login page

```
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { validateEmail } from "../utils/validation";
import { InputField } from "../components/InputField";
import { loginRequest } from "../services/auth.api";
import { getApiErrorMessage } from "../services/apiErrors";
import { useAuthStore } from "../store/authStore";
import { getPostAuthRedirectPath } from "../utils/authRouting";

export default function LoginPage() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [emailError, setEmailError] = useState<string | null>(null);
  const [submitError, setSubmitError] = useState<string | null>(null);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const navigate = useNavigate();
  const { setAuthSession } = useAuthStore();

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault();
    setSubmitError(null);

    const eError = validateEmail(email);
    setEmailError(eError);
    if (eError) return;
```

```

try {
  setIsSubmitting(true);
  const response = await loginRequest({ email, password });
  setAuthSession(response.user);
  navigate(getPostAuthRedirectPath(response.user));
} catch (error) {
  setSubmitError(getApiErrorMessage(error));
} finally {
  setIsSubmitting(false);
}
};

return (
  <div className="flex min-h-screen items-center justify-center bg-slate-50 p-4
selection:bg-emerald-100 selection:text-emerald-900">
    <div className="w-full max-w-md overflow-hidden rounded-3xl bg-white p-8
shadow-[0_8px_30px_rgb(0,0,0,0.04)] ring-1 ring-slate-900/5 sm:p-10">
      <div className="mb-10 text-center">
        <Link to="/" className="mx-auto mb-6 flex h-14 w-14 items-center justify-center
rounded-2xl bg-gradient-to-br from-emerald-400 to-emerald-600 shadow-lg
shadow-emerald-500/30 transition-transform hover:scale-105 active:scale-95">
          <svg className="h-7 w-7 text-white" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2.5}
d="M4.318 6.318a4.5 4.5 0 00 6.364L12 20.364l7.682-7.682a4.5 4.5 0
00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5 0 00-6.364 0z" />
          </svg>
        </Link>
        <h1 className="text-2xl font-bold tracking-tight text-slate-900
sm:text-3xl">Welcome back</h1>
        <p className="mt-2 text-sm text-slate-500">Please sign in to your account</p>
      </div>

```

```

<form onSubmit={handleLogin} className="space-y-5">
  <InputField id="email" type="email" label="Email Address" required
value={email} onChange={(e) => setEmail(e.target.value)} error={emailError}
placeholder="you@example.com" />

  <div>
    <div className="mb-1.5 flex items-center justify-between">
      <label htmlFor="password" className="block text-sm font-medium
text-slate-700">Password</label>
      <Link to="/forgot-password" className="text-sm font-medium
text-emerald-600 hover:text-emerald-500">Forgot password?</Link>
    </div>
    <input
      id="password"
      type="password"
      required
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      className="block w-full rounded-xl border bg-slate-50/50 px-4 py-3 text-sm
text-slate-900 transition-all placeholder:text-slate-400 focus:bg-white focus:outline-none
focus:ring-2 focus:ring-offset-1 border-slate-200 hover:border-slate-300
focus:border-emerald-500 focus:ring-emerald-500/20"
      placeholder="••••••••"
    />
  </div>

  <div className="pt-2">
    <button type="submit" disabled={isSubmitting} className="w-full rounded-xl
bg-emerald-500 px-4 py-3.5 text-sm font-semibold text-white shadow-lg
shadow-emerald-500/30 transition-all hover:bg-emerald-600

```

```

hover:shadow-emerald-500/40 focus:outline-none focus:ring-2 focus:ring-emerald-500
focus:ring-offset-2 active:scale-[0.98] disabled:pointer-events-none disabled:opacity-70">
    {isSubmitting ? "Signing in..." : "Sign In"}
</button>
</div>

    {submitError && (
        <div className="rounded-lg bg-red-50 p-3">
            <p className="text-sm font-medium text-red-600
text-center">{submitError}</p>
        </div>
    )}
</form>

<p className="mt-10 text-center text-sm text-slate-500">
    Not a member?{" "}
    <Link to="/register" className="font-semibold text-emerald-600 transition-colors
hover:text-emerald-500">
        Create an account
    </Link>
</p>
</div>
</div>
);
}

```

2. Auth controller

```

import { Request, Response } from "express";
import { asyncHandler } from "../utils/asyncHandler";
import { ApiError } from "../utils/ApiError";
import {
    registerUser, loginUser, getCurrentUser, getUserProfile, updateUserProfile,

```

```

    resetPassword, forgotPassword, verifyPasswordResetOtp, resetPasswordWithToken
  } from "../services/auth.service";
import { env } from "../utils/env";
import { uploadFile } from "../services/file.service";

const AUTH_COOKIE_NAME = "cms_auth";

const parseJwtExpiresToMs = (v: string): number => {
  const m = v.match(/^(d+)([smhd])$/i);
  if (!m) return 7 * 24 * 60 * 60 * 1000;
  const [, n, u] = m;
  const num = Number(n);
  return u === "s" ? num * 1000 :
    u === "m" ? num * 60 * 1000 :
    u === "h" ? num * 60 * 60 * 1000 :
    num * 24 * 60 * 60 * 1000;
};

const cookieOptions = () => ({
  httpOnly: true,
  secure: env.NODE_ENV === "production",
  sameSite: env.NODE_ENV === "production" ? "none" : "lax",
  maxAge: parseJwtExpiresToMs(env.JWT_EXPIRES_IN),
  path: "/",
});

/* register */
export const register = asyncHandler(async (req: Request, res: Response) => {
  const { name, email, password, role } = req.body;
  if (!name || !email || !password || !role) throw new ApiError(400, "Missing required
fields");

```

```

    const { token, user } = await registerUser({ name, email, password, role });
    res.cookie(AUTH_COOKIE_NAME, token, cookieOptions());
    res.status(201).json({ success: true, user, token });
  });

  /* login */
  export const login = asyncHandler(async (req: Request, res: Response) => {
    const { email, password } = req.body;
    if (!email || !password) throw new ApiError(400, "Missing credentials");

    const { token, user } = await loginUser({ email, password });
    res.cookie(AUTH_COOKIE_NAME, token, cookieOptions());
    res.json({ success: true, user, token });
  });

  /* logout */
  export const logout = asyncHandler(async (_req, res) => {
    res.clearCookie(AUTH_COOKIE_NAME, { ...cookieOptions(), maxAge: 0 });
    res.json({ success: true });
  });

  /* profile */
  export const getMe = asyncHandler(async (req, res) => {
    if (!req.user) throw new ApiError(401, "Unauthorized");
    const user = await getCurrentUser(req.user.id);
    res.json({ success: true, user });
  });

  export const getMyProfile = asyncHandler(async (req, res) => {
    if (!req.user) throw new ApiError(401, "Unauthorized");
    const data = await getUserProfile(req.user.id);
    res.json({ success: true, data });
  });

```



```

});

/* update profile */
export const updateMyProfile = asyncHandler(async (req, res) => {
  if (!req.user) throw new ApiError(401, "Unauthorized");

  const file = (req as any).file;
  const { name, bio, phone, removeProfileImage } = req.body;

  const profileImage =
    file ? uploadFile(file, "Profile image") :
    removeProfileImage === "true" ? null : undefined;

  const data = await updateUserProfile(req.user.id, { name, bio, phone, profileImage });
  res.json({ success: true, message: "Updated", data });
});

/* password */
export const resetUserPassword = asyncHandler(async (req, res) => {
  if (!req.user) throw new ApiError(401, "Unauthorized");

  const { oldPassword, newPassword } = req.body;
  if (!oldPassword || !newPassword) throw new ApiError(400, "Missing passwords");

  const user = await resetPassword(req.user.id, oldPassword, newPassword);
  res.json({ success: true, user });
});

/* forgot password flow */
export const forgotUserPassword = asyncHandler(async (req, res) => {
  await forgotPassword(req.body.email);
  res.json({ success: true, message: "If email exists, otp sent" });
});

```

```
});
```

```
export const verifyPasswordResetCode = asyncHandler(async (req, res) => {  
  const { email, otp } = req.body;  
  if (!/^\d{6}$/.test(otp)) throw new ApiError(400, "Invalid otp");  
  
  const { resetToken } = await verifyPasswordResetOtp(email, otp);  
  res.json({ success: true, data: { resetToken, expiresIn: "10m" } });  
});
```

```
export const resetForgottenPassword = asyncHandler(async (req, res) => {  
  const { resetToken, newPassword } = req.body;  
  await resetPasswordWithToken(resetToken, newPassword);  
  res.json({ success: true });  
});
```

3. Dashboard page for charity

```
import { useEffect, useMemo, useState } from "react";  
import { Link } from "react-router-dom";  
  
import CharitySidebar from "../../components/CharitySidebar";  
import DashboardStatCard from "../../components/dashboard/DashboardStatCard";  
import DashboardSectionCard from "../../components/dashboard/DashboardSectionCard";  
import DashboardLoadingState from  
  "../../components/dashboard/DashboardLoadingState";  
import DashboardErrorState from "../../components/dashboard/DashboardErrorState";  
  
import { getCharityDashboard } from "../../services/charityDashboard.api";  
import { getApiErrorMessage } from "../../services/apiErrors";  
import formatCurrency from "../../utils/format";  
import CategoryBadge from "../../components/CategoryBadge";
```

```

export default function CharityDashboard({ user }) {
  const [sidebarOpen, setSidebarOpen] = useState(true);
  const [dashboard, setDashboard] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  /* fetch dashboard */
  useEffect(() => {
    (async () => {
      try {
        setLoading(true);
        const res = await getCharityDashboard();
        setDashboard(res.data);
      } catch (e) {
        setError(getApiErrorMessage(e));
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  const stats = dashboard?.stats || {};
  const campaigns = dashboard?.activeCampaigns || [];
  const donations = dashboard?.recentContributions || [];

  const monthLabel = useMemo(
    () => new Date().toLocaleString(undefined, { month: "long", year: "numeric" }),
    []
  );

  if (loading) return <DashboardLoadingState />;
  if (error) return <DashboardErrorState message={error} />;

```

```

return (
  <div className="flex min-h-screen bg-slate-50">
    <CharitySidebar isOpen={sidebarOpen} onClose={() => setSidebarOpen(false)} />

    <main className="flex-1 px-6 py-10 lg:px-10">
      <header className="mb-8 flex flex-col gap-4 sm:flex-row sm:items-center
sm:justify-between">
        <div>
          <h1 className="text-2xl font-bold text-slate-900">
            welcome back, {user.name}
          </h1>
          <p className="text-sm text-slate-500">charity dashboard overview</p>
        </div>

        <Link
          to="/dashboard/create-campaign"
          className="inline-flex items-center rounded-xl bg-emerald-600 px-5 py-2.5
text-sm font-semibold text-white shadow-sm hover:bg-emerald-500 transition"
        >
          new campaign
        </Link>
      </header>

      <div className="grid grid-cols-1 gap-5 sm:grid-cols-2 lg:grid-cols-4 mb-10">
        <DashboardStatCard
          title="total raised"
          value={`$${formatCurrency(stats.totalRaised)} etb`}
          className="bg-white shadow-sm hover:shadow-md transition"
        />
        <DashboardStatCard title="active campaigns" value={stats.activeCampaigns} />
        <DashboardStatCard title="total campaigns" value={stats.totalCampaigns} />
      </div>
    </main>
  </div>
)

```

```

<DashboardStatCard
  title={monthLabel}
  value={`$${formatCurrency(stats.monthlyContributions)} etb`}
  className="bg-gradient-to-br from-emerald-50 to-white"
/>
</div>

<DashboardSectionCard
  title="active campaigns"
  className="mb-10 bg-white rounded-2xl shadow-sm p-6"
>
  {campaigns.length === 0 ? (
    <p className="text-sm text-slate-500">no active campaigns</p>
  ) : (
    <div className="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
      {campaigns.map((c) => {
        const progress = (Number(c.currentAmount) / Number(c.targetAmount || 1)) *
100;

        return (
          <Link
            key={c.id}
            to={`/${campaigns}/${c.id}`}
            className="group rounded-xl border bg-white p-4 hover:shadow-md
transition"
          >
            <h3 className="font-semibold text-slate-900
group-hover:text-emerald-600">
              {c.title}
            </h3>

            <div className="mt-2">

```

```

        <CategoryBadge category={c.category} />
      </div>

      <p className="mt-3 text-sm text-slate-600">
        {formatCurrency(c.currentAmount)} / {formatCurrency(c.targetAmount)}
      </p>

      <div className="mt-2 h-2 w-full rounded-full bg-slate-100">
        <div
          className="h-2 rounded-full bg-emerald-500 transition-all"
          style={{ width: `${progress}%` }}
        />
      </div>
    </Link>
  );
  }}}
</div>
)}
</DashboardSectionCard>

<DashboardSectionCard
  title="recent contributions"
  className="bg-white p-6 rounded-2xl shadow-sm"
>
  {donations.length === 0 ? (
    <p className="text-sm text-slate-500">no donations yet</p>
  ) : (
    <div className="space-y-4">
      {donations.map((d) => (
        <div
          key={d.id}

```

```

        className="flex items-center justify-between rounded-lg bg-slate-50 p-4"
    >
        <div>
            <p className="font-medium text-slate-900">{d.campaign?.title}</p>
            <p className="text-xs text-slate-500">
                {d.isAnonymous ? "anonymous" : d.donor?.name}
            </p>
            </div>

            <span className="font-semibold text-emerald-600">
                {formatCurrency(d.amount)} etb
            </span>
        </div>
    )}
</div>
)}
</DashboardSectionCard>
</main>
</div>
);
}

```

4. Charity Dashboard controller

```

import { Request, Response } from "express";
import { asyncHandler } from "../utils/asyncHandler";
import { ApiError } from "../utils/ApiError";
import { CampaignLocation } from "@prisma/client";
import {
    getCharityDashboardService,
    getCharityCampaignsService,
    getCharityContributionsService,
    getCharityCampaignContributionsService,

```

```

} from "../services/charityDashboard.service";

/* dashboard summary */
export const getCharityDashboard = asyncHandler(async (req: Request, res: Response)
=> {
    const data = await getCharityDashboardService(req.user!.id);
    res.json({ success: true, data });
});

/* campaigns (filtered + paginated) */
export const getCharityCampaigns = asyncHandler(async (req: Request, res: Response)
=> {
    const { page = 1, limit = 12, search, status, sortBy = "createdAt", sortOrder = "desc",
location } = req.query as any;

    if (location && !Object.values(CampaignLocation).includes(location)) {
        throw new ApiError(400, "Invalid campaign location");
    }

    const data = await getCharityCampaignsService(req.user!.id, {
        page: Number(page),
        limit: Number(limit),
        search: search ? String(search) : undefined,
        status,
        location,
        sortBy,
        sortOrder,
    });
    res.json({ success: true, data });
});

/* all contributions (global view) */

```



```

export const getCharityContributions = asyncHandler(async (req: Request, res: Response)
=> {
  const {
    page = 1,
    limit = 6,
    donationLimit = 4,
    search,
    campaignId,
    dateFrom,
    dateTo,
    sortBy = "donatedAt",
    sortOrder = "desc",
  } = req.query as any;

  const data = await getCharityContributionsService(req.user!.id, {
    page: Number(page),
    limit: Number(limit),
    donationLimit: Number(donationLimit),
    search: search ? String(search) : undefined,
    campaignId: campaignId ? Number(campaignId) : undefined,
    dateFrom,
    dateTo,
    sortBy,
    sortOrder,
  });
  res.json({ success: true, data });
});

/* single campaign contributions */
export const getCharityCampaignContributions = asyncHandler(async (req: Request, res:
Response) => {
  const campaignId = Number(req.params.campaignId);

```

```

if (!Number.isFinite(campaignId) || campaignId <= 0) {
  throw new ApiError(400, "Invalid campaign id");
}
const {
  page = 1,
  limit = 8,
  search,
  dateFrom,
  dateTo,
  sortBy = "donatedAt",
  sortOrder = "desc",
} = req.query as any;
const data = await getCharityCampaignContributionsService(req.user!.id, campaignId, {
  page: Number(page),
  limit: Number(limit),
  search,
  dateFrom,
  dateTo,
  sortBy,
  sortOrder,
});
res.json({ success: true, data });
});

```

5. TESTING

5.1. Introduction

Charitable organizations play a significant role in addressing social, economic, educational, and humanitarian challenges within society. However, traditional fundraising methods often face challenges such as limited reach, lack of transparency, inefficient record management, and difficulties in monitoring donations. Donors may also find it difficult to verify the legitimacy of organizations and track how their contributions are being utilized.

To address these challenges, the Charity Donation Management System was developed as a web-based platform that connects donors, charity organizations, and administrators within a single centralized environment. The system enables charity organizations to create and manage fundraising campaigns, allows donors to discover and support charitable causes, and provides administrators with tools to monitor donations and verify organizations. By digitizing the fundraising process, the system promotes transparency, accountability, efficiency, and accessibility for all stakeholders.

The Charity Donation Management System includes several core functionalities such as user registration and authentication, charity profile management, charity verification, campaign creation and management, donation processing, campaign following, donation history tracking, donation summary reporting, and administrative monitoring of platform activities. These features work together to provide a secure and reliable platform for managing charitable fundraising activities.

Testing is an essential phase in the software development life cycle because it helps ensure that the implemented system satisfies its functional requirements and performs as expected. Through systematic testing, defects can be identified and corrected before deployment, thereby improving software quality, reliability, usability, and user satisfaction.

This document presents the testing process conducted for the Charity Donation Management System. It describes the testing objectives, testing strategy, test cases, expected results, actual outcomes, and pass/fail criteria used to validate the functionality of the system. The document serves as evidence that the developed system has been evaluated thoroughly and that its major features operate correctly according to the specified requirements.

5.1.1 Purpose

The purpose of this Software Testing Document is to define and document the testing activities performed on the Charity Donation Management System. The document provides a structured approach for verifying that all implemented functionalities operate correctly, consistently, and in accordance with the system requirements.

Specifically, this document aims to:

- Verify that the system satisfies its functional requirements.

- Ensure that users can successfully register, authenticate, and manage their accounts.
- Validate the creation, modification, and management of fundraising campaigns by verified charity organizations.
- Confirm that donors can search for campaigns, follow campaigns, make donations, and review their donation history.
- Verify that administrators can approve or reject charity organizations, review donation logs, and monitor financial transactions.
- Ensure that proper access control mechanisms are enforced based on user roles.
- Identify defects, inconsistencies, or unexpected behavior within the system.
- Provide evidence that the system has been tested and validated before deployment.

5.2. Features to be Tested / Not to be Tested

5.2.1 Features to be Tested

Authentication and User Management

- User registration as Donor or Charity Organization.
- Charity profile completion after charity registration.
- User login and logout.
- Password reset.
- Role-based access control for Donors, Charity Organizations, and Administrators.

Campaign Management

- Create campaign.
- Edit campaign.
- Close campaign.
- View campaign details.
- View charity organization's campaigns.

Campaign Discovery

- Search campaigns using keywords.
- Filter campaigns by category, status, and creation date.
- Follow and unfollow campaigns.

Donation Management

- Make donations to campaigns.
- Anonymous donations.
- Chapa payment integration and payment verification.
- Donation history for donors.
- Donation summary for charity organizations.

- Download donation receipts.

Charity Verification

- Submission of charity verification requests.
- Viewing uploaded verification documents.
- Approval and rejection of charity organizations by administrators.

Administrative Functions

- Review donation logs.
- Monitor campaign activities.
- Manage charity verification requests.
- Manage campaigns and donations.
- View platform statistics and reports.

Validation and Security

- Form input validation.
- Authentication and authorization checks.
- Access restrictions based on user roles.
- Protection of restricted pages and APIs.

5.2.2 Features Not to be Tested

Third-Party Services

- Internal functionality of the Chapa payment gateway.
- Email service provider infrastructure and reliability.
- External cloud storage services used for document uploads.

Performance and Load Testing

- High-volume concurrent user testing.
- Stress testing under heavy system load.
- Large-scale scalability testing.

Infrastructure Testing

- Network infrastructure reliability.
- Operating system compatibility testing.
- Server hardware performance evaluation.

Security Penetration Testing

- Advanced vulnerability assessments.
- Ethical hacking and penetration testing.
- Distributed denial-of-service (DDoS) resistance testing.

Browser and Device Compatibility Testing

- Comprehensive cross-browser testing.
- Mobile device compatibility testing across all platforms.

These features were excluded due to project scope limitations, time constraints, and reliance on third-party services that are managed independently from the Charity Donation Management System.

5.3. Pass/Fail Criteria

The pass/fail criteria define the conditions used to determine whether a test case has been successfully completed. Each test case executed during the testing process is evaluated against its expected result. The outcome of the evaluation is recorded as either Pass or Fail.

5.3.1 Pass Criteria

A test case shall be marked as **Pass** if:

- The system behaves exactly as specified in the requirements.
- The actual result matches the expected result.
- All input validations operate correctly.
- The correct data is stored, retrieved, or updated in the database.
- Appropriate success messages, error messages, and notifications are displayed when required.
- Access control and authorization rules are enforced correctly.
- The feature completes its intended operation without producing unexpected errors.

5.3.2 Fail Criteria

A test case shall be marked as **Fail** if:

- The actual result differs from the expected result.
- The system produces incorrect outputs or behavior.
- Required validations are not enforced.
- Data is stored, modified, or retrieved incorrectly.
- Unauthorized users gain access to restricted functionality.
- The system crashes, freezes, or generates unexpected errors.
- Required success or error messages are not displayed appropriately.
- Any functional requirement is not implemented as specified.

5.3.3 Test Execution Result

After executing each test case, the tester records the actual outcome and compares it with the expected result. If the observed behavior satisfies all expected conditions, the test case is marked as **Pass**. Otherwise, it is marked as **Fail** and the identified issue is documented for correction and retesting.

5.4. Approach/Strategy

The testing approach adopted for the Charity Donation Management System is primarily based on **Black Box Testing**. This approach focuses on validating the functionality of the system by examining inputs and outputs without considering the internal program structure or source code. The system was tested from the perspective of end users, including Donors, Charity Organizations, and Administrators, to ensure that all functional requirements were implemented correctly.

The testing process was conducted incrementally as system features were developed. Each module was tested individually and then integrated with other modules to verify that the complete system operated as expected. The primary objective was to identify functional defects, validate business rules, verify user permissions, and ensure that all system requirements were satisfied.

5.4.1 Testing Approach

Functional Testing

Functional testing was conducted to verify that all features of the system perform according to the specified requirements. Major functionalities tested include:

- User registration and authentication.
- Password recovery and reset.
- Charity profile setup and verification.
- Campaign creation, editing, and closure.
- Campaign search and filtering.
- Donation processing and payment verification.
- Donation history and donation summaries.
- Charity verification by administrators.
- Review of donation logs and administrative activities.

Integration Testing

Integration testing was performed to verify communication between system components. Examples include:

- Frontend interaction with backend APIs.
- Backend interaction with the database.

- Integration between donation processing and campaign updates.
- Integration with the Chapa payment gateway.

Validation Testing

Input validation testing was performed to ensure that users cannot submit invalid or incomplete data. Validation checks included:

- Required field validation.
- Email format validation.
- Password validation.
- Date validation.
- Numeric amount validation.
- Role-based access validation.

Security and Authorization Testing

Security testing focused on verifying access control mechanisms and ensuring that users can only access authorized resources.

Examples include:

- Only verified charity organizations can create campaigns.
- Only donors can make donations.
- Only administrators can approve charities and view administrative functions.
- Protected routes require valid authentication tokens.

5.5 Testing Tools

Table 5.1: Testing Tools

Tool	Purpose
Postman / REST Client	API endpoint testing
Browser Developer Tools	Debugging and frontend testing

5.6 Metrics Collected

The following testing metrics were collected:

- Total number of test cases executed.
- Number of passed test cases.
- Number of failed test cases.

- Defects identified during testing.
- Defects resolved during testing.
- Functional coverage of system requirements.

5.7 Configuration Management

Configuration management was handled using Git and GitHub. Source code changes were maintained through version control, allowing team members to work on separate branches and merge tested features into the main branch after successful validation.

All testing activities were performed on controlled versions of the application to ensure consistency and reproducibility of results.

5.8 Test Environment

Hardware Configuration

Testing was performed using personal computers with:

- Intel/AMD multi-core processors.
- Minimum 8 GB RAM.
- Stable internet connection.

Software Configuration

Testing was conducted using:

- Windows operating system.
- Node.js runtime environment.
- React frontend framework.
- Express.js backend framework.
- Prisma ORM.
- MySQL database.
- Modern web browsers such as Google Chrome and Microsoft Edge.

5.9 Regression Testing

Regression testing was performed whenever new features or bug fixes were introduced into the system. Previously tested functionalities were re-executed to ensure that modifications did not introduce new defects.

Priority was given to critical functionalities such as:

- Authentication and authorization.
- Campaign management.
- Donation processing.

- Charity verification.

5.10. Test cases with specifications

Table 5.2: Test Case Specification for Login

Name: Login			
Purpose: To allow registered users (Donors, Charity Organizations, and Administrators) to access the system using their valid credentials.			
Test Data = Email Address(Valid email, Invalid email, Empty) Password Valid password, Invalid password, Empty) User Account Status (Existing account, Non-existing account) User Role (Donor, Charity Organization, Administrator)			
Input	Expected Result	Actual Output	Pass/Fail
All fields are empty	“All fields are required”	“All fields are required”	Pass
Empty email and valid password	“Email is required”	“Email is required”	Pass
Invalid email format and valid password	“Enter a valid email address”	“Enter a valid email address”	Pass
Empty password and valid email	“Password is required”	“Password is required”	Pass
Incorrect password and valid email	“Invalid email or password”	“Invalid email or password”	Pass
Unregistered email and valid password	“Invalid email or password”	“Invalid email or password”	Pass
Valid email and valid password	“Login successful”	“Login successful”	Pass
charity organization logs in with valid credentials	User is redirected to charity dashboard	User is redirected to charity dashboard	Pass
Administrator logs in with valid credentials	User is redirected to admin dashboard	User is redirected to admin dashboard	Pass
Donor logs in with valid credentials	User is redirected to donor dashboard	User is redirected to donor dashboard	Pass

Table 5.3: Test Case Specification for Reset Password

Name: Login			
Purpose: To allow users to securely reset their account password when they forget their current password and regain access to the Charity Donation Management System			
Test Data = Email Address(Valid email, Invalid email, Empty) Verification Code(Valid code, Invalid code, Expired code, Empty) New password(Valid password, Invalid password, Empty) Confirm password(Valid password, Invalid password, Empty)			
Input	Expected Result	Actual Output	Pass/Fail
User clicks “Forgot Password”	System redirects user to forgot password page	System redirects user to forgot password page	Pass
Empty email and valid password	“Email is required”	“Email is required”	Pass
Invalid email format and valid password	“Enter a valid email address”	“Enter a valid email address”	Pass
Unregistered email entered and user clicks “Send”	If an account with that email exists, a reset code has been sent.” message is displayed	If an account with that email exists, a reset code has been sent.” message is displayed	Pass
Registered email entered and user clicks “Send”	Verification code is sent to user email and success message displayed	Verification code sent successfully	Pass
Empty verification code and user clicks “Verify”	“Verification code is required” message is displayed	“Verification code is required” message is displayed	Pass
Incorrect verification code entered	“Invalid or expired OTP” message is displayed	“Invalid or expired OTP” message is displayed	Pass
User clicks “Resend” button	New verification code is sent to user email	New verification code sent successfully	Pass
Correct verification code entered and user clicks “Verify”	System displays password reset form	Password reset form displayed	Pass
Empty new password	“New password is	“New password is	Pass

field	required” message is displayed	required” message is displayed	
Empty confirm password field	“Confirm password is required” message is displayed	“Confirm password is required” message is displayed	Pass
Different new password and confirm password	“Passwords do not match” message is displayed	“Passwords do not match” message is displayed	Pass
Valid current password and valid new password entered	Password is reset successfully and user is redirected to login page	Password reset successful and user redirected to login page	Pass

Table 5.4: Test Case Specification for Register

Name: Register			
Purpose: To allow new users to create an account as a Donor or Charity Organization and gain access to the system.			
Test Data = Full Name(Valid name, Invalid name, Empty) Email Address(Valid email, Invalid email, Existing email, Empty) Password(Valid password, Invalid password, Empty) User Role(Donor, Charity Organization, Empty) Charity Profile Information (for Charity Organization)(Valid information, Invalid information, Incomplete information, Empty)			
Input	Expected Result	Actual Output	Pass/Fail
All registration fields are empty	System redirects user to forgot password page	System redirects user to forgot password page	Pass
Empty full name field	“Email is required”	“Email is required”	Pass
Invalid email format entered	“Enter a valid email address”	“Enter a valid email address”	Pass
Password shorter than required length	“Password must be at least 8 characters” message is displayed	“Password must be at least 8 characters” message is displayed	Pass
Register as Donor with valid information	Donor account is created successfully and user is redirected to homepage/dashboard	Donor account created successfully	Pass

Register as Charity with valid information	User is redirected to charity profile setup page	User redirected to charity profile setup page	Pass
Charity profile setup submitted with required information	Charity profile is completed successfully	Charity profile is completed successfully	Pass
Charity submits registration without organization name	“Organization name is required” message is displayed	“Organization name is required” message is displayed	Pass
Charity adds bank account successfully	Bank account is added to charity profile	Bank account added successfully	Pass
Charity completes registration with valid information	Charity registration is completed and pending verification status is assigned	Charity registration completed successfully	Pass
Email already exists during registration	“Email already in use” message is displayed	“Email already in use” message is displayed	Pass

Table 5.5: Test Case Specification for Create Campaign

Name: Create Campaign			
Purpose: To allow verified charity organizations to create fundraising campaigns and submit them for administrator approval.			
Test Data = Campaign Banner=(Valid image, Empty) Campaign Title =(Valid title, Invalid title, Empty) Description = (Valid description, Invalid description, Empty) Category = (Valid category, Invalid category, Empty) Fundraising Goal = (Valid amount, Invalid amount, Zero, Negative value, Empty) Start Date = (Valid date, Invalid date, Empty) End Date = (Valid date, End date before start date, Invalid date, Empty) Charity Organization Status = (Verified charity, Unverified charity)			
Input	Expected Result	Actual Output	Pass/Fail
Unverified charity organization attempts to Create Campaign.	“Please complete your charity profile before creating campaigns.” message is displayed	“Please complete your charity profile before creating campaigns.” message is displayed	Pass
Verified charity	“Email is required”	“Email is required”	Pass

organization accesses Create Campaign page			
All required fields are empty and user clicks Create Campaign	Validation messages are displayed for required fields	Validation messages displayed	Pass
Empty campaign title	“Campaign title is required” message is displayed	“Campaign title is required” message is displayed	Pass
Campaign title less than minimum length	Appropriate validation message is displayed	Validation message displayed	Pass
Empty description	“Description is required” message is displayed	“Description is required” message is displayed	Pass
Description shorter than minimum required length	Appropriate validation message is displayed	Validation message displayed	Pass
No category selected	“Category is required” message is displayed	“Category is required” message is displayed	Pass
Fundraising goal is zero or negative	“Fundraising goal must be greater than 0” message is displayed	Validation message displayed	Pass
Start date is empty	“Start date is required” message is displayed	Validation message displayed	Pass
End date is empty	“End date is required” message is displayed	Validation message displayed	Pass
End date is earlier than start date	“End date must be after the start date” message is displayed	Validation message displayed	Pass
Invalid banner image format uploaded	Invalid banner image format uploaded	Validation message displayed	Pass
Valid campaign information entered without banner image	Campaign is created successfully	Campaign created successfully	Pass
Valid campaign	Campaign is created	Campaign is created	Pass

information entered with valid banner image	successfully	successfully	
Campaign is created successfully	Campaign status set to Pending	Campaign status set to Pending	Pass

Table 5.6: Test Case Specification for Edit Campaign

Name: Edit Campaign			
Purpose: To allow charity organizations to modify the details of their existing campaigns before or during the fundraising period.			
Test Data = Campaign Banner=(Valid image, Empty) Campaign Title =(Valid title, Invalid title, Empty) Description = (Valid description, Invalid description, Empty) Category = (Valid category, Invalid category, Empty) Fundraising Goal = (Valid amount, Invalid amount, Zero, Negative value, Empty) End Date = (Valid date, End date before start date, Invalid date, Empty)			
Input	Expected Result	Actual Output	Pass/Fail
Charity organization clicks Edit campaign	Campaign information is loaded into the edit form	Campaign information loaded successfully	Pass
Campaign title is cleared and user submits	“Campaign title is required” message is displayed	Validation message displayed	Pass
Description is cleared and user submits	“Description is required” message is displayed	Validation message displayed	Pass
Fundraising goal is set to zero or negative value	“Fundraising goal must be greater than 0” message is displayed	Validation message displayed	Pass
End date is set before start date	“End date must be after the start date” message is displayed	Validation message displayed	Pass
Invalid banner image format is uploaded	“Only JPG, PNG, or WEBP files are allowed” message is displayed	Validation message displayed	Pass
User updates campaign title only and submits	Campaign title is updated successfully	Campaign updated successfully	Pass

User updates campaign description and submits	Campaign description is updated successfully	Campaign updated successfully	Pass
User updates fundraising goal and submits	Campaign fundraising goal is updated successfully	Campaign updated successfully	Pass
User updates banner image and submits	Campaign banner image is updated successfully	Campaign updated successfully	Pass
User updates multiple campaign fields and submits valid information	Campaign information is updated successfully	Campaign updated successfully	Pass
Campaign is updated successfully	Updated campaign details are displayed to the user	Updated details displayed successfully	Pass

Table 5.7: Test Case Specification for Close Campaign

Name: Close Campaign			
Purpose: To allow charity organizations to permanently close an active campaign and prevent further donations.			
Test Data = Confirmation Action =(OK, Cancel) Campaign Status =(Active, Pending, Closed) User Authentication = (Authenticated user, Unauthenticated user)			
Input	Expected Result	Actual Output	Pass/Fail
Charity organization clicks the Close button on an active campaign	Confirmation dialog is displayed	Confirmation dialog displayed successfully	Pass
User clicks Cancel	Campaign remains active and no changes are made	Campaign remains active	Pass
User clicks OK	Campaign status is changed to Closed	Campaign status changed to Closed	Pass
User attempts to close an already closed campaign	System prevents the action	Action prevented successfully	Pass
Unauthorized user attempts to close a campaign	Access is denied	Access denied successfully	Pass

Campaign is closed successfully	Campaign displays Closed status badge	Closed status displayed successfully	Pass
Campaign is closed successfully	Donation actions are disabled for the campaign	Donations disabled successfully	Pass

Table 5.8: Test Case Specification for View Campaign Details

Name: View Campaign Details			
Purpose: To allow users to view complete information about a fundraising campaign, including its description, fundraising progress, charity organization information, and donation options.			
Test Data = Campaign ID = (Valid campaign ID, Invalid campaign ID, Non-existing campaign ID) Campaign Status =(Active, Pending, Closed) User Role = (Donor, Charity Organization, Administrator)			
Input	Expected Result	Actual Output	Pass/Fail
User clicks the "View Details" button on a campaign card	User is redirected to the Campaign Details page	User redirected successfully	Pass
User opens a campaign with a valid campaign ID	Complete campaign information is displayed successfully	Campaign information displayed successfully	Pass
User opens a campaign with an invalid or non-existing campaign ID	Appropriate error message is displayed	Error message displayed successfully	Pass
User views an active campaign	Donation options are available	Donation options displayed successfully	Pass
User views a closed campaign	Campaign information is displayed and donation actions are disabled	Campaign displayed correctly and donations disabled	Pass

Table 5.9: Test Case Specification for Search Campaign

Name: Search Campaign
Purpose: To allow users to search for campaigns using keywords and find relevant fundraising campaigns quickly.

Test Data = Search Keyword = (Valid keyword, Invalid keyword, Partial keyword, Empty) Campaign Title = (Existing title, Non-existing title) Campaign Description = (Matching description keyword, Non-matching keyword) Search Result = (Matching campaigns found, No matching campaigns found)			
Input	Expected Result	Actual Output	Pass/Fail
User enters a keyword that matches one or more campaigns and clicks Search	List of matching campaigns is displayed	Matching campaigns displayed successfully	Pass
User enters a partial campaign title	Relevant campaigns are displayed	Relevant campaigns displayed successfully	Pass
User enters a keyword with no matching campaigns	"No campaigns found" message is displayed	Message displayed successfully	Pass
User submits an empty search field	All available campaigns are displayed	All campaigns displayed successfully	Pass
User searches using valid keywords	Search results are returned accurately and efficiently	Search results displayed successfully	Pass

Table 5.10: Test Case Specification for Filter Campaign

Name: Filter Campaign			
Purpose: To allow users to narrow down campaign listings based on selected criteria such as status, category, and creation date.			
Test Data = Campaign Status=(Active, Closed, Empty) Category = (Valid category, Invalid category, Empty) Date Created = (Valid date range, Invalid date range, Empty) Multiple Filters = (Status + Category, Status + Date, Category + Date) Filter Result = (Matching campaigns found, No matching campaigns found)			
Input	Expected Result	Actual Output	Pass/Fail
User selects Active status filter	Only active campaigns are displayed	Active campaigns displayed successfully	Pass
User selects Closed status filter	Only closed campaigns are displayed	Closed campaigns displayed successfully	Pass

User selects a valid campaign category	Only campaigns belonging to the selected category are displayed	Filtered campaigns displayed successfully	Pass
User selects a valid date created filter	Campaigns matching the selected creation date criteria are displayed	Filtered campaigns displayed successfully	Pass
User applies multiple filters (e.g., Status and Category)	Campaigns matching all selected filters are displayed	Filtered campaigns displayed successfully	Pass
User applies filters that produce no results	"No campaigns found" message is displayed	Message displayed successfully	Pass
User clears all filters	All available campaigns are displayed	All campaigns displayed successfully	Pass

Table 5.11: Test Case Specification for Follow Campaign

Name: Follow Campaign			
Purpose: To allow users to follow campaigns and receive updates about campaign activities and progress.			
Test Data = Follow Action (Follow, Unfollow) Campaign Status=(Active, Closed) User Authentication = (Authenticated user, Unauthenticated user) User Role = (Donor, Charity Organization, Administrator) Multiple Filters = (Status + Category, Status + Date, Category + Date) Filter Result = (Matching campaigns found, No matching campaigns found)			
Input	Expected Result	Actual Output	Pass/Fail
Logged-in Donor clicks the Follow button on a campaign	Campaign is added to the Donor's followed campaigns list	Campaign followed successfully	Pass
Donor follows a campaign successfully	Follow button changes to Following	Follow status updated successfully	Pass
Donor clicks Following (Unfollow) on a followed campaign	Campaign is removed from the user's followed campaigns list	Campaign unfollowed successfully	Pass
User views their followed campaigns list	All followed campaigns are displayed	Followed campaigns displayed successfully	Pass

User follows a campaign and refreshes the page	Follow status remains saved	Follow status retained successfully	Pass
--	-----------------------------	-------------------------------------	------

Table 5.12: Test Case Specification for Make Donation

Name: Make Donation			
Purpose: To allow donors and guest users to contribute funds to a fundraising campaign through the Chapa payment gateway.			
Test Data = Donation Amount(Valid amount, Invalid amount, Zero, Negative amount, Empty) Full Name (Valid name, Invalid email, Empty) Email Address (Valid email, Invalid email, Empty) Donate Anonymously = (Checked, Unchecked) User Authentication = (Authenticated donor, Unauthenticated user) Payment Status = (Successful payment, Failed payment) User Role = (Donor, Charity Organization, Administrator)			
Input	Expected Result	Actual Output	Pass/Fail
Unauthenticated user opens the donation form	Donation form is displayed with Full Name and Email fields	Donation form displayed successfully	Pass
Unauthenticated user enters valid information and donation amount	Donation is processed successfully through Chapa	Donation processed successfully	Pass
Logged-in donor opens the donation form	User's Full Name and Email Address are automatically displayed	User information displayed successfully	Pass
Logged-in donor selects a predefined donation amount (100, 250, 500, 1000 ETB)	Selected amount is populated in the donation amount field	Amount populated successfully	Pass
User enters a valid custom donation amount	Custom donation amount is accepted	Custom amount accepted successfully	Pass
User enters an invalid donation amount (zero or negative value)	Validation message is displayed	Validation message displayed successfully	Pass
User enables "Donate Anonymously" option	Donation is recorded as anonymous	Anonymous donation recorded successfully	Pass

User clicks "Donate via Chapa" with valid information	User is redirected to Chapa payment gateway	User redirected successfully	Pass
Payment is completed successfully through Chapa	Donation is recorded and confirmation is displayed	Donation recorded successfully	Pass
Payment fails or is cancelled	Donation is not recorded and an appropriate message is displayed	Failure message displayed successfully	Pass

Table 5.13: Test Case Specification for View Donation Summary

Name: View Donation Summary			
Purpose: To allow charity organizations to monitor donations received across their campaigns and review detailed donor contribution activities.			
Test Data = Search Input Valid donor name, Valid campaign name, Valid transaction ID, Invalid search, Empty) Date Range (Valid date range, Invalid date range, Empty) Sort Order = (Newest, Oldest) Donation Status = (Completed, Pending)			
Input	Expected Result	Actual Output	Pass/Fail
Charity organization opens the Contributions page	Donation summaries for all campaigns are displayed	Donation summaries displayed successfully	Pass
Charity organization has received donations	Campaign contribution statistics are displayed correctly	Statistics displayed successfully	Pass
User clicks Expand on a campaign	Detailed donation records for that campaign are displayed	Donation records displayed successfully	Pass
User searches by donor name	Matching donation records are displayed	Search results displayed successfully	Pass
User searches by campaign name	Matching campaign contributions are displayed	Search results displayed successfully	Pass

User filters donations by date range	Only donations within the selected date range are displayed	Filtered donations displayed successfully	Pass
User sorts donations by newest	Donation records are sorted from newest to oldest	Donations sorted successfully	Pass
Campaign contains anonymous donations	Anonymous donations are displayed without donor identity	Anonymous donations displayed correctly	Pass
No donations match the selected search or filter criteria	"No donations found" message is displayed	Message displayed successfully	Pass
Unauthorized user attempts to access the Donation Summary page	Access is denied	Access denied successfully	Pass

Table 5.14: Test Case Specification for Verify Charity Organization

Name: Verify Charity Organization			
Purpose: To allow administrators to review, approve, or reject charity organization registration requests after examining the submitted information and verification documents.			
Test Data = Charity Status (Pending, Approved, Rejected) Verification Document (Valid document, Invalid document, Missing document) Approval Action (Approve, Reject)			
Input	Expected Result	Actual Output	Pass/Fail
Administrator opens the Charity Verification page	List of pending charity verification requests is displayed	Verification requests displayed successfully	Pass
Administrator clicks View Document	Submitted verification document is displayed	Document displayed successfully	Pass
Administrator reviews a pending charity application	Charity details and submitted information are displayed	Charity information displayed successfully	Pass
Administrator clicks Approve on a pending charity	Charity status is changed from Pending to Approved	Charity approved successfully	Pass

Administrator clicks Reject on a pending charity	Charity status is changed from Pending to Rejected	Charity rejected successfully	Pass
Administrator approves a charity organization	Approved charity gains access to charity-specific features	Access granted successfully	Pass
Administrator rejects a charity organization	Rejected charity cannot access charity-specific features	Access restricted successfully	Pass
No pending charity verification requests exist	Appropriate message is displayed	Message displayed successfully	Pass

Table 5.15: Test Case Specification for Review Donation Logs

Name: Review Donation Logs			
Purpose: To allow administrators to monitor and review all donation transactions made within the system for auditing and management purposes.			
Test Data = Donation Record (Existing donation, Non-existing donation) Search Criteria (Valid document, Invalid document, Missing Donor name, Campaign name, Empty) Donation Date (Valid date, Invalid date) Donation Amount (Valid amount, Multiple amounts) Donation Status = (Completed, Pending)			
Input	Expected Result	Actual Output	Pass/Fail
Administrator opens the Donation Logs page	All donation records are displayed in a tabular format	Donation logs displayed successfully	Pass
Administrator reviews a completed donation	Donation status is displayed as Completed	Status displayed correctly	Pass
Administrator reviews a pending donation	Donation status is displayed as Pending	Status displayed correctly	Pass
Administrator searches for a donation using a transaction ID	Matching donation record is displayed	Donation record displayed successfully	Pass

Administrator searches for a donation using a donor name	Matching donation records are displayed	Search results displayed successfully	Pass
Administrator searches for a donation using a campaign name	Matching donation records are displayed	Search results displayed successfully	Pass
No donation records match the search criteria	"No donation records found" message is displayed	Message displayed successfully	Pass
No donation records exist in the system	Appropriate message is displayed	Message displayed successfully	Pass

Table 5.16: Test Case Specification for Donation History

Name: Donation History			
Purpose: To view and track previous donations made by a donor.			
Test Data = Campaign Name (Existing campaign, Non-existing campaign) Donation Amount (Valid date range, Invalid date range, Empty) Donation Date (Valid date, Invalid date) Donation Status (Completed, Pending) User Authentication (Authenticated donor, Unauthenticated user)			
Input	Expected Result	Actual Output	Pass/Fail
Donor opens the Donation History page	All donations made by the donor are displayed	Donation history displayed successfully	Pass
Donor clicks the Download receipt button	Donation receipt is downloaded successfully	Receipt downloaded successfully	Pass
Donor has made multiple donations	All donation records are displayed correctly	Donation records displayed successfully	Pass
Donor has not made any donations	"No donation history found" message is displayed	Message displayed successfully	Pass

Unauthorized user attempts to access the Donation History page	Access is denied	Access restricted successfully	Pass
--	------------------	--------------------------------	------

REFERENCES

- [1] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.
- [2] Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill Education, 2014.
- [3] Texila International Journal (2025). "Enhancing Non-profit Efficiency and Impact Through Data-Driven Strategies."
- [4] BoardEffect (2024). "Nonprofit embezzlement cases: examples to learn from."
- [5] Unified Modeling Language (UML) Specification
- [6] GoFundMe. *Online Fundraising Platform*
- [7] Charity Navigator. *Charity Accountability and Transparency*.